



Projeto e Análise de Algoritmos

Árvores: 2-3, B e B+

André Tavares da Silva
andre.silva@udesc.br

Introdução à Árvores

- Estruturas de dados tradicionais
 - Listas representam dados de maneira sequencial
 - Árvores são adequadas para representação hierárquica
 - Árvores possui uma definição recursiva
 - Cada nó da árvore forma uma subárvore
- Uma árvore é composta por
 - Um conjunto de nós
 - Um dos nós é denominado raiz
 - Cada nó pode ter múltiplos filhos
 - Os nós que não possuem filhos são chamados de folhas

Árvores 2-3

- Estrutura de árvore autobalanceada para dados classificados
 - Acesso sequencial
 - Acesso aleatório (pesquisa)
- Possui complexidade de tempo logarítmo
 - Acesso aleatório: $O(\log n)$
 - Inserção: $O(\log n)$
 - Remoção: $O(\log n)$
- Idealizada por J. E. Hopcroft em 1970
 - São árvores com ótima isometria – cada direita, centro ou esquerda contém aproximadamente a mesma quantidade de dados.

Árvores 2-3 – Características

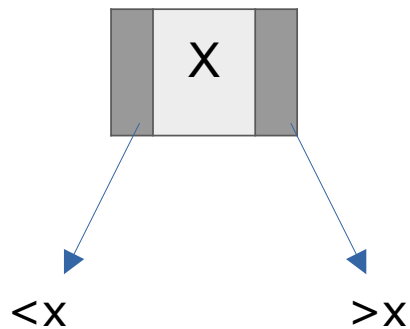
- Cada nó pode ter uma ou duas chaves;
- Todas as folhas estão no mesmo nível;
- Caso um nó não folha tiver uma única chave ela terá duas sub-árvores, como uma árvore ABB:
 - Filho à esquerda são menores que o pai
 - Filho à direita são maiores que o pai
- Caso um nó tenha duas chaves, ela possuirá três sub-árvores:
 - Filho à esquerda é menor que a menor chave
 - Filho à direita é maior que a maior chave
 - Filho central tem valor entre a maior e a menor chave
- Pode ser chamada de árvore multivias por não ser binária

Conceitos sobre Árvores 2-3

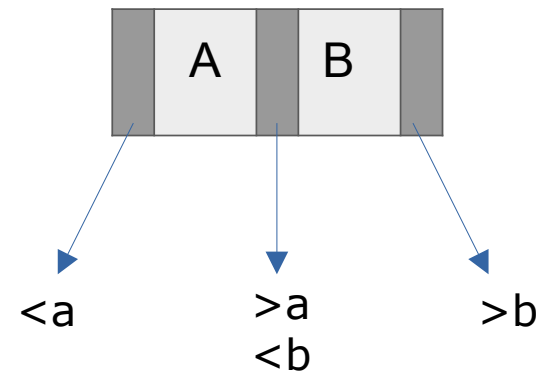
- Propriedades da árvore
 - Todos os nós folhas estão no mesmo nível
 - A árvore cresce para a raiz, diferente das árvores binárias anteriores (ABB, AVL e Rubro-Negra) que crescem sempre nas folhas Em vez de crescer da raiz para folhas (de cima para baixo), é ao contrário
 - As chaves de um nó com chave dupla são ordenados de forma crescente
- Operações em uma árvore 2-3
 - Adicionar uma chave
 - Remover uma chave
 - Localizar uma chave
 - Percorrer a árvore

Estrutura da Árvores 2-3

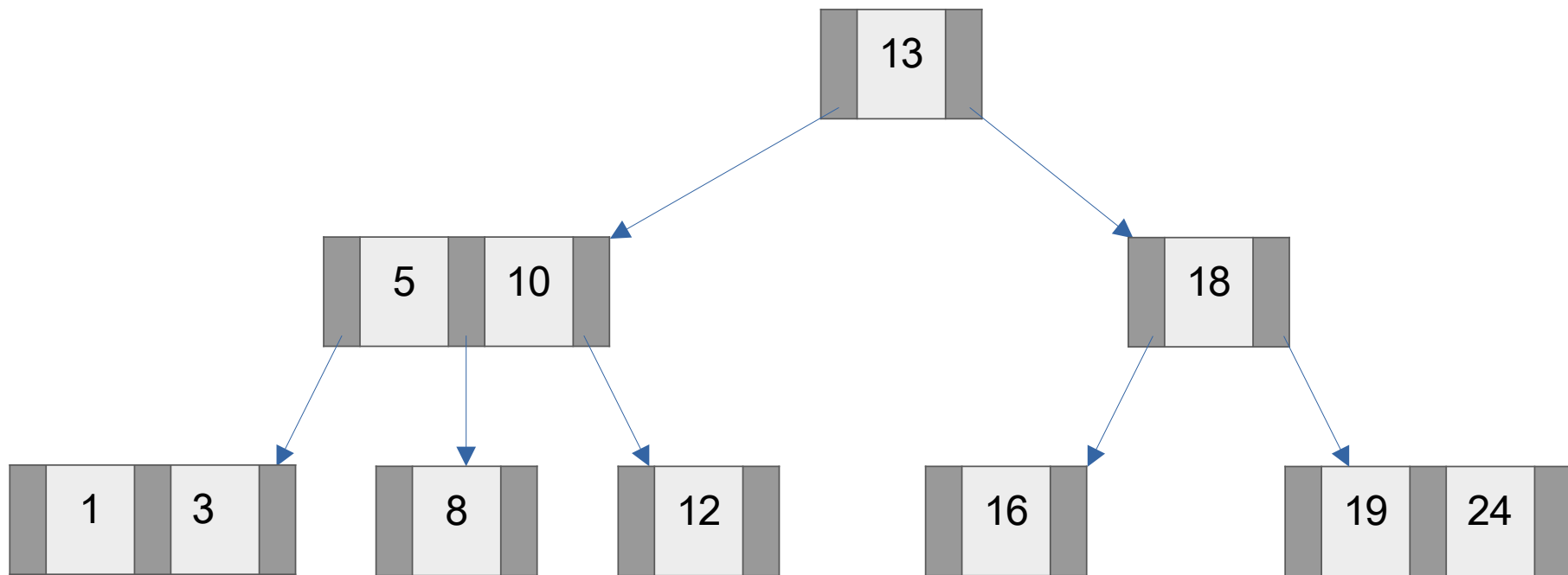
Nó com uma chave



Nó com duas chaves



Estrutura da Árvores 2-3



Estrutura da Árvores 2-3

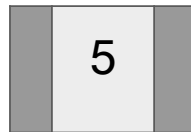
- Regra 1: Se a árvore estiver vazia:
 - Cria um nó e adiciona o valor ao nó.
- Se a árvore não estiver vazia, localiza o nó em que o valor pertence e utiliza as demais regras;
- Regra 2: Se o nó tiver apenas um valor:
 - Transforma o nó em duplo e insere o valor;
 - O balanceamento da árvore não sofre alteração.
- Regra 3: Se o nó folha tiver dois valores:
 - Chave com valor mediano é promovido para o pai;
 - Caso seja o nó raiz, cria uma nova raiz e divide o nó mantendo o balanceamento;
 - Caso o nó pai (não raiz) também tenha dois valores, a operação é repetida até encontrar um nó com apenas um valor ou até chegar na raiz.

Exemplo – árvore vazia

Inserir 5

Exemplo – árvore vazia

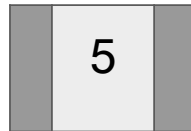
Inserir 5



Regra 1: árvore vazia

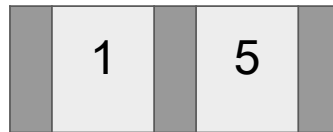
Exemplo – Inserção em nó com chave única:

Inserir 1



Exemplo – Inserção em nó com chave única:

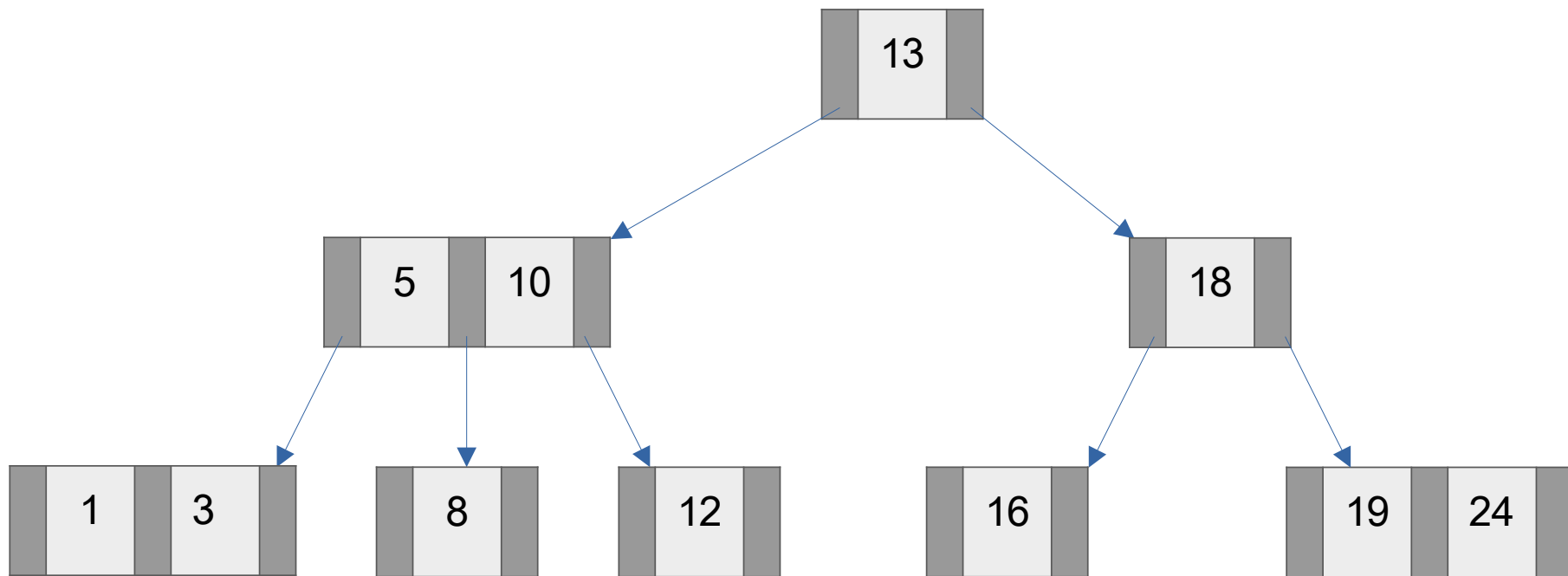
Inserir 1



Regra 2: inserção em
nó de valor único

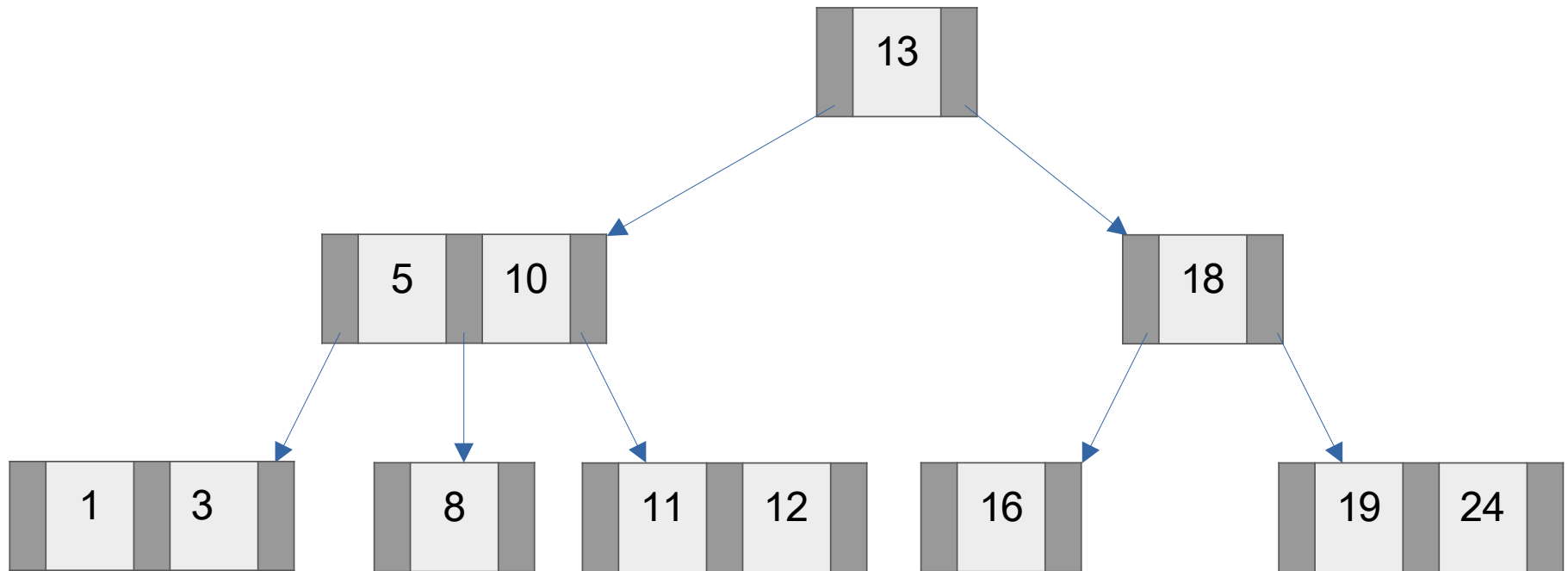
Exemplo

Inserir 11



Exemplo

Insere 11



Regra 2: inserção em
nó de valor único

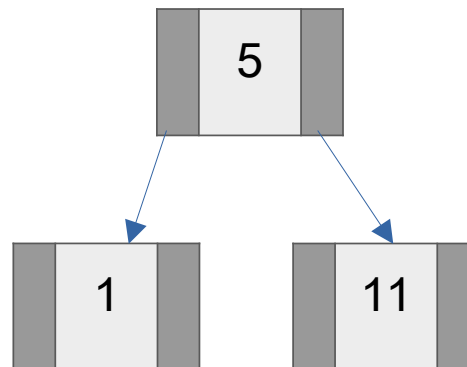
Exemplo – Inserção em nó duplo raiz:

Inserere 11



Exemplo – Inserção em nó duplo raiz:

Inserir 11

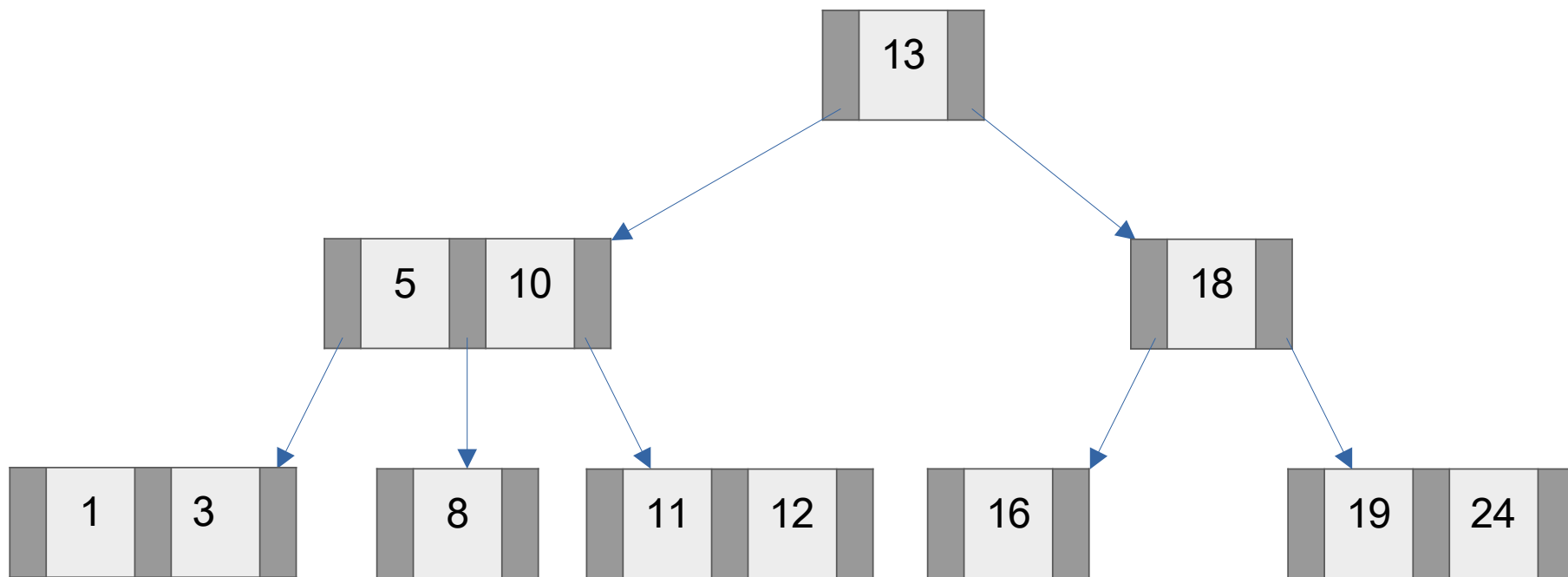


Regra 3: Se o nó folha tiver dois valores:

- Chave com valor mediano é promovido para o pai;
- Caso seja o nó raiz, cria uma nova raiz e divide o nó mantendo o balanceamento;

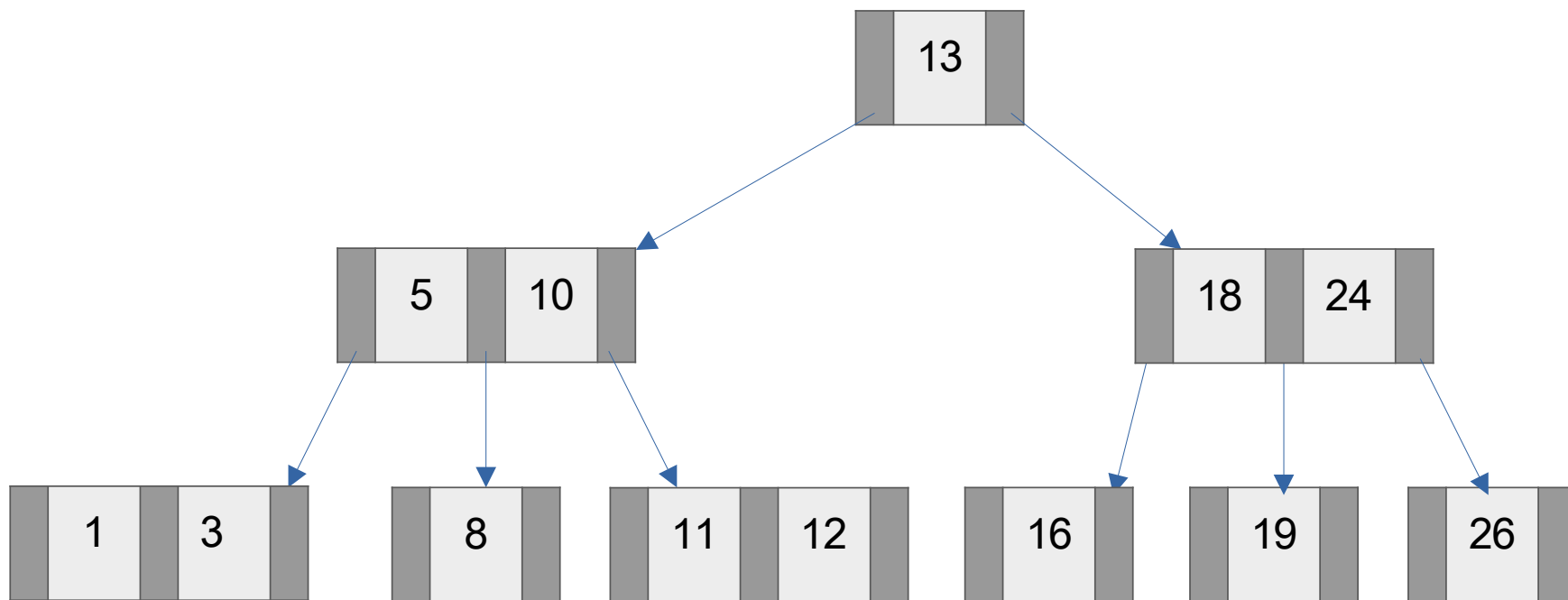
Exemplo – Inserção em nó duplo raiz:

Inserir 26



Exemplo – Inserção em nó duplo raiz:

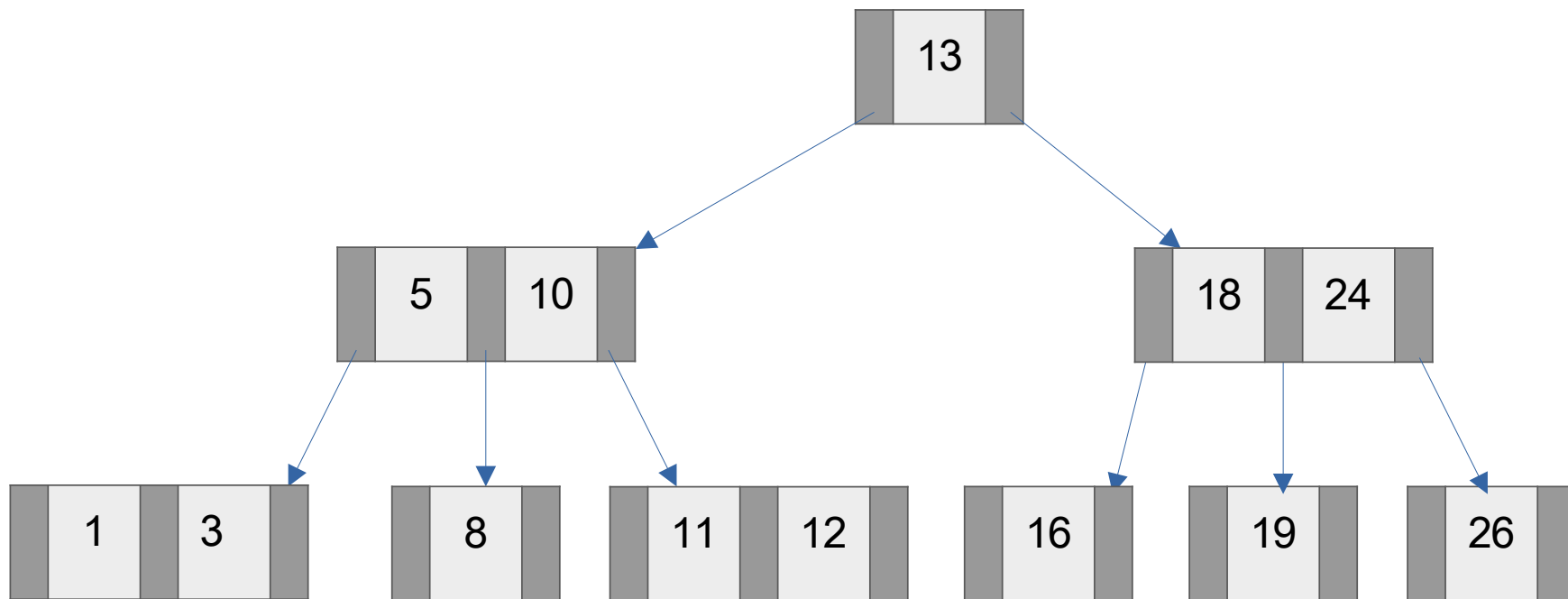
Insere 26



Regra 3: inserção em nó folha duplo
- Chave com valor mediano é promovido para o pai

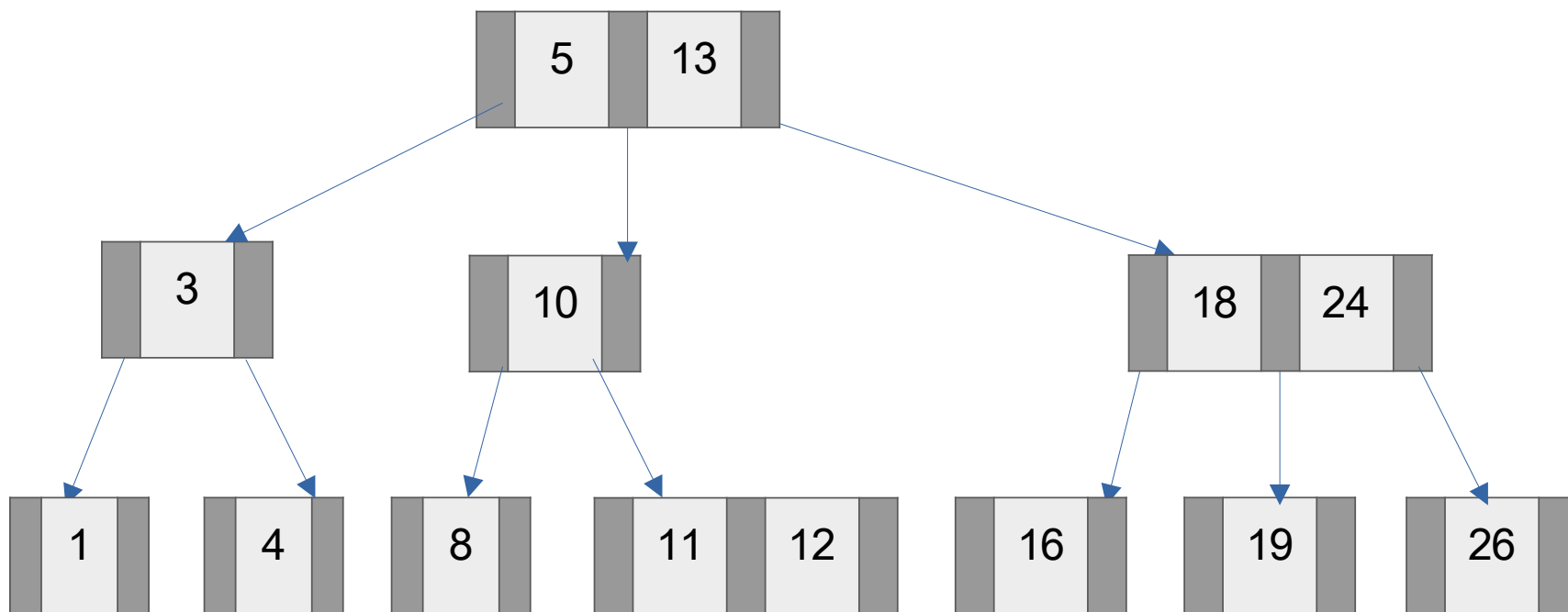
Exemplo – Inserção em nó duplo com pai duplo:

Inserere 4



Exemplo – Inserção em nó duplo com pai duplo:

Inserir 4

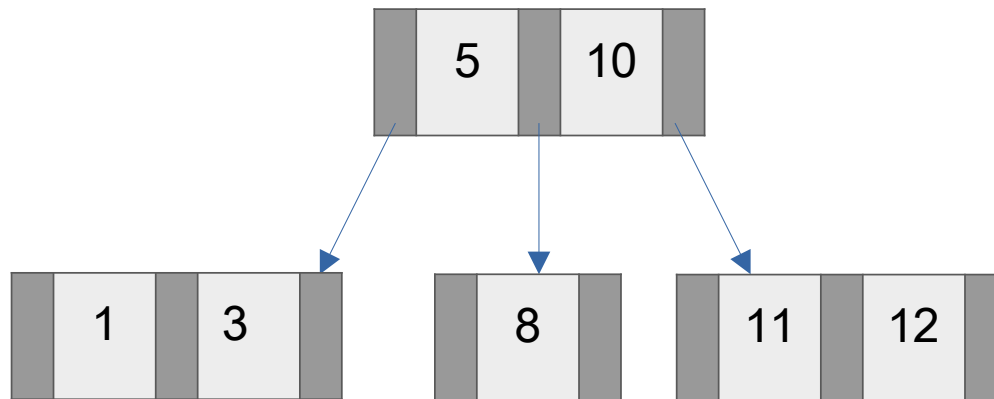


Regra 3: inserção em nó folha duplo com pai duplo

- quebra nó folha e repete operação até encontrar um nó simples
- neste caso a altura da árvore não muda

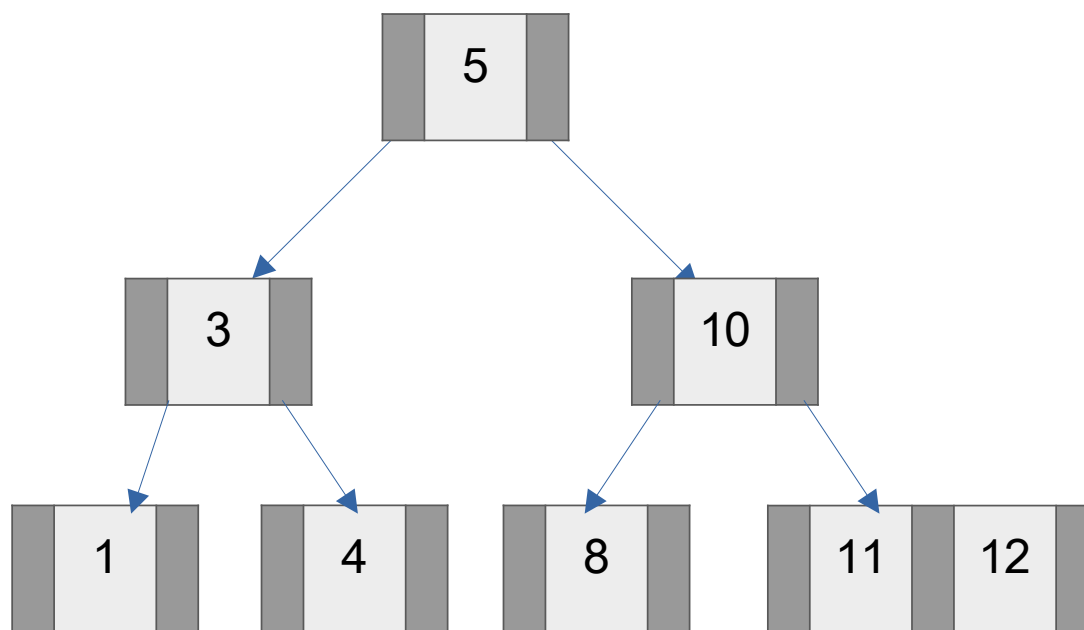
Exemplo – Inserção em nó duplo com pai duplo:

Inserir 4



Exemplo – Inserção em nó duplo com pai duplo:

Inserir 4



Regra 3: inserção em nó folha duplo com pai duplo

- quebra nó folha e repete operação até encontrar a raiz
(nenhum nó simples até a raiz)
- neste caso a altura da árvore aumenta um nível

Estrutura da árvores 2-3

- Estrutura da árvore

```
typedef struct _no {  
    int num_chaves;  
    int chave_e, chave_d;  
    struct _no *esq, *dir, *centro;  
} No;
```

```
// cria um nó com chaves e ponteiros
```

```
No *criaNo(int ch1, int ch2, int num_chaves, No *pe, No *pc, No *pd);
```

```
// verifica se o nó em questão é folha
```

```
int ehFolha(No *no);
```

```
// coloca uma nova chave e sub-arvore em nó com apenas uma chave
```

```
void adicionaChave(No *no, int ch, No *p);
```

Operações em árvores 2-3

- Localizar um nó a partir de uma chave

```
No*  localizaNo (No* no, int chave) {  
    if (no==NULL)  
        return NULL;           // não encontrou  
    if (chave == no->chave_e)  
        return no;              // é a chave esquerda  
    if ((no->num_chaves == 2) && (chave == no->chave_d))  
        return no;              // é a chave direita  
    if (chave < no->chave_e)  
        return localizaNo(no->esq, chave);  
    else if ((no->num_chaves == 1) || (chave < no->chave_d))  
        return localizaNo(no->centro, chave);  
    else  
        return localizaNo(no->dir, chave);  
}
```


Operações em árvores 2-3

• Quebra nó (split) ao inserir em chave dupla

```
No *quebraNo(No *no, int valor, int *rval, No *subarvore){
    No *aux;
    if (valor > no->chave_r) { // valor esta mais a direita
        *rval = no->chave_d; // promove a antiga maior
        aux = no->dir;
        no->dir = NULL; // elimina o terceiro filho
        no->num_chaves = 1; // atualiza o número de chaves
        return criaNo(valor, 0, 1, aux, subarvore, NULL);
    } else if (valor >= no->chave_l){ // valor esta no meio
        *rval = valor; // continua sendo promovido
        aux = no->dir;
        no->dir = NULL;
        no->num_chaves = 1;
        return criaNo(no->rkey, 0, 1, subarvore, aux, NULL);
    } else { // val esta a mais a esquerda
        *rval = no->esq;
        // primeiro cria o nó a direita...
        aux = criaNo(no->chave_r, 0, 1, no->centro, no->dir, NULL);
        no->esq = valor; // ...em seguida arruma o nó a esquerda
        no->num_chaves = 1;
        no->dir = NULL;
        no->centro = subarvore;
        return aux;
    }
}
```

Operações em árvores 2-3

- Insere novo valor na árvore (se necessário retorna novo nó e valor rval)

```
No *insere(No **no, int valor, int *rval){
    No *aux;
    int vaux, promov;

    if (*no == NULL) {        // arvore vazia
        *no = criaNo(valor, 0, 1, NULL, NULL, NULL); // cria no folha com o novo valor
        return NULL;          // nada a fazer depois
    }
    if (ehFolha(*no)){        // chegou a folha
        if ((*no)->num_chaves == 1){ // caso fácil
            adicionaChave(*no, valor, NULL);
            return NULL;
        } else {
            paux = quebraNo(*no, valor, &vaux, NULL);
            *rval = vaux;
            return aux;
        }
    } else {                  // continua a procura
        ...
    }
}
```

Operações em árvores 2-3

● Inserir novo valor na árvore (cont.)

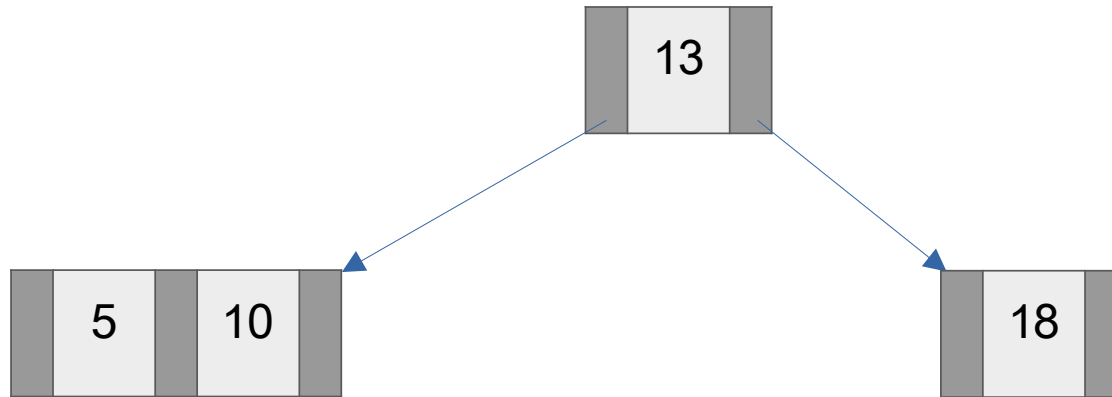
```
No *insere(No **no, int valor, int *rval){
    No *aux;
    int vaux, promov;
    if (*no == NULL) { ... // arvore vazia
    if (ehFolha(*no)){ ... // chegou a folha
    } else {
        // continua a procura
        if (valor < (*no)->chave_e)
            aux = insere( &((*no)->esq), valor, &vaux);
        else if (((*no)->num_chaves == 1) || (valor < (*no)->chave_e))
            aux = insere( &((*no)->centro), valor, &vaux);
        else
            aux = insere( &((*no)->dir), valor, &vaux);
        if (aux == NULL) // nao promoveu
            return NULL;
        else if ((*no)->num_chaves == 1){
            adicionaChave(*no, vaux, aux);
            return NULL;
        } else {
            No *aux2 = quebraNo(*no, vaux, &promov, aux);
            *rval = promov;
            return aux2;
        }
    }
}
```

Remoção em árvores 2-3

- Passo 1: Localizar a chave a ser removida;
- Passo 2: Verificar se o nó é folha. Caso não for folha, trocar por seu sucessor. A remoção somente ocorre nas folhas:
 - Se o nó folha tiver duas chaves, basta remover a chave desejada;
 - Caso o nó tiver apenas a chave a ser removida, é preciso reequilibrar os nós.
- Passo 3: Reequilíbrio. Após a remoção, podem ser necessárias operações para reequilibrar a árvore:
 - Fusão: Se um nó com uma chave for removido e o seu irmão for também um nó com uma chave, os nós podem ser fundidos em um nó com duas chaves (nó 3).
 - Rotação: Se um nó com uma chave for removido e o seu irmão for um nó com duas chaves, pode ser realizada uma rotação para redistribuir as chaves e manter o equilíbrio.

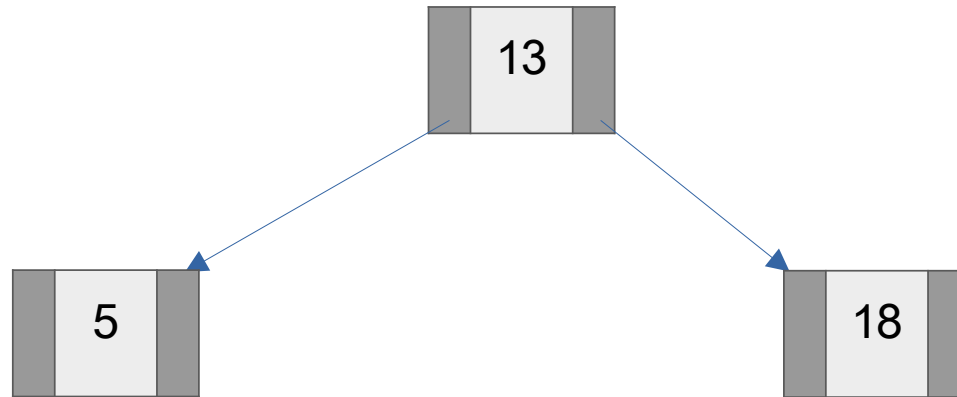
Exemplo – Remoção em nó duplo folha:

Remove 10



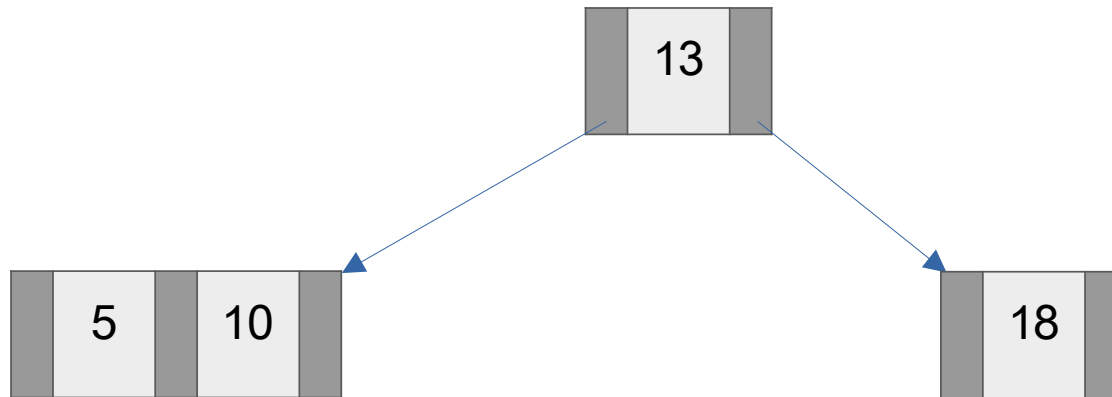
Exemplo – Remoção em nó duplo folha:

Remove 10



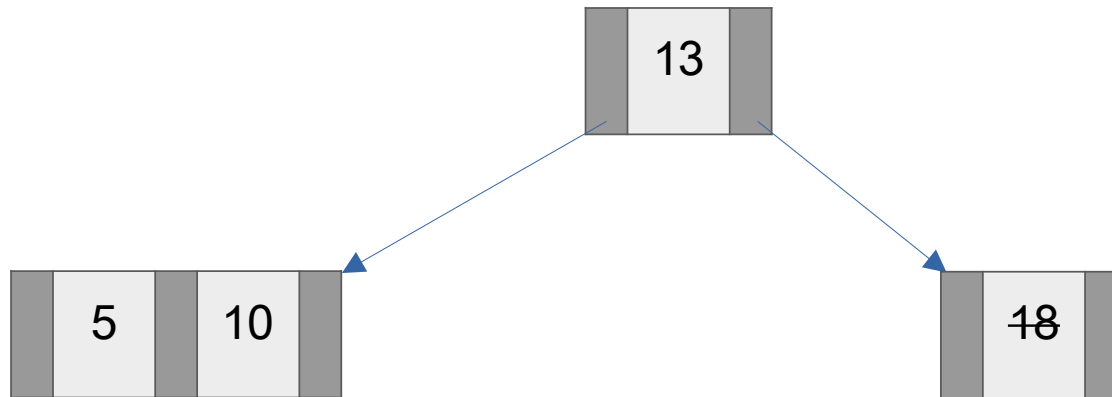
Exemplo – Remoção em nó simples folha:

Remove 18



Exemplo – Remoção em nó simples folha:

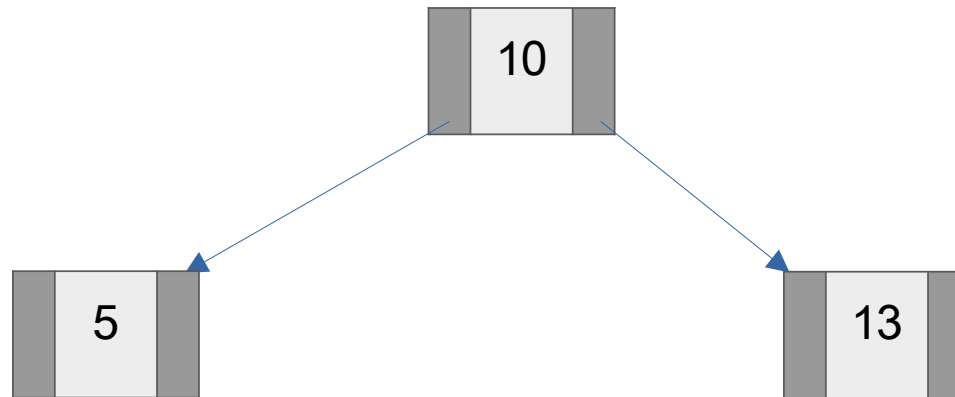
Remove 18



Rotação: Se um nó com uma chave for removido e o seu irmão for um nó com duas chaves, pode ser realizada uma rotação para redistribuir as chaves e manter o equilíbrio.

Exemplo – Remoção em nó simples folha:

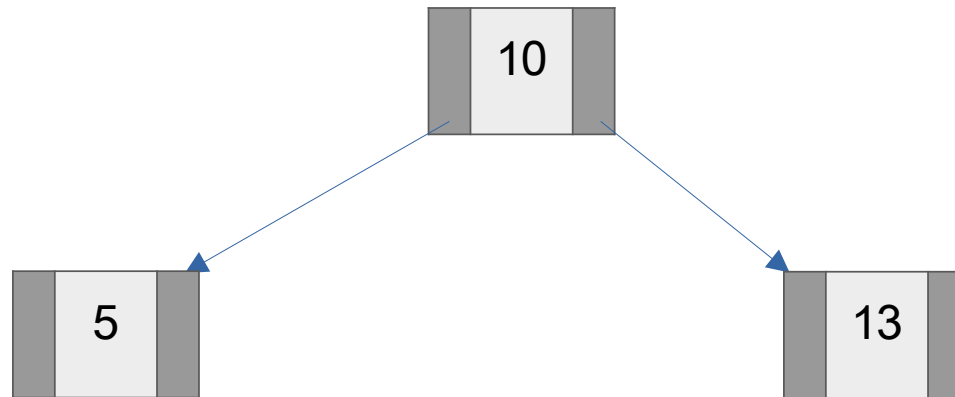
Remove 18



Rotação: Se um nó com uma chave for removido e o seu irmão for um nó com duas chaves, pode ser realizada uma rotação para redistribuir as chaves e manter o equilíbrio.

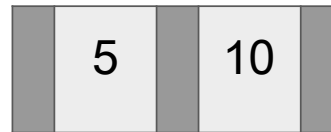
Exemplo – Remoção em nó simples folha:

Remove 13



Exemplo – Remoção em nó simples folha:

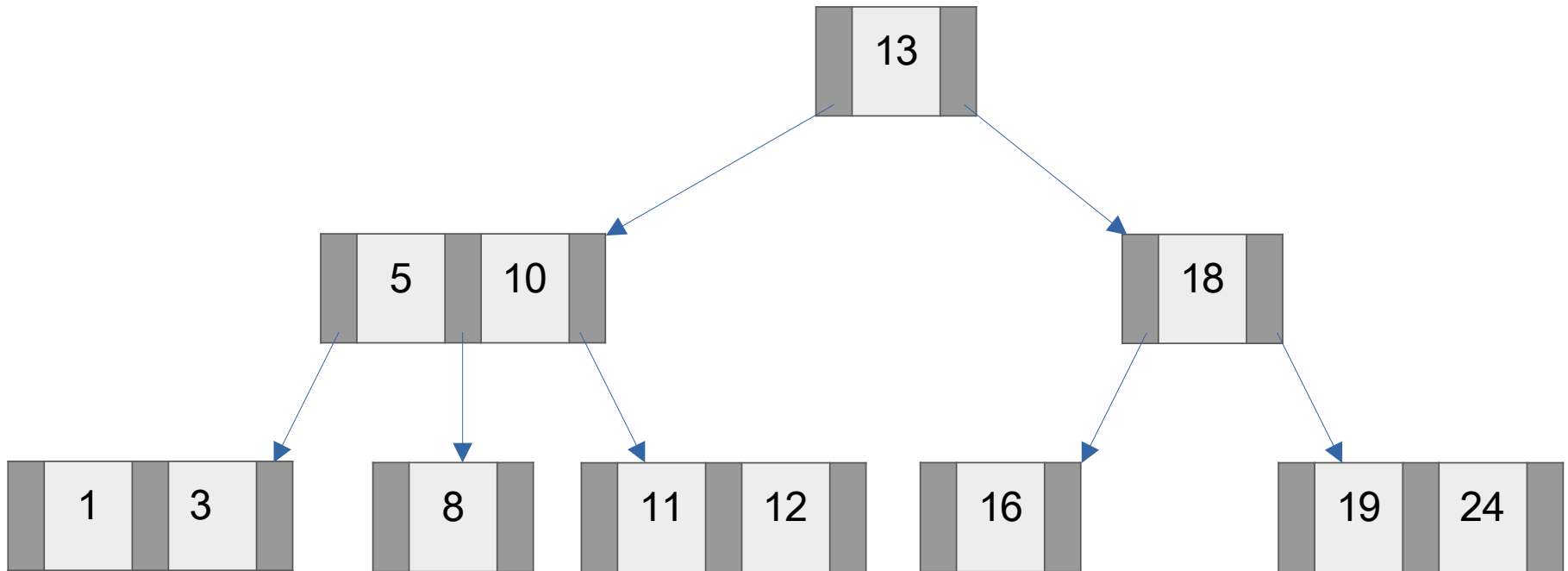
Remove 13



Fusão: Se um nó com uma chave for removido e o seu irmão for também um nó com uma chave, os nós podem ser fundidos em um nó com duas chaves (nó 3).

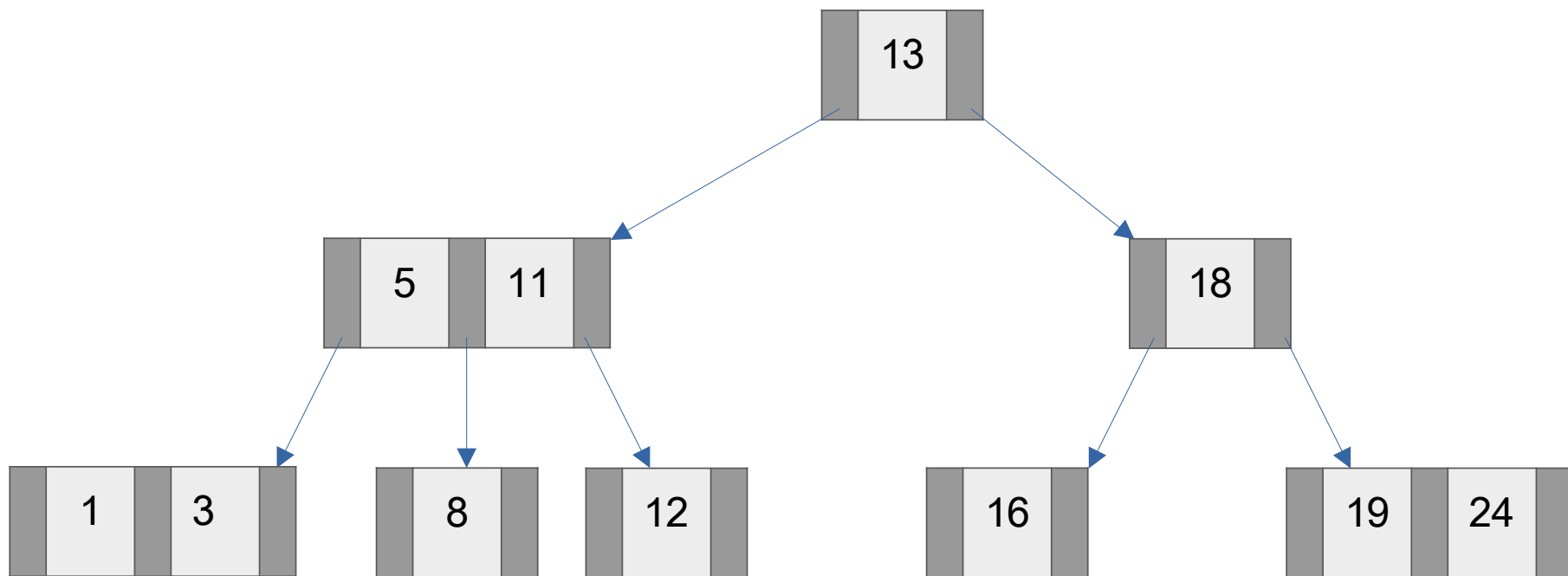
Exemplo – Remoção em duplo:

Remove 10



Exemplo – Remoção em duplo:

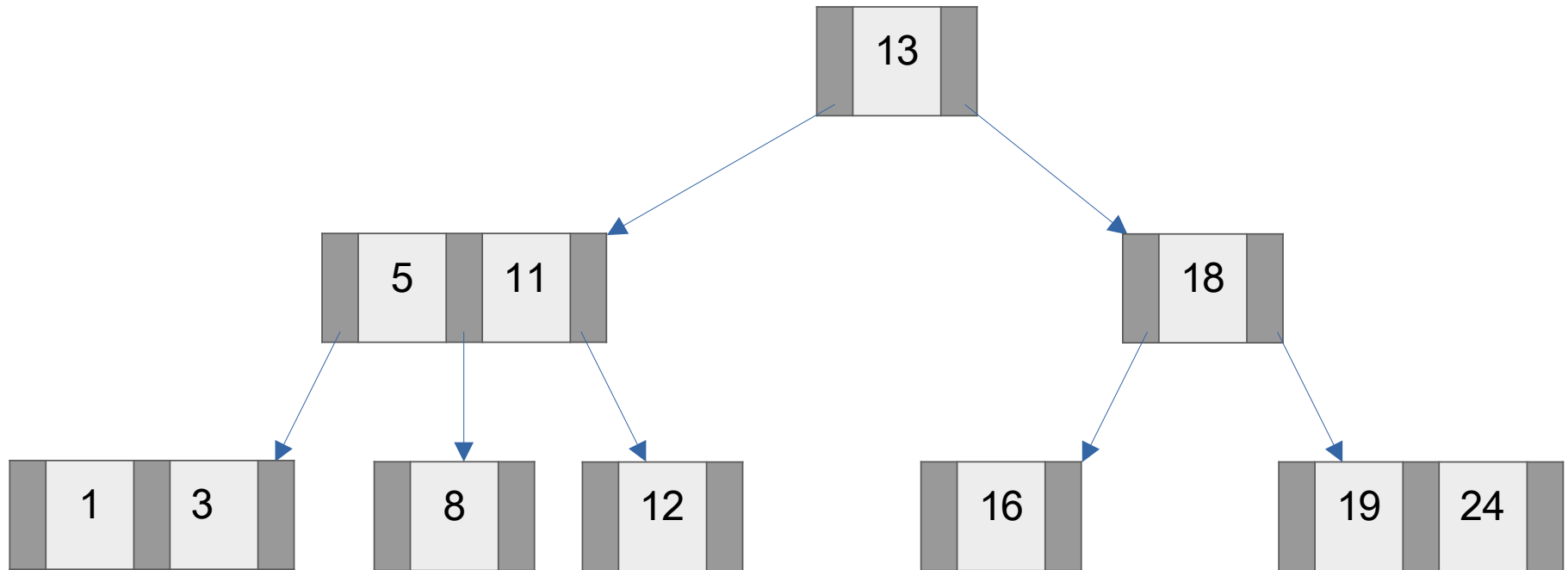
Remove 10



Caso não for folha, trocar por seu sucessor. A remoção somente ocorre nas folhas:
- Se o nó folha tiver duas chaves, basta remover a chave desejada

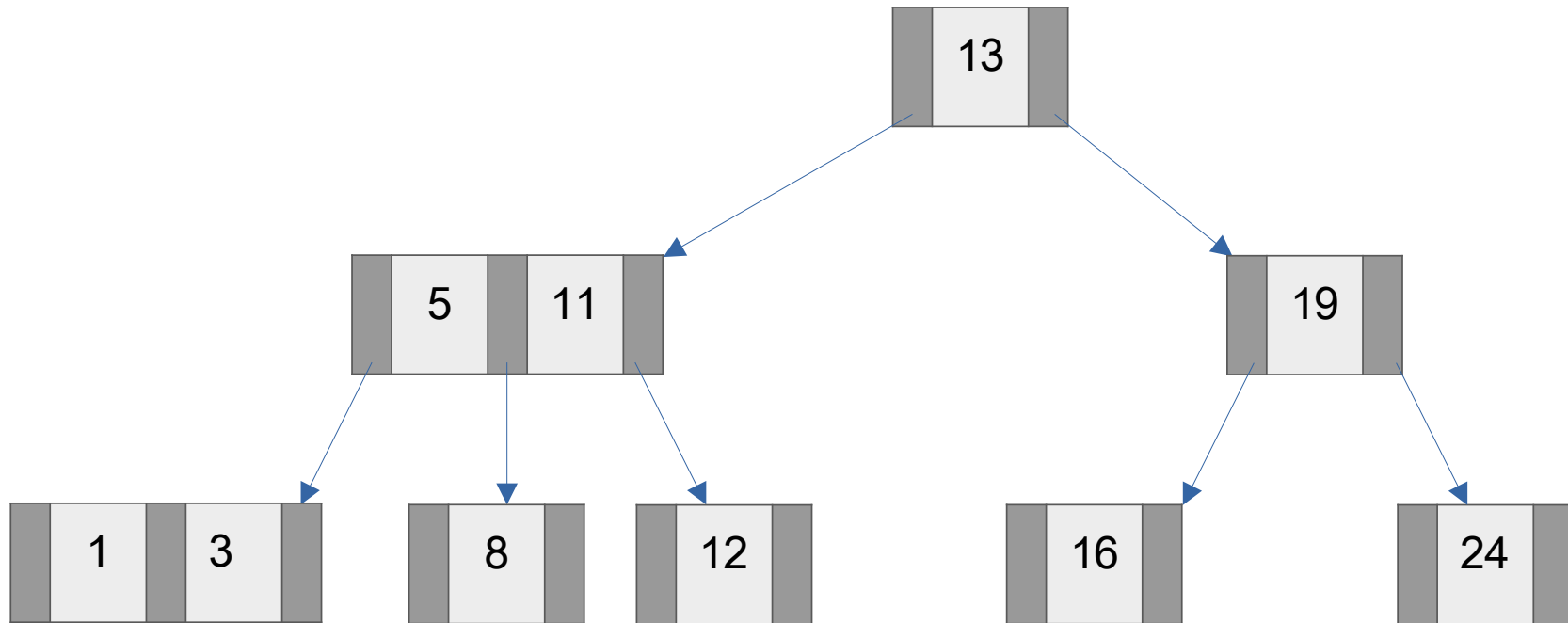
Exemplo – Remoção em simples:

Remove 18



Exemplo – Remoção em simples:

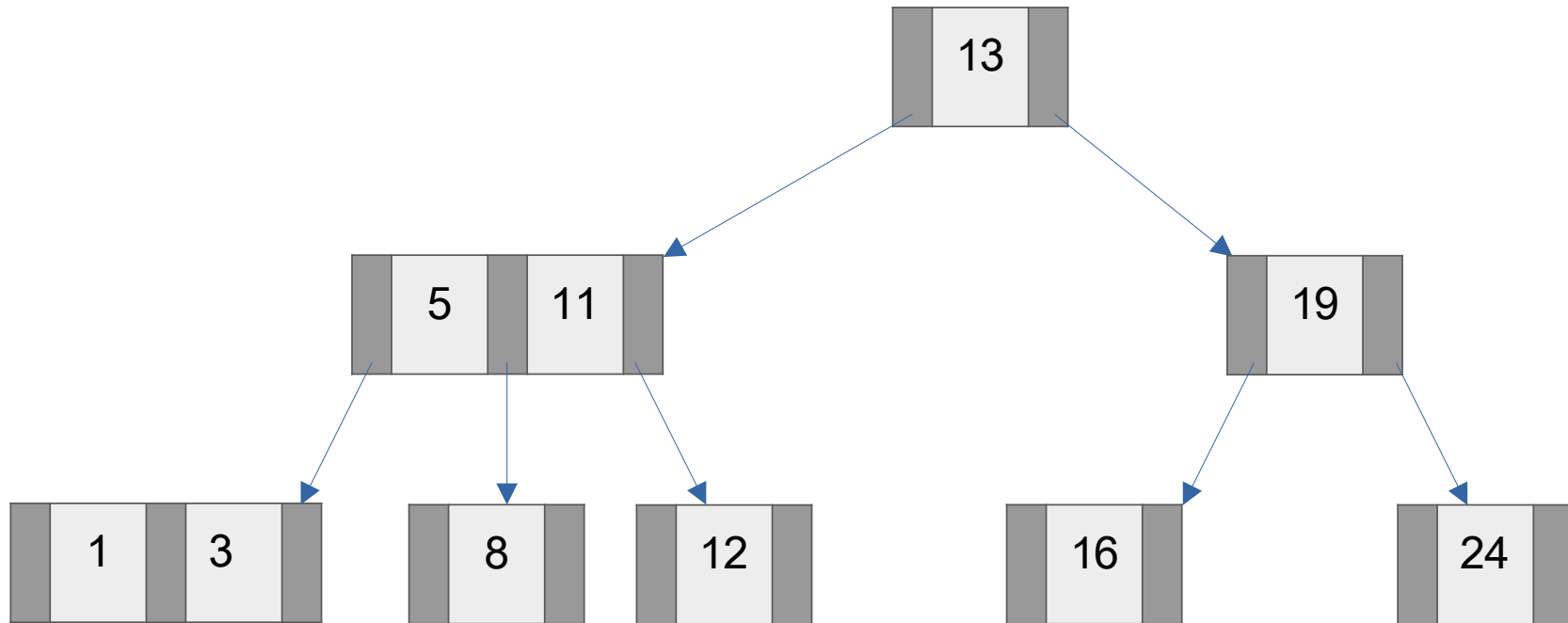
Remove 18



Caso não for folha, trocar por seu sucessor. A remoção somente ocorre nas folhas:
- Se o nó folha tiver duas chaves, basta remover a chave desejada

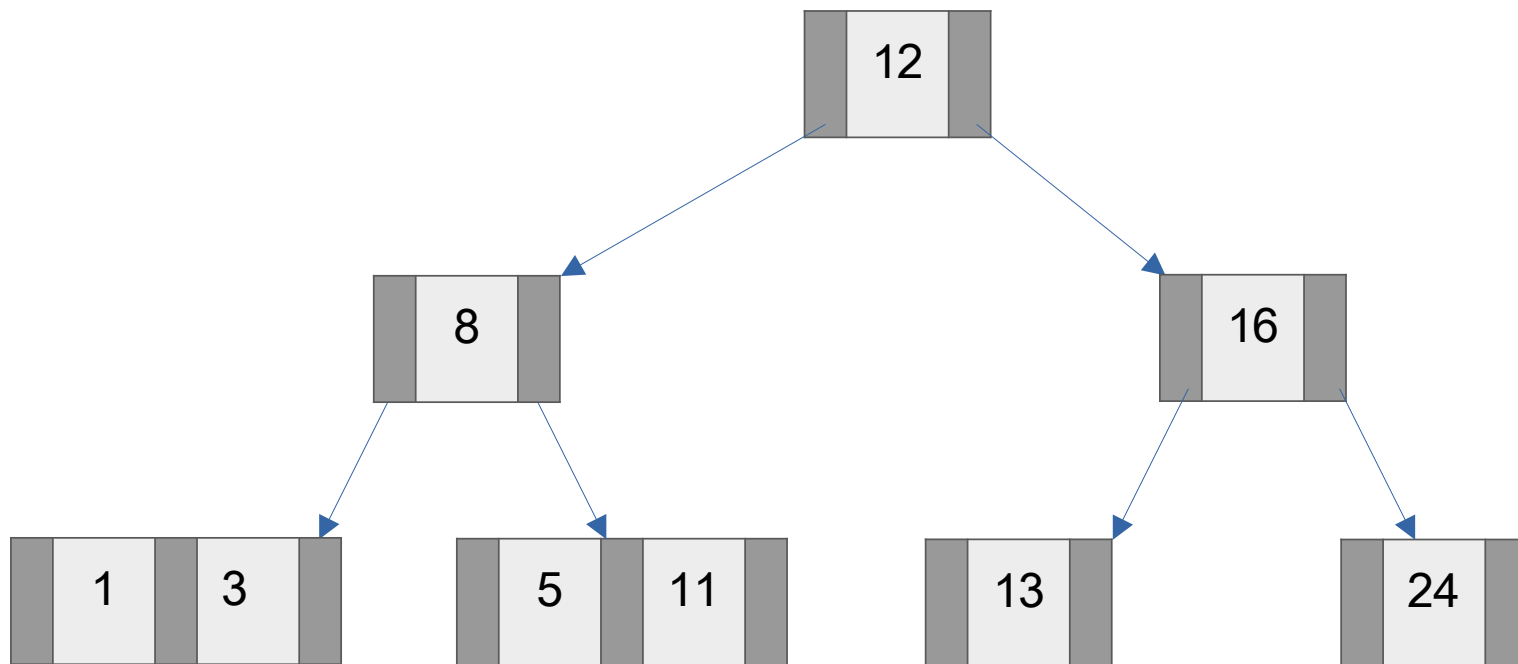
Exemplo – Remoção em simples:

Remove 19



Exemplo – Remoção em simples:

Remove 19



Após sucessivas rotações e fusões

Exercícios

1. Quantas chaves, no máximo, pode conter uma árvore 2-3 de altura 2? Qual o número mínimo de chaves em uma árvore 2-3 de altura 2? Desenhe exemplos dessas árvores.
2. Desenhe a árvore 2-3 que resulta da inserção das chaves [10, 1, 20, 30, 18, 25, 24, 11, 3], nesta ordem, em uma árvore inicialmente vazia.
3. Os dados numa árvore 2-3 são mantidos ordenados? Justifique.
4. Existe alguma relação entre a árvore Rubro-Negra e a árvore 2-3? Seria possível representar qualquer árvore Rubro-Negra em uma árvore 2-3?
5. Pesquise sobre árvores 2-3-4 (também chamada de 2-4). Verifique qual a relação entre ela e a árvore Rubro-Negra. Qual vantagem e desvantagem em relação à árvore 2-3?

Árvores B

Árvores B

- Estrutura de árvore autobalanceada para dados classificados
 - Acesso sequencial
 - Acesso aleatório (pesquisa)
- Possui complexidade de tempo logarítmo
 - Acesso aleatório: $O(\log n)$
 - Inserção: $O(\log n)$
 - Remoção: $O(\log n)$
- Idealizada por Rudolf Bayer e Edward McCreight (1971)
 - Muito utilizada atualmente em banco de dados e sistemas de arquivos

Relação entre árvores binárias e árvores B

- Árvore binária
 - Recomendado para **classificação interna**
 - Trabalha com dados em memória principal
 - Estrutura de árvore autobalanceada para dados classificados
 - Possui complexidade de tempo logarítimo
- Árvore B
 - Recomendado para **classificação externa**
 - Trabalha com dados em memória secundária
 - Objetiva reduzir o acesso aos dispositivos de memória secundária
 - É uma generalização de árvores binárias
 - Porém, os nós podem ter mais de dois filhos

Conceitos da árvores B

- Nó da árvore
 - Armazena um conjunto não vazio de chaves
 - As chaves são armazenadas em ordem crescente
 - Os nós também são chamados de páginas

- Páginas
 - Refere-se à organização de dados em blocos
 - Visa reduzir o acesso de E/S a partir técnica de paginação de dados
 - Exemplos
 - Paginação de memória principal
 - Sistema de arquivos
 - Gerenciadores de bancos de dados

Conceitos da árvores B

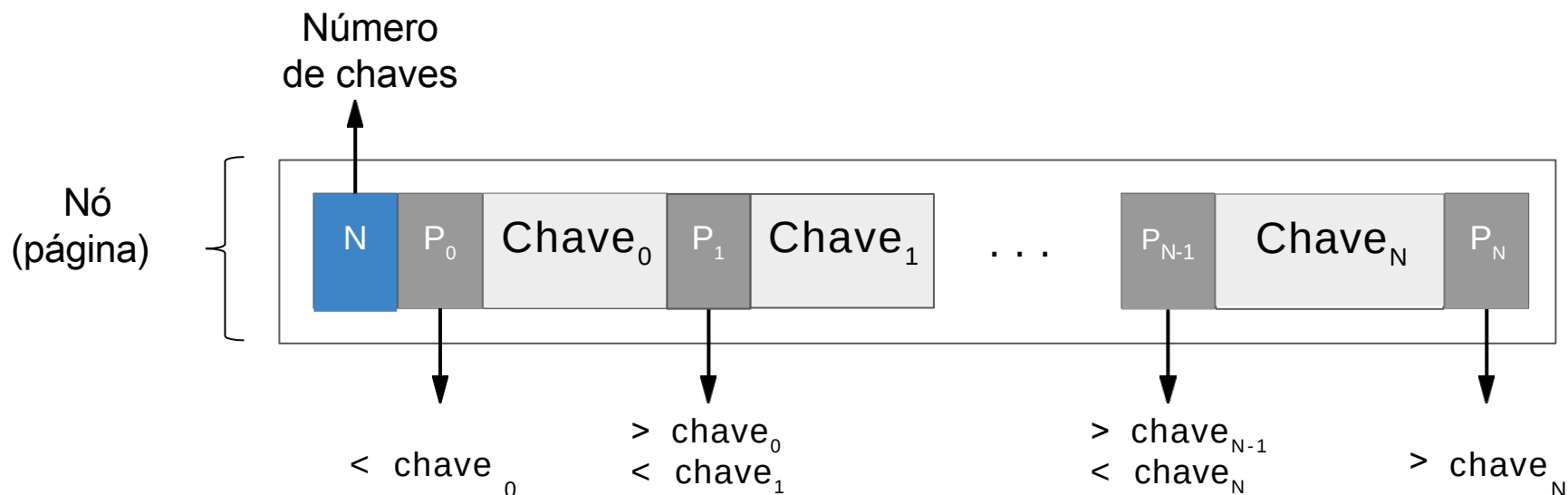
- Quantidade de chaves em um nó
 - Todo nó possui uma quantidade mínima e máxima de chaves
 - Exceto o nó raiz que não possui quantidade mínima
 - Para cada chave há um ponteiro a esquerda e direita para os nós filhos
 - A quantidade de filhos de um nó será o número de chaves mais um
- Ordem da árvore (Bayer e McCreight, 1972)
 - O número mínimo de chaves representa ordem da árvore
 - Todo nó (exceto a raiz) tem no máximo $M-1$ chaves e no mínimo $M/2$ (50%) de ocupação
 - Nós intermediários e nós folhas devem seguir esta taxa de ocupação
 - Essa característica é válida para $M > 4$, já que se com $M \leq 4$, quando dividir sempre ficará uma chave única em mais de um nó.

Conceitos da árvores B

- Propriedades da árvore
 - Todos os nós folhas estão no mesmo nível
 - A árvore cresce para a raiz, como a árvore 2-3
 - Uma árvore binária típica cresce nas folhas
 - As chaves de um nó devem estar ordenados de forma crescente
- Operações em uma árvore B
 - Adicionar uma chave
 - Remover uma chave
 - Localizar uma chave
 - Percorrer a árvore

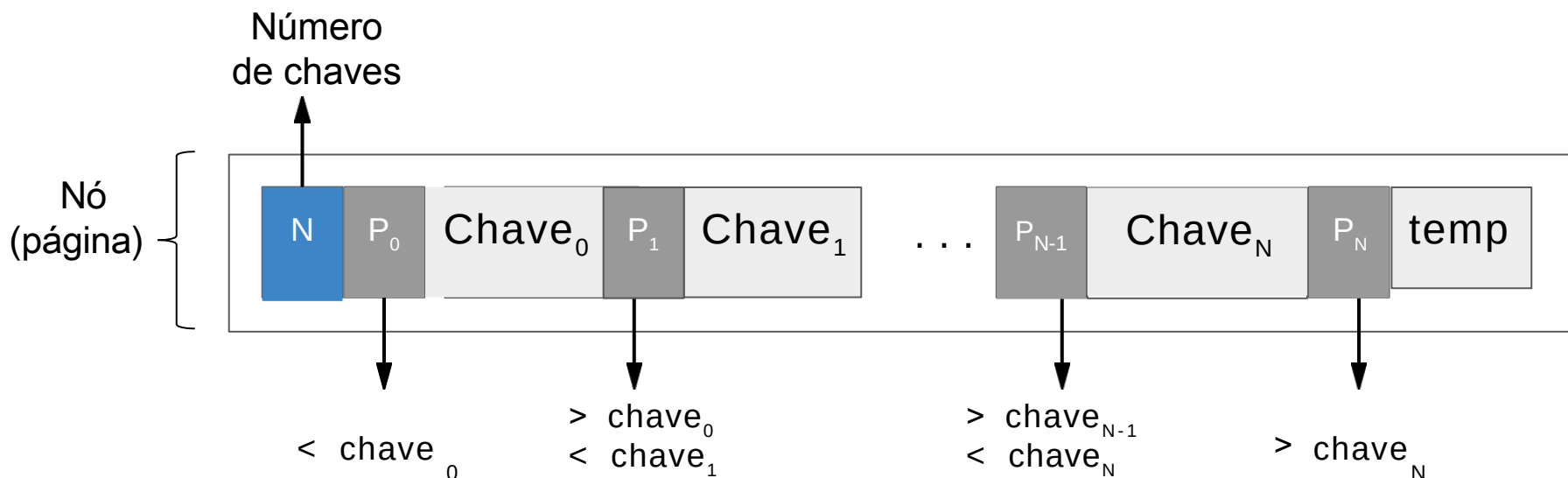
Estrutura da árvores B

- Estrutura do nó
 - Conjunto não vazio de chaves
 - Cada chave é precedida por um ponteiro para um nó filho
 - As chaves de registros são usualmente códigos numéricos



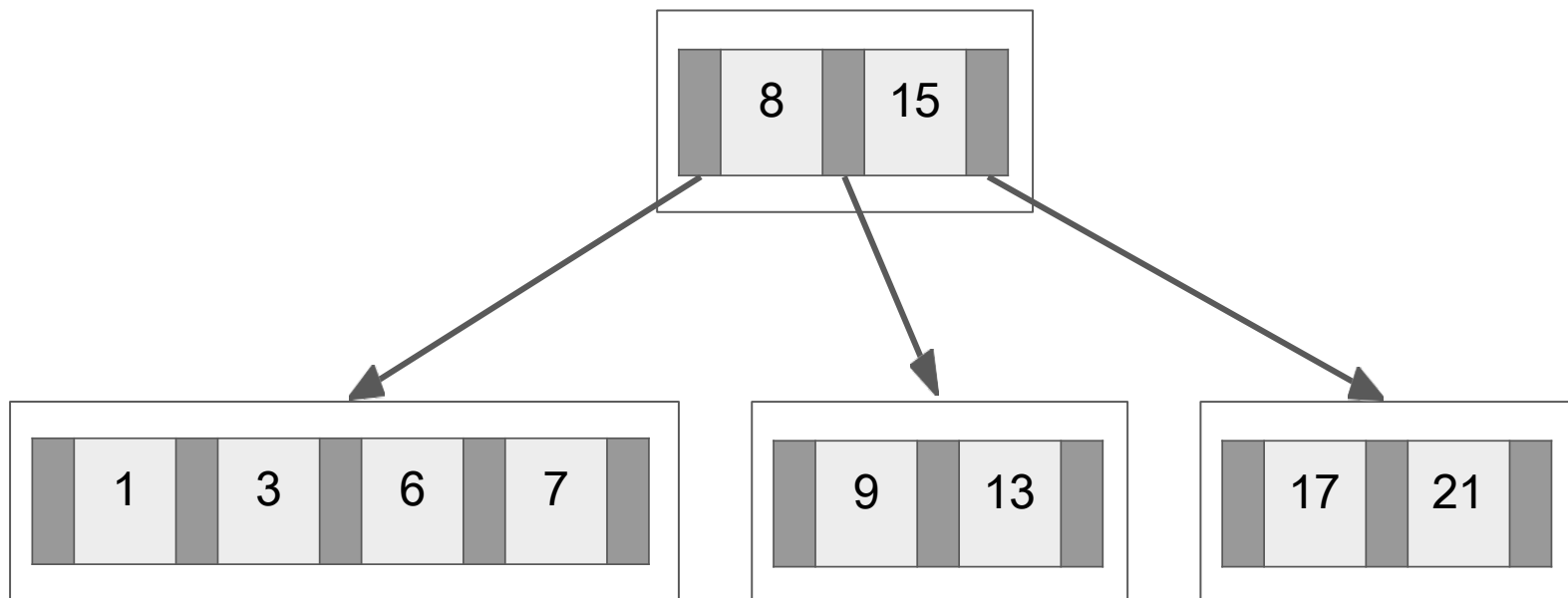
Estrutura da árvores B

- Estrutura do nó
 - Conjunto não vazio de chaves
 - Cada chave é precedida por um ponteiro para um nó filho
 - As chaves de registros são usualmente códigos numéricos
 - Pode conter um nó temporário (overflow) para uso na divisão



Estrutura da árvores B

- Estrutura da árvore
 - Nó raiz é a exceção para quantidade mínima de chaves
 - Os filhos de um nó é igual ao número de chaves mais um
 - Todos os nós folhas estão no mesmo nível da árvore



Operações em árvores B

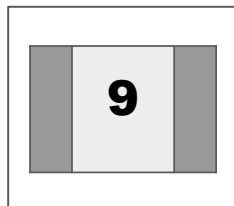
- Adicionar novas chaves
 - Inicia localizando o nó que deverá ser adicionada a chave
 - Use uma busca tradicional, porém, retorna o nó ao invés da chave
 - Insere a chave no nó localizado anteriormente
 - Deve prever os limites de chaves em um nó
 - Transbordo ou *overflow*: número de chaves excede a capacidade
 - Se ocorrer transbordo, deve ser realizado uma divisão do nó
 - Divisão ou *split*: separa as chaves contidas em um nó em dois
 - Cada nó (atual e o novo) possuirão 50% de ocupação
 - Após a divisão, a chave central da divisão deve ser promovida
 - Esta adição da chave promovida deve ser recursiva
 - Caso a divisão ocorrer na raiz, será adicionado um novo nível na árvore

Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, 8, 5, 1

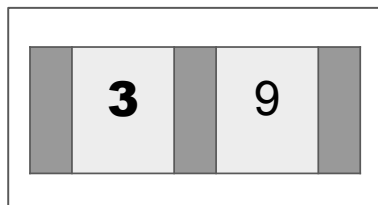
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: **9**, 3, 11, 7, 8, 5, 1



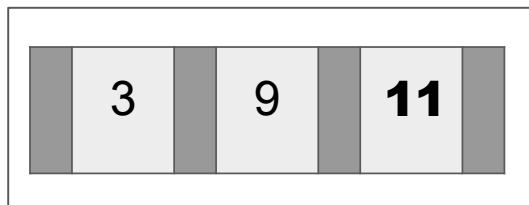
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, **3**, 11, 7, 8, 5, 1



Operações em árvores B

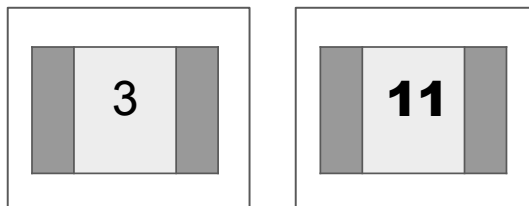
- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, **11**, 7, 8, 5, 1



Overflow

Operações em árvores B

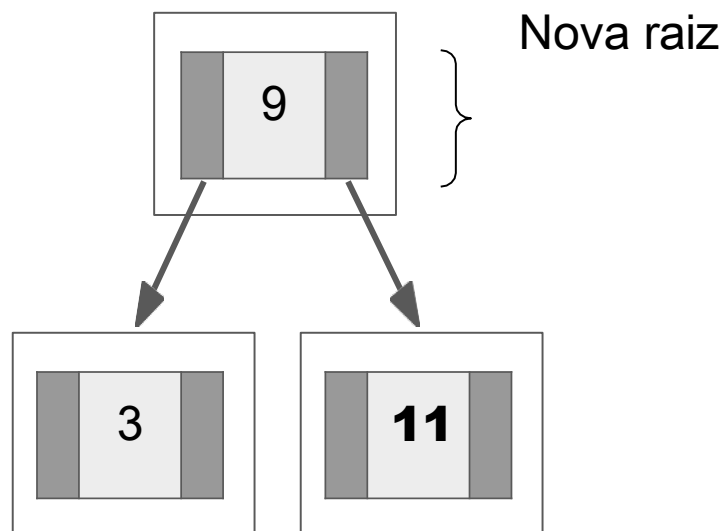
- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, **11**, 7, 8, 5, 1



Split

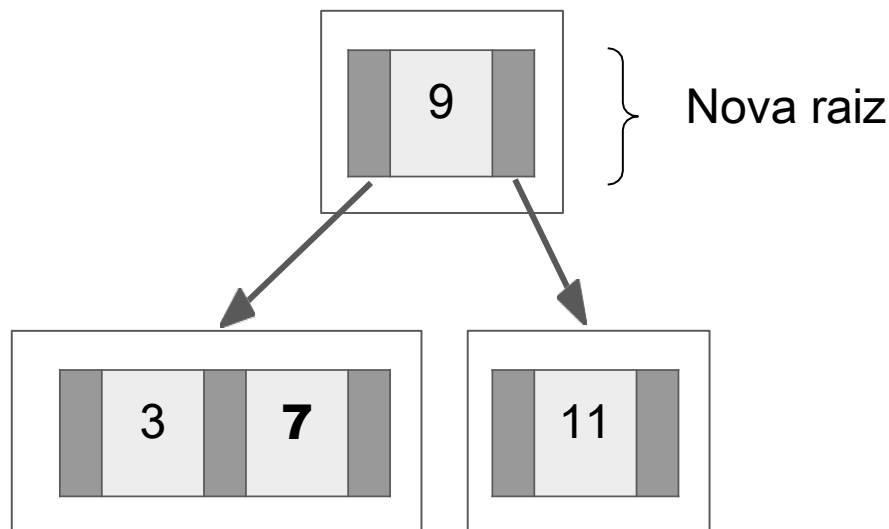
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, **11**, 7, 8, 5, 1



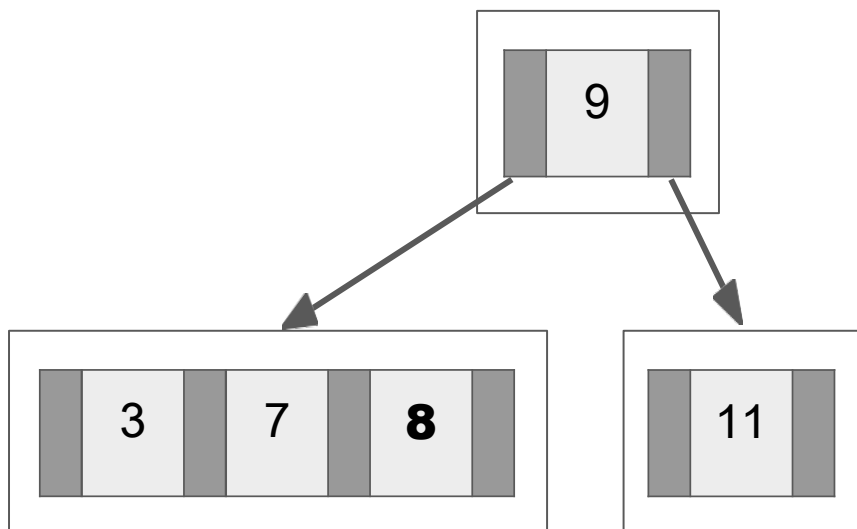
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, **7**, 8, 5, 1



Operações em árvores B

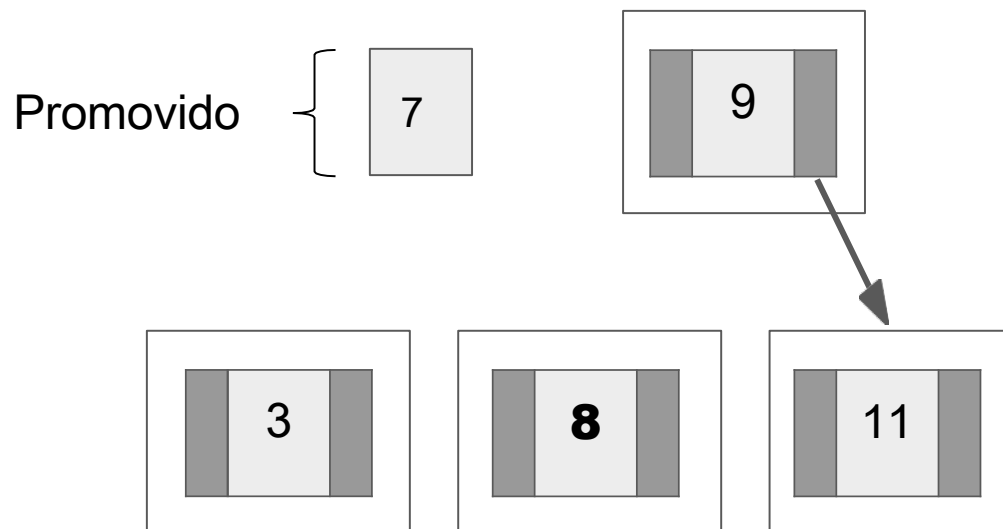
- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, **8**, 5, 1



Overflow

Operações em árvores B

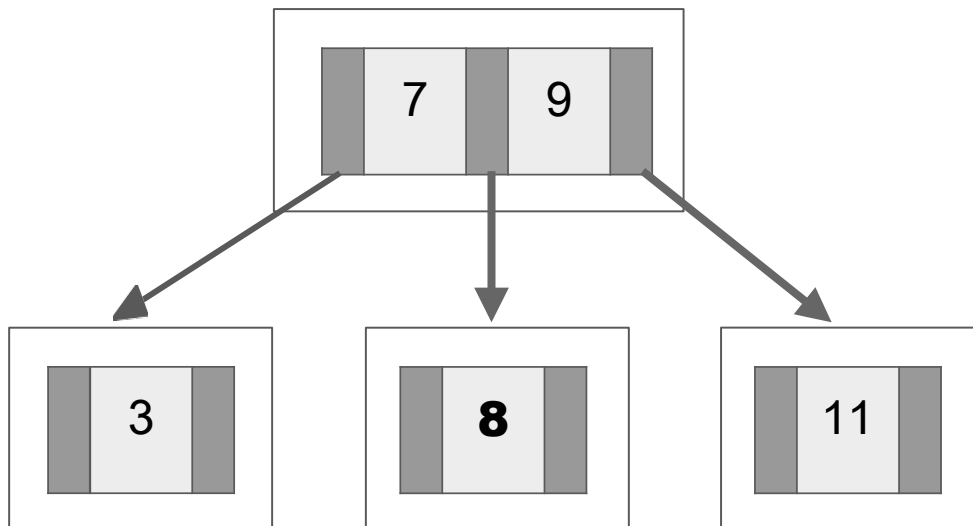
- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, **8**, 5, 1



Split

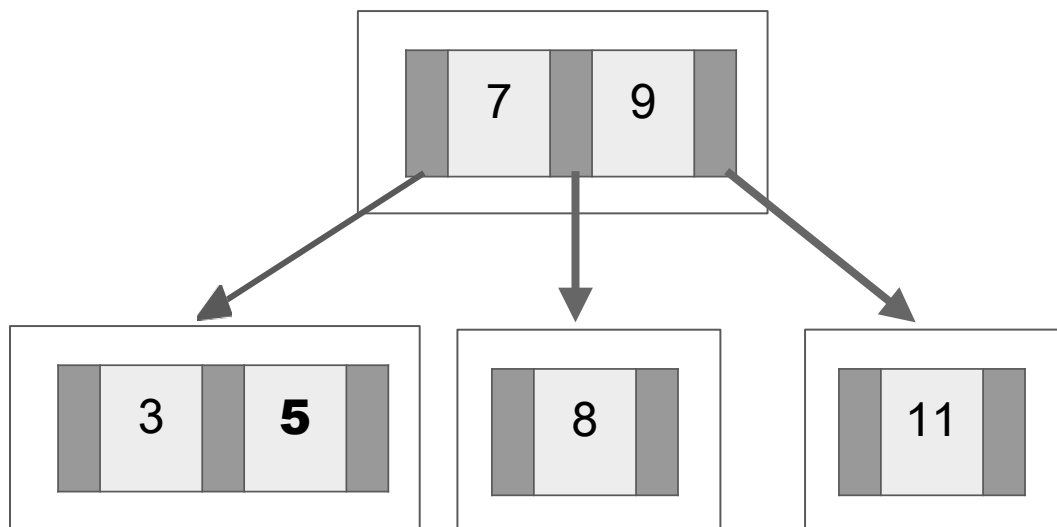
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, **8**, 5, 1



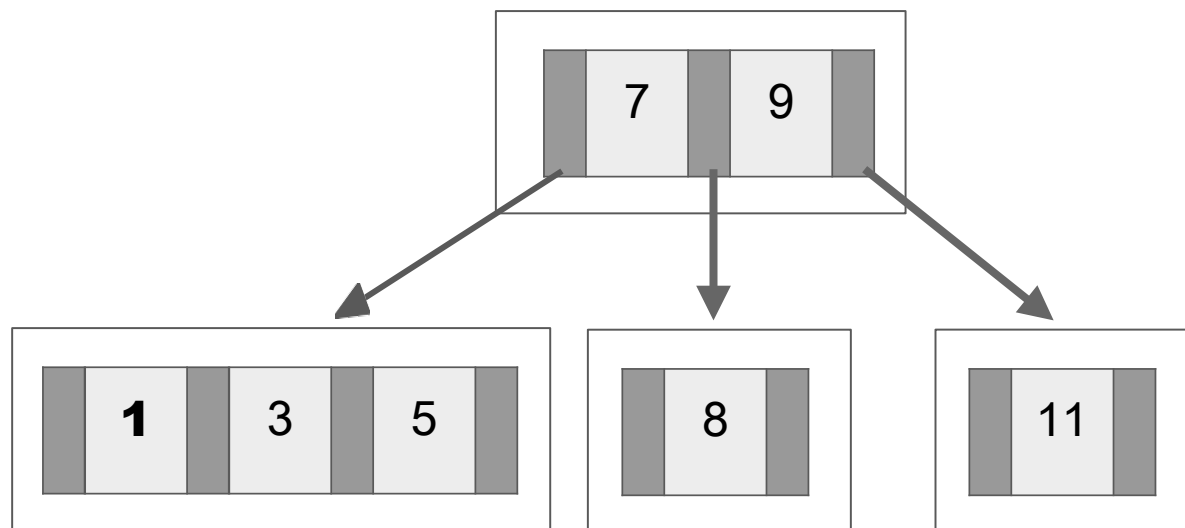
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, 8, **5**, 1



Operações em árvores B

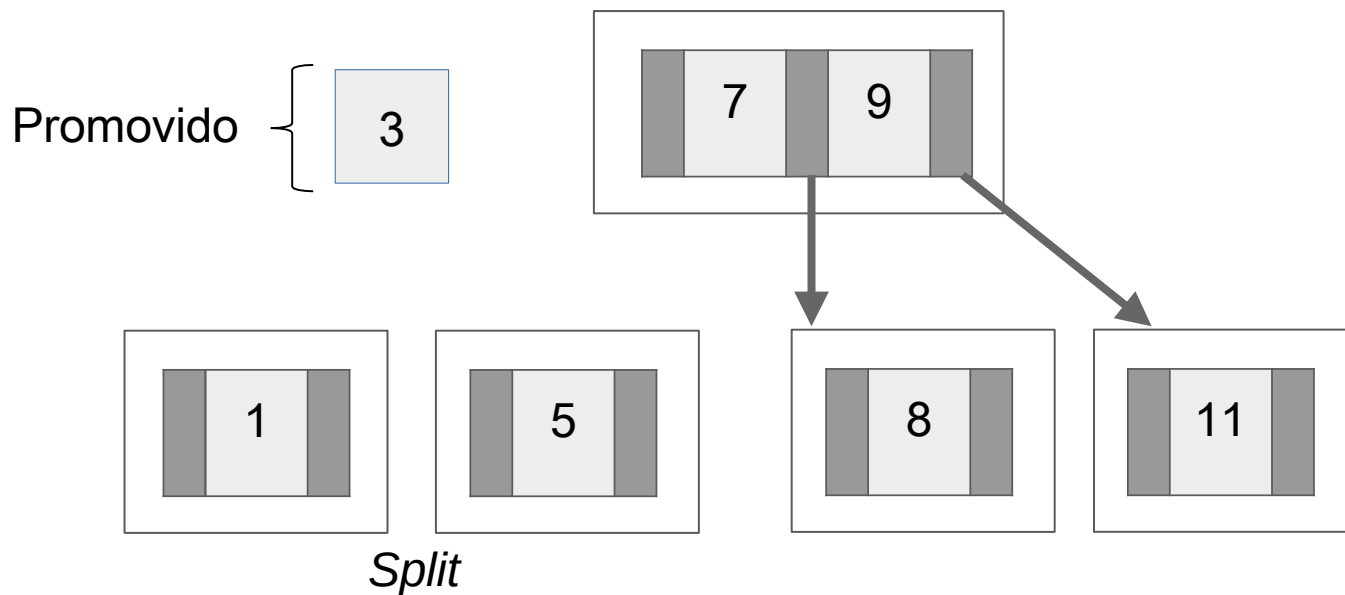
- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, 8, 5, **1**



Overflow

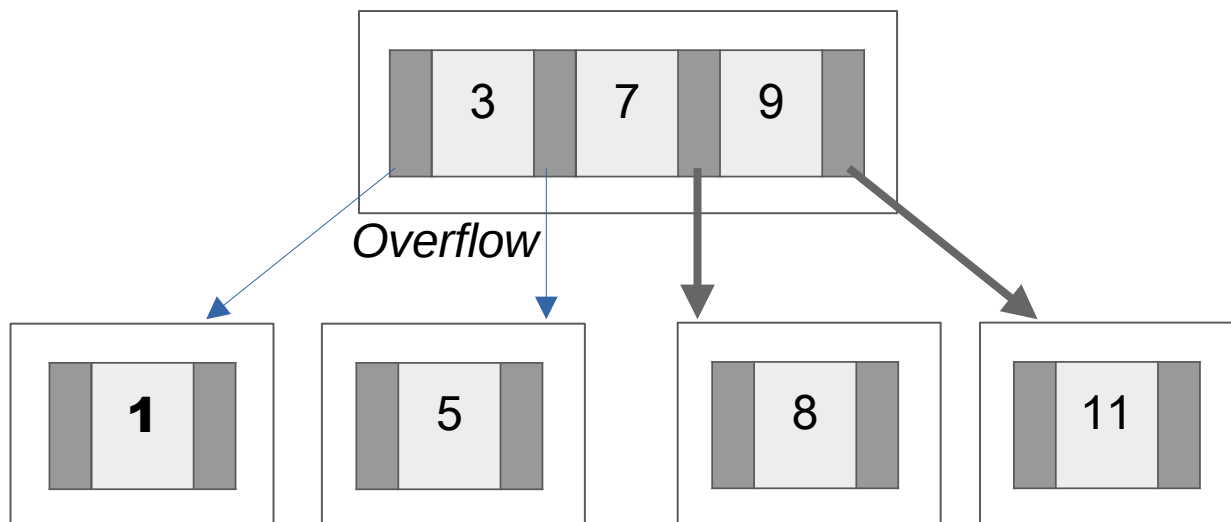
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, 8, 5, **1**



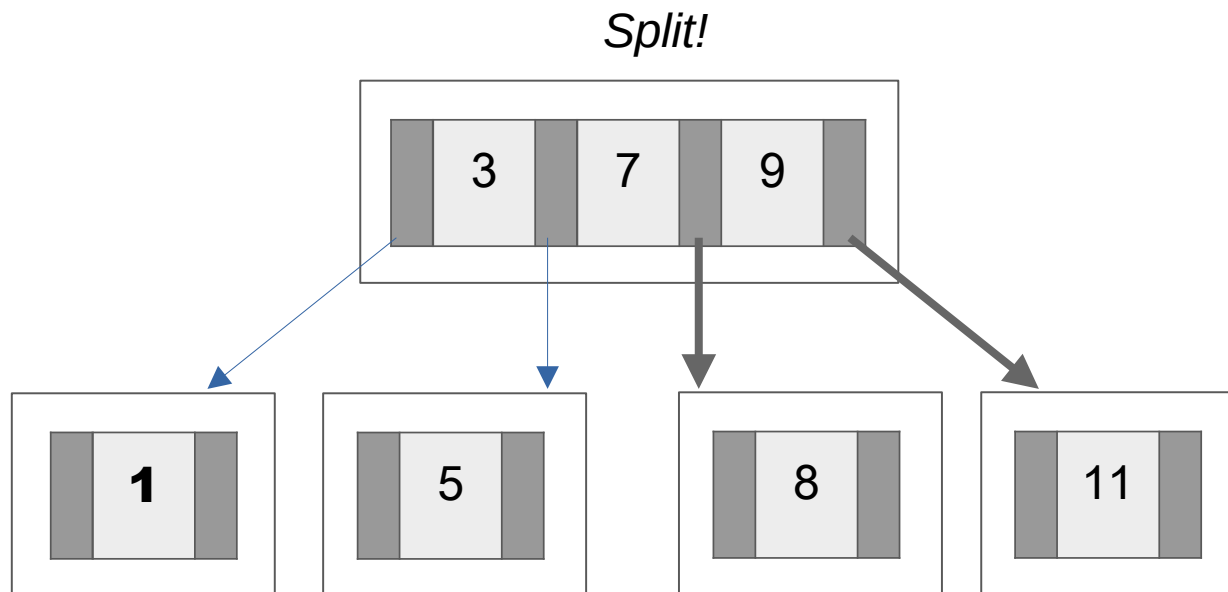
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, 8, 5, **1**



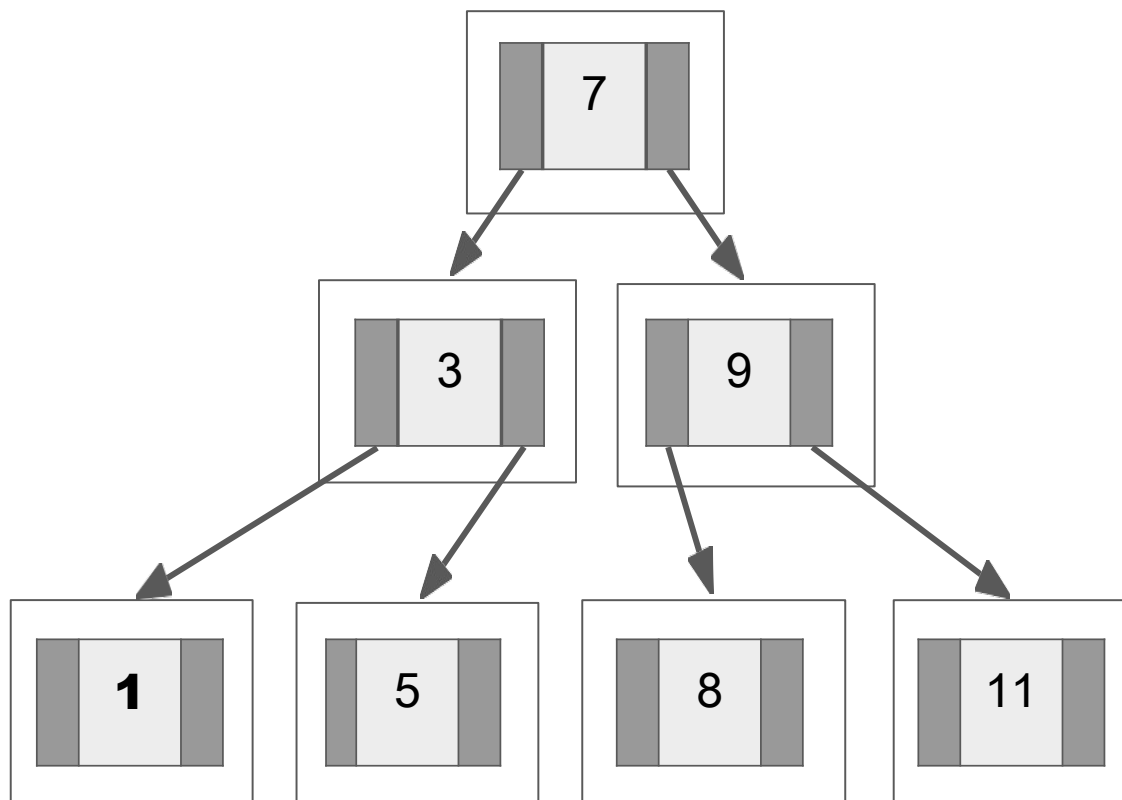
Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, 8, 5, **1**



Operações em árvores B

- Exemplo de adição de chaves em uma árvore de ordem $M=2$
 - Chaves: 9, 3, 11, 7, 8, 5, **1**



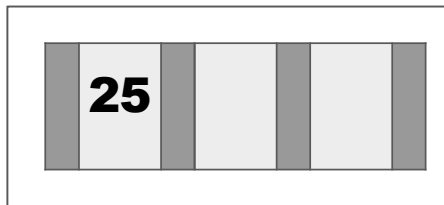
Operações em árvores B

Exemplo

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$

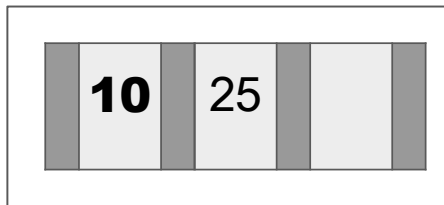
Operações em árvores B

$M = 3 \rightarrow$ **25**, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1



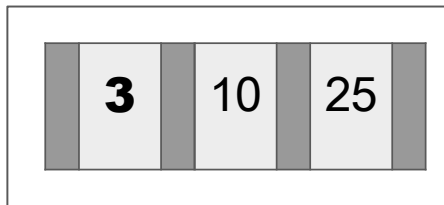
Operações em árvores B

$M = 3 \rightarrow 25, \mathbf{10}, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$



Operações em árvores B

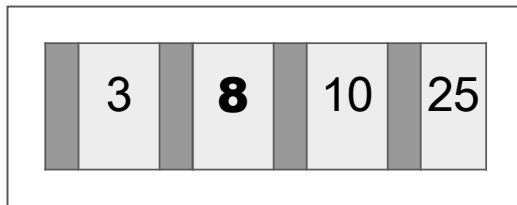
$M = 3 \rightarrow 25, 10, \mathbf{3}, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$



Operações em árvores B

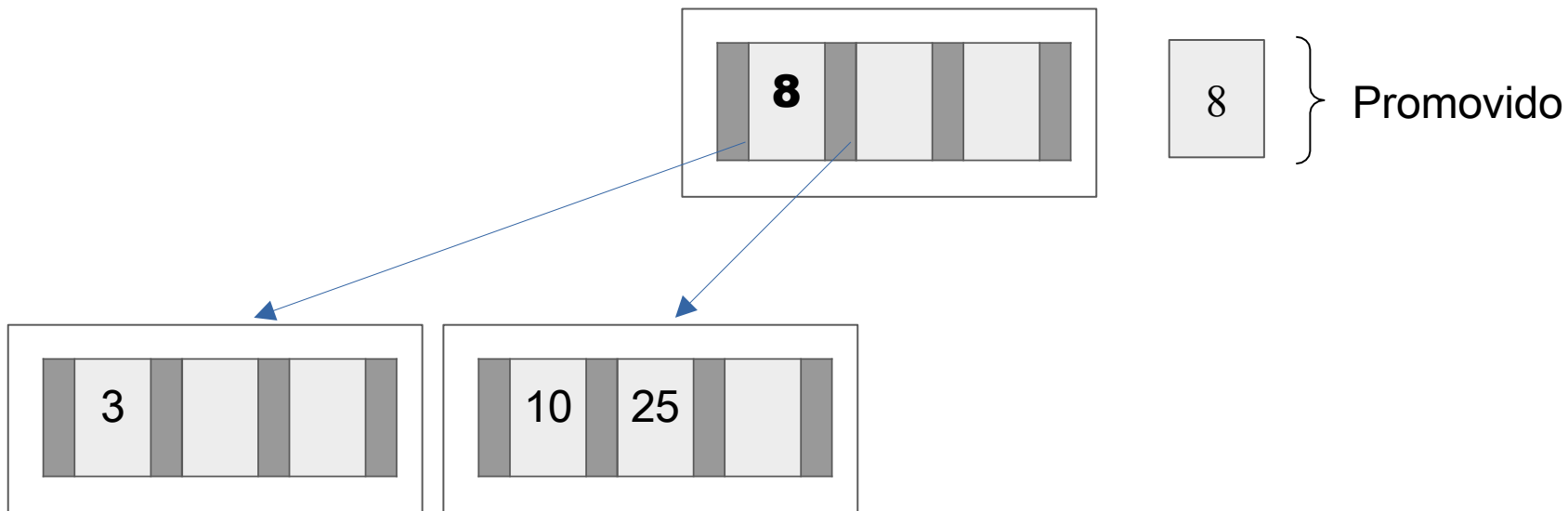
$M = 3 \rightarrow 25, 10, 3, \mathbf{8}, 14, 40, 20, 9, 2, 6, 28, 11, 1$

Split!



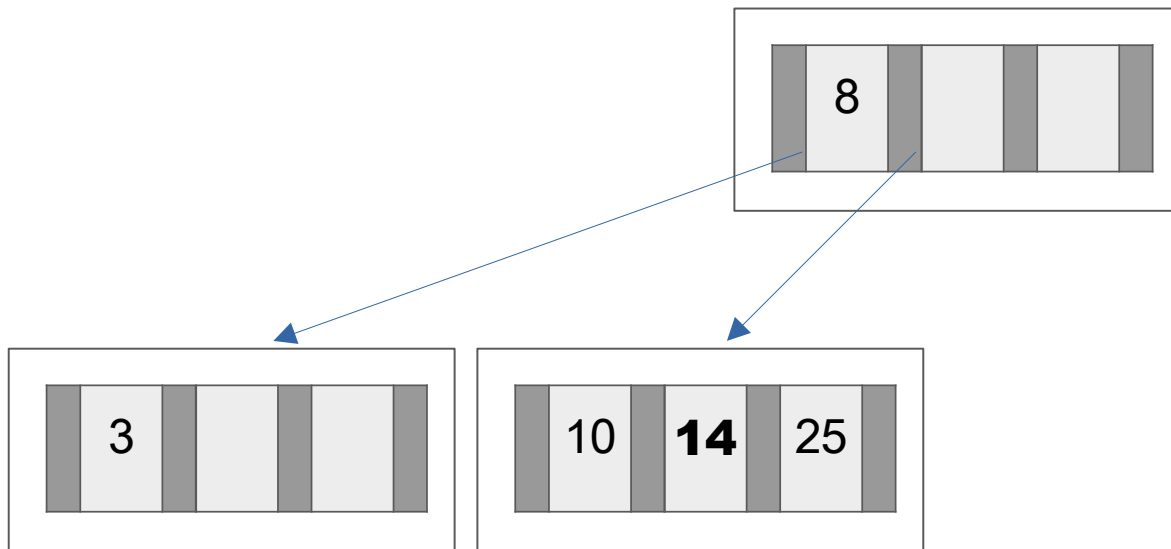
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, \mathbf{8}, 14, 40, 20, 9, 2, 6, 28, 11, 1$



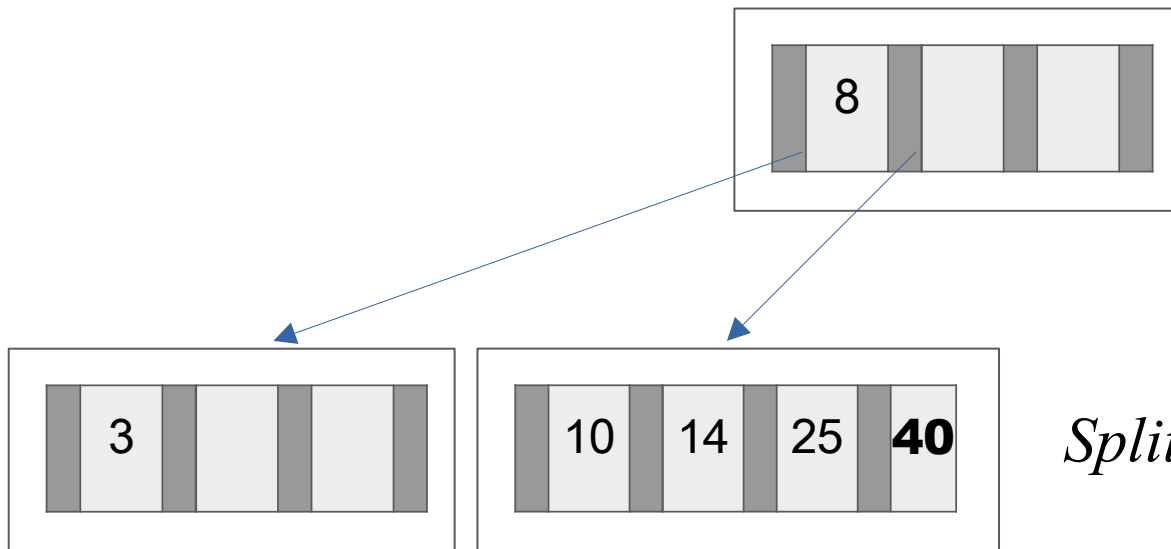
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, \mathbf{14}, 40, 20, 9, 2, 6, 28, 11, 1$



Operações em árvores B

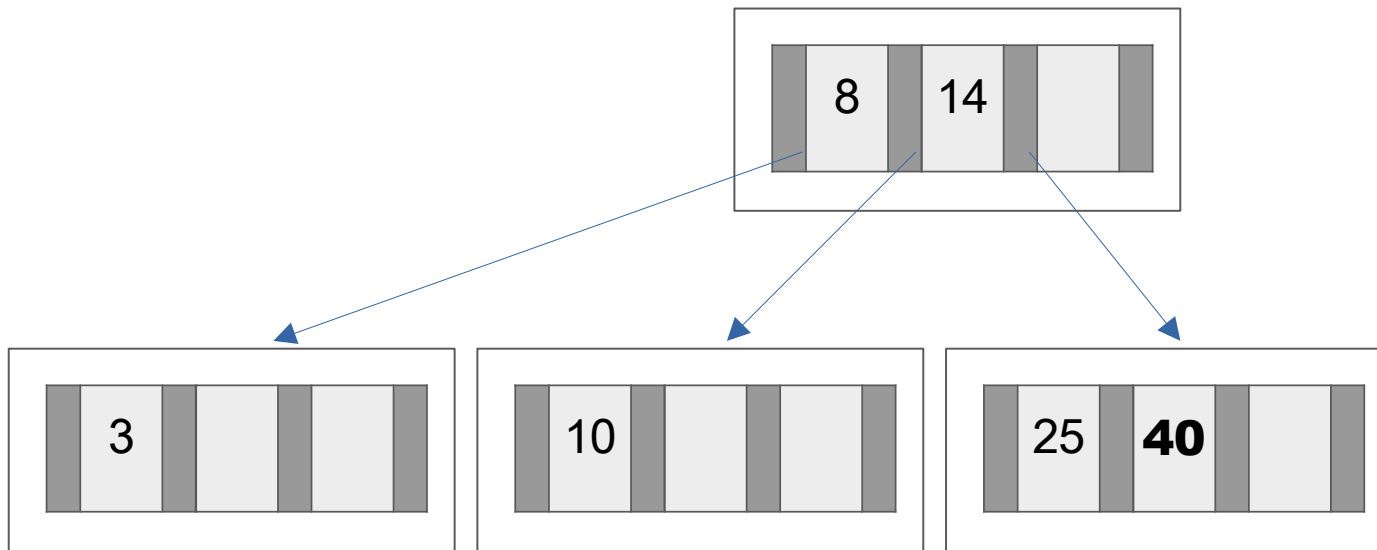
$M = 3 \rightarrow 25, 10, 3, 8, 14, \mathbf{40}, 20, 9, 2, 6, 28, 11, 1$



Split!

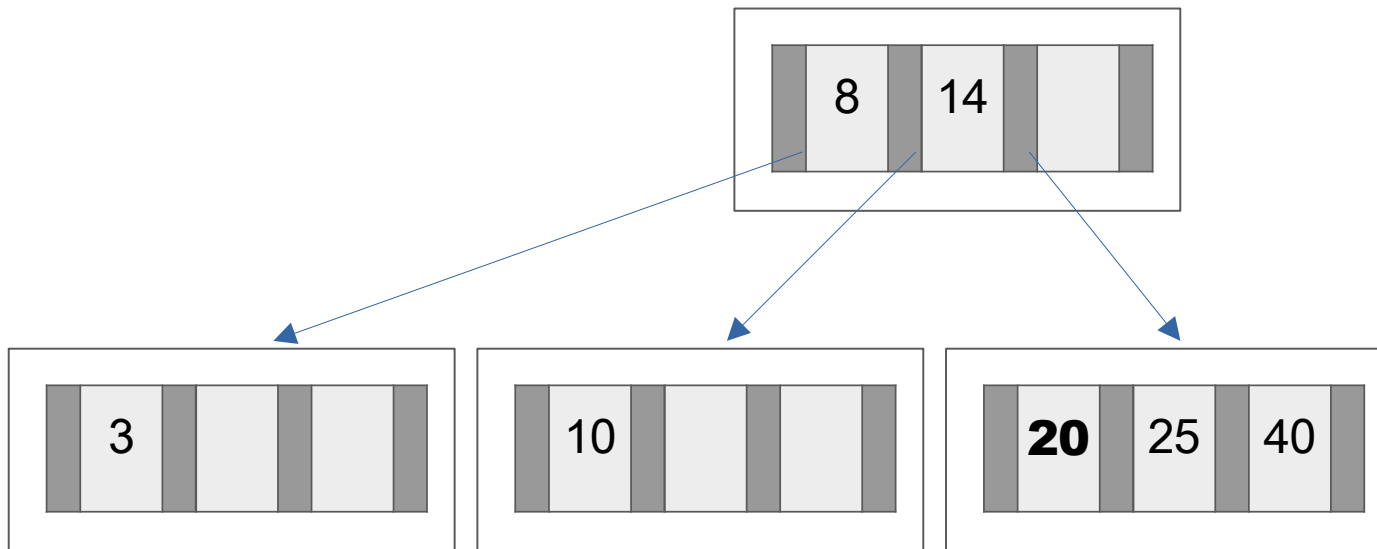
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, \mathbf{40}, 20, 9, 2, 6, 28, 11, 1$



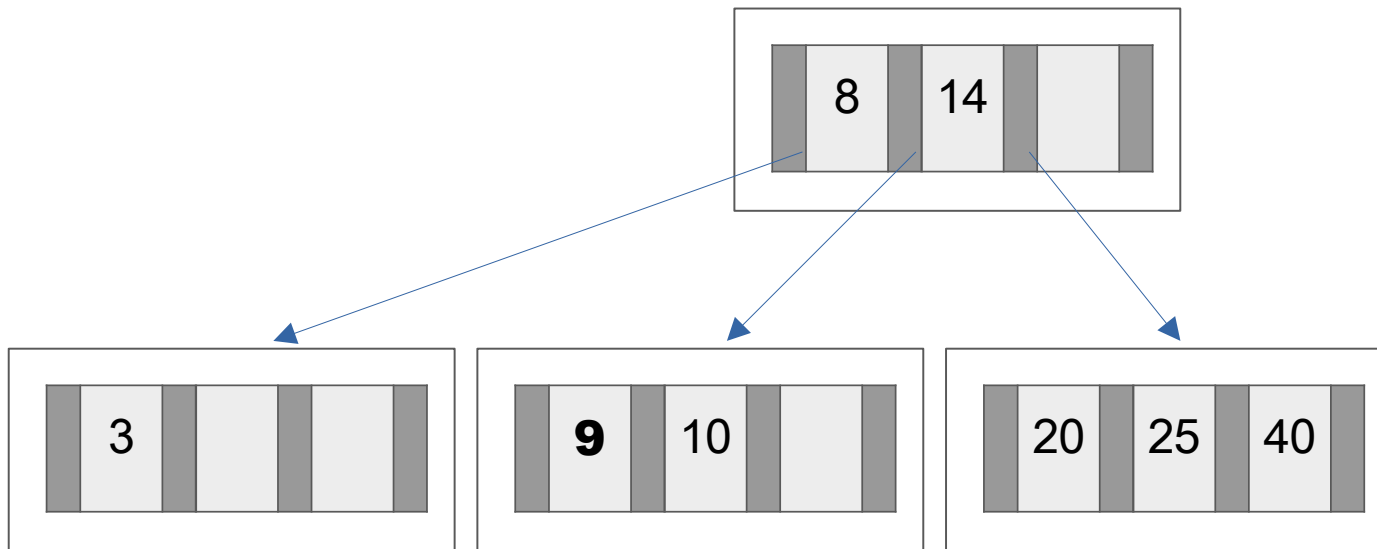
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, \mathbf{20}, 9, 2, 6, 28, 11, 1$



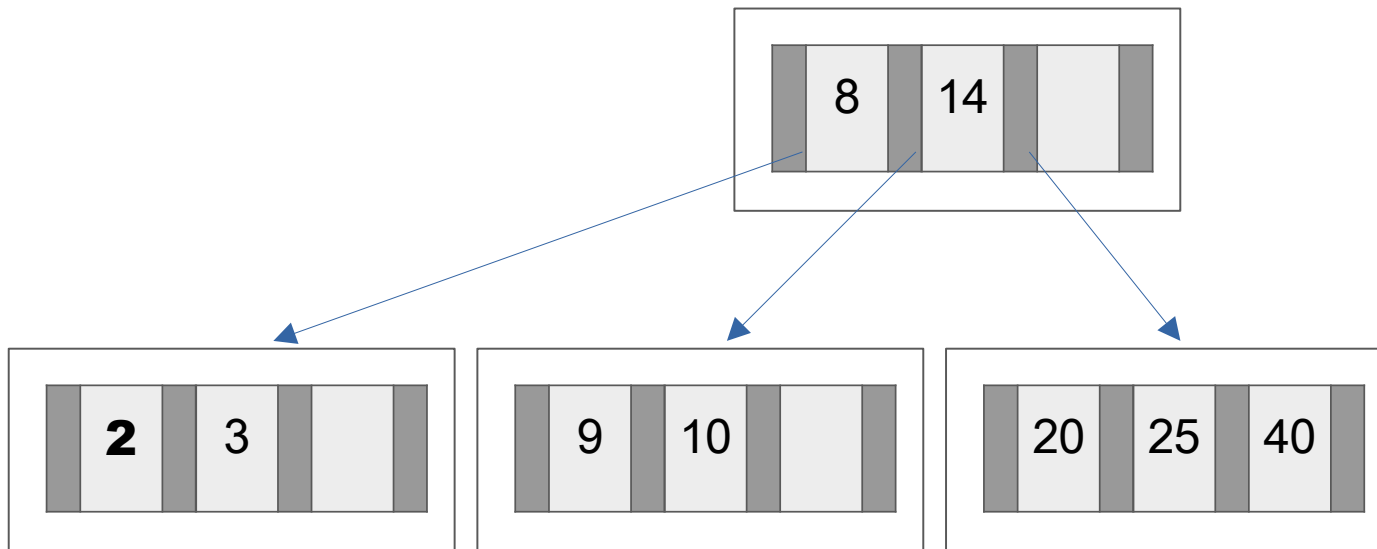
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, \mathbf{9}, 2, 6, 28, 11, 1$



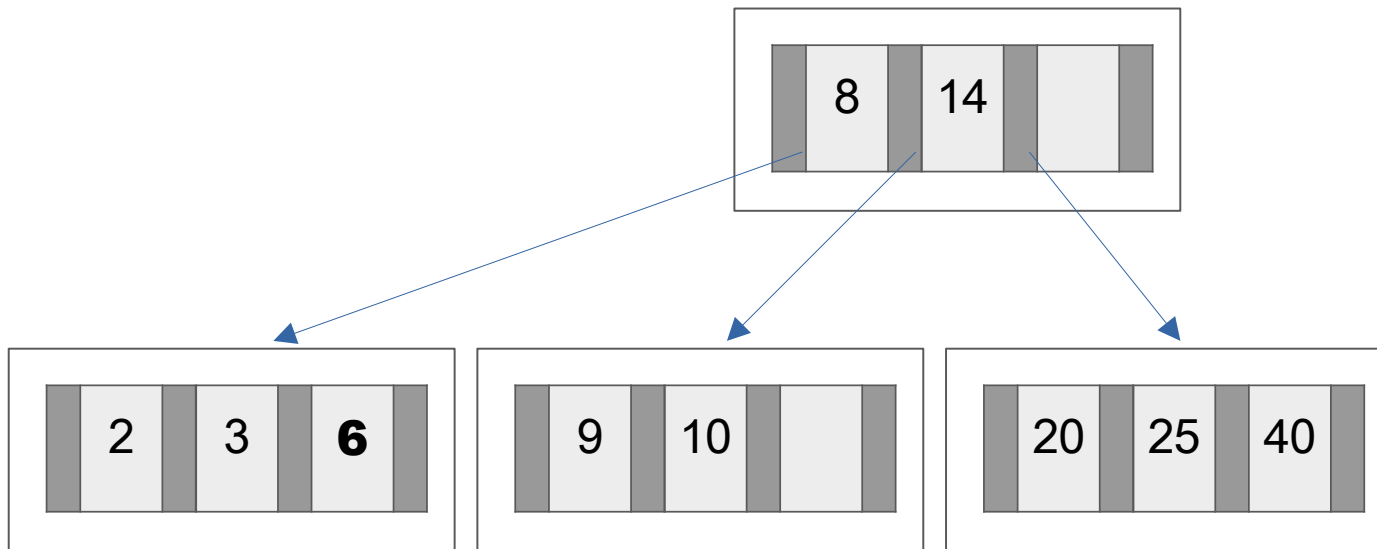
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, \mathbf{2}, 6, 28, 11, 1$



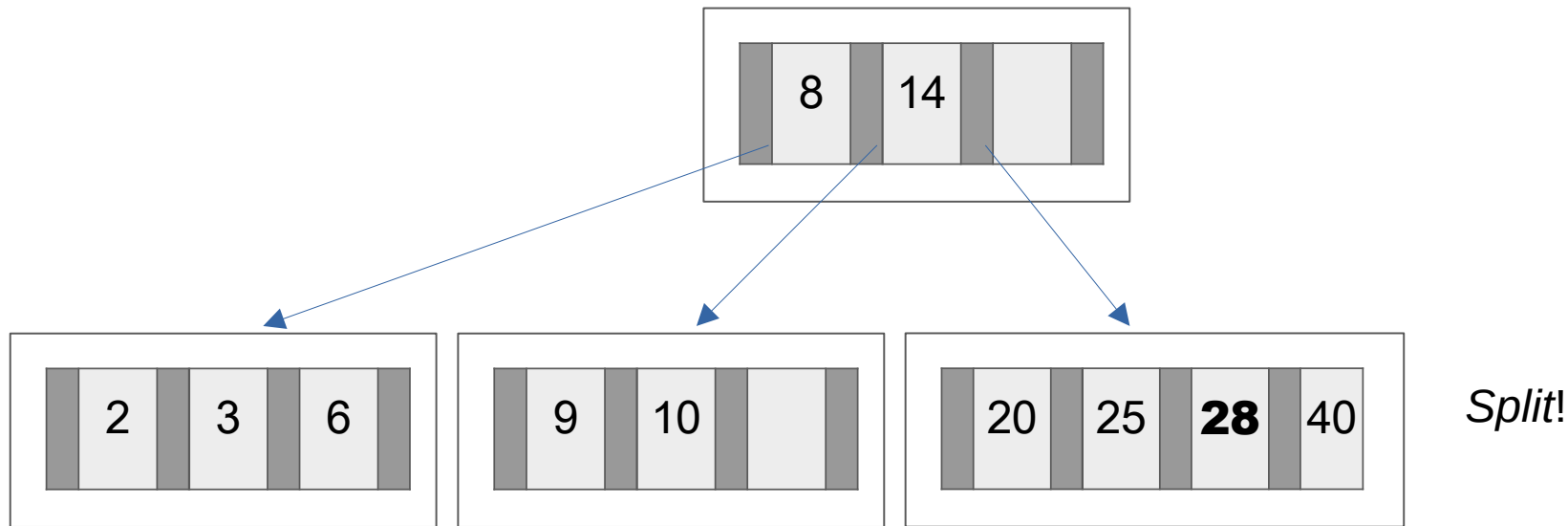
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, \mathbf{6}, 28, 11, 1$



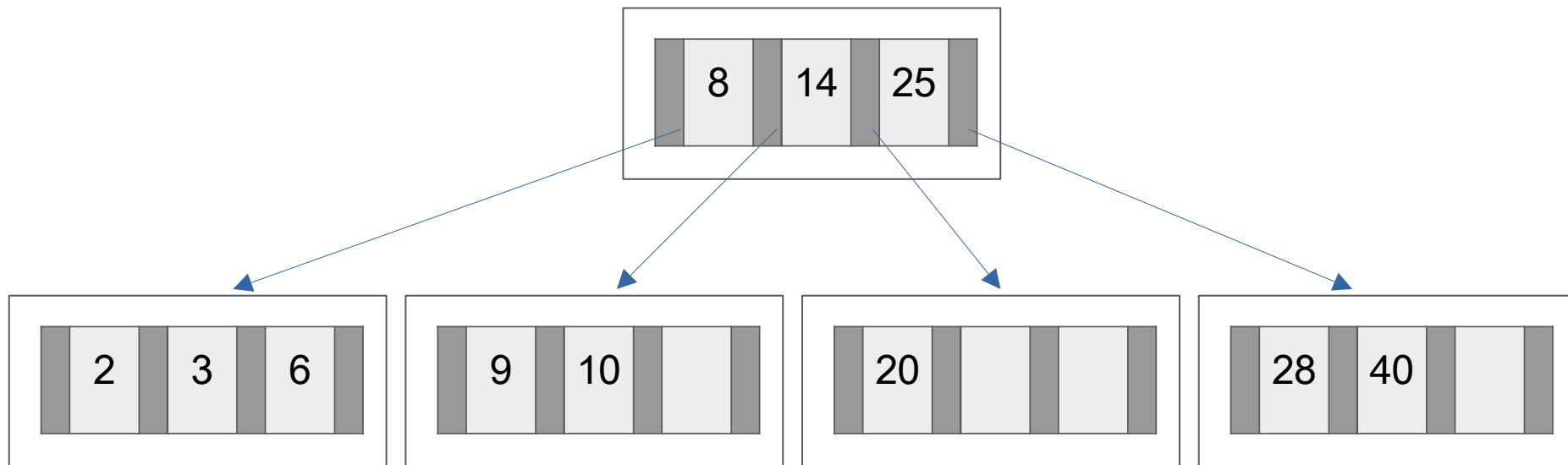
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, \mathbf{28}, 11, 1$



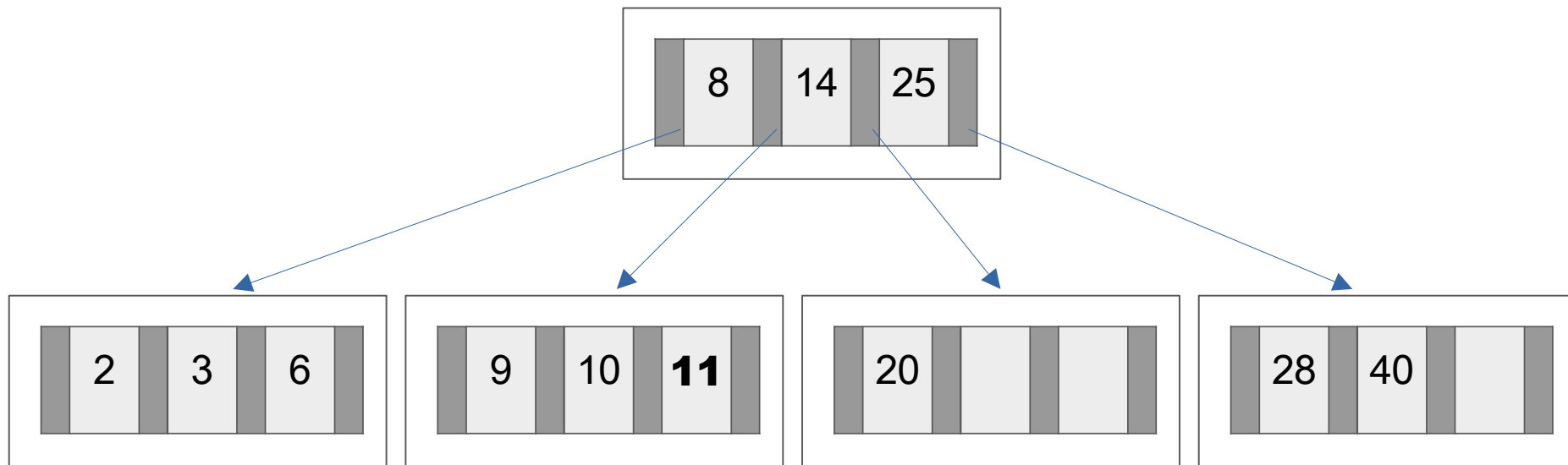
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, \mathbf{28}, 11, 1$



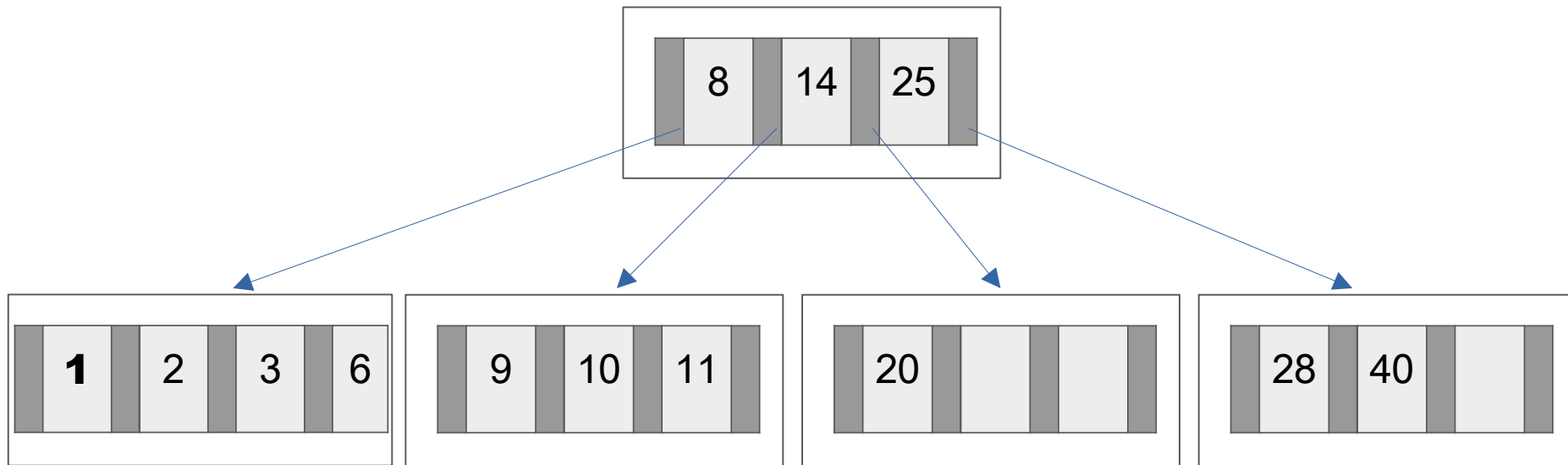
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, \mathbf{11}, 1$



Operações em árvores B

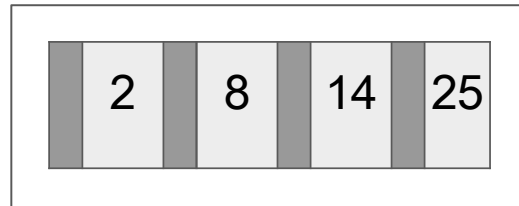
$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$



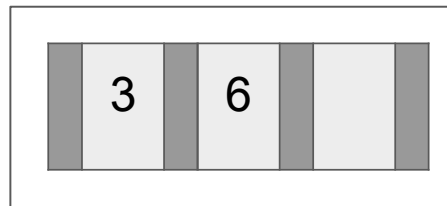
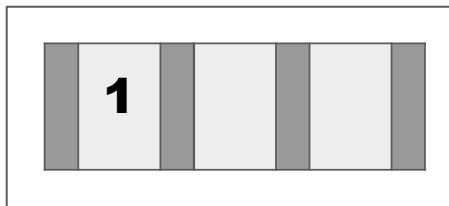
Split!

Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$



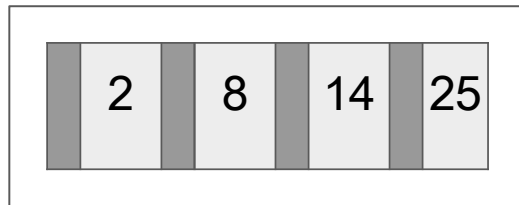

 } Promovido



Operações em árvores B

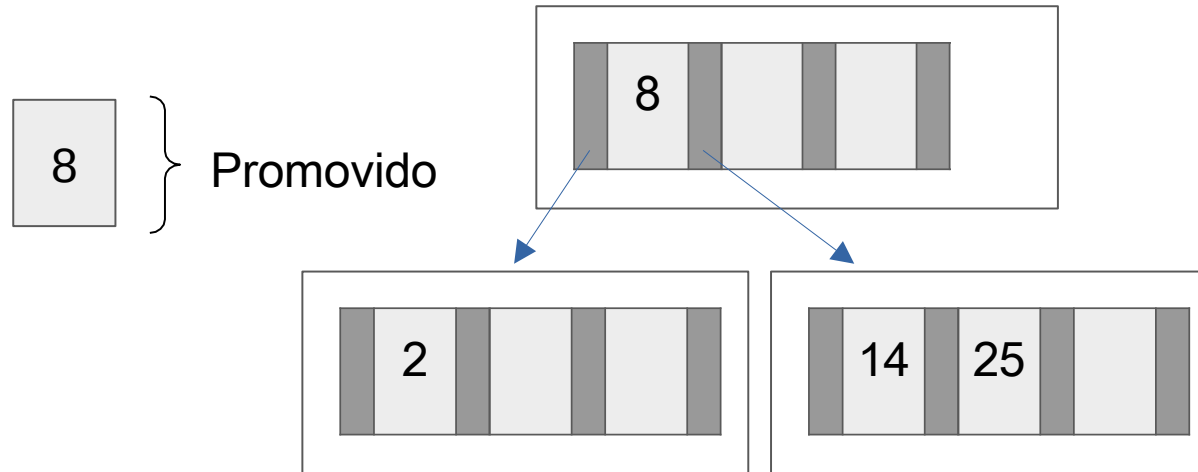
$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$

Split!



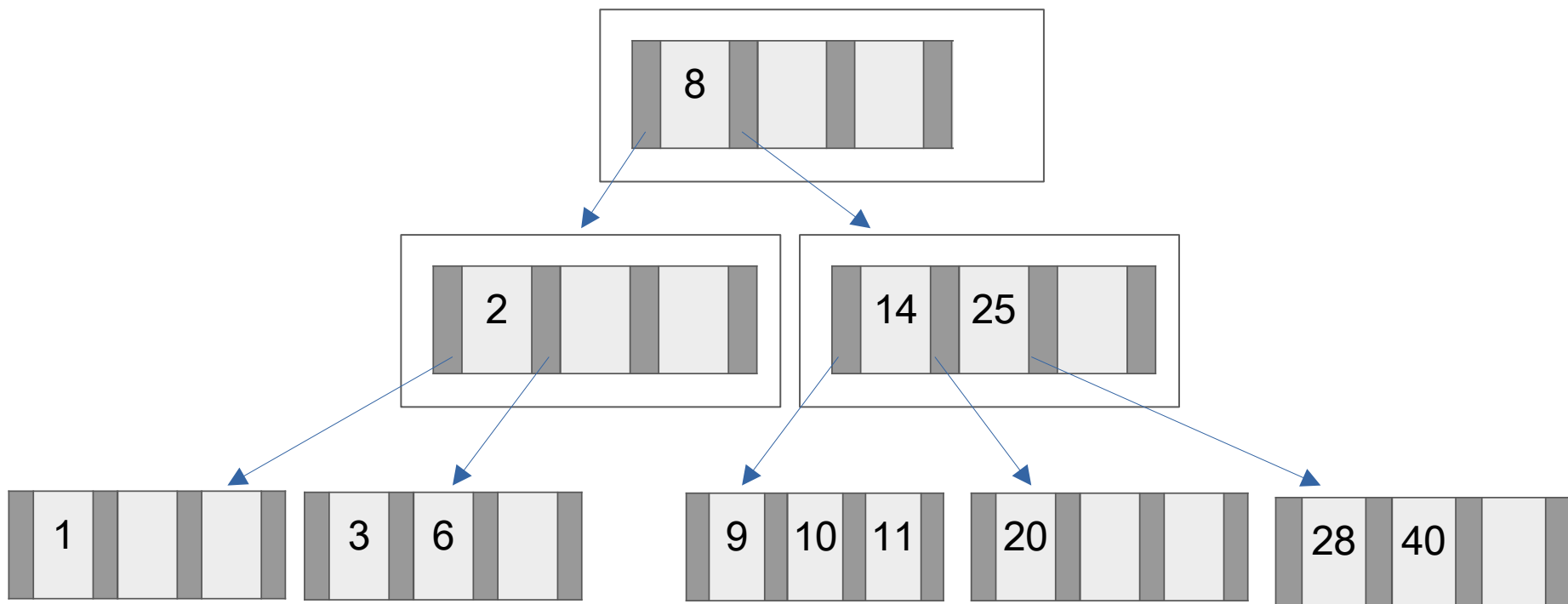
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$



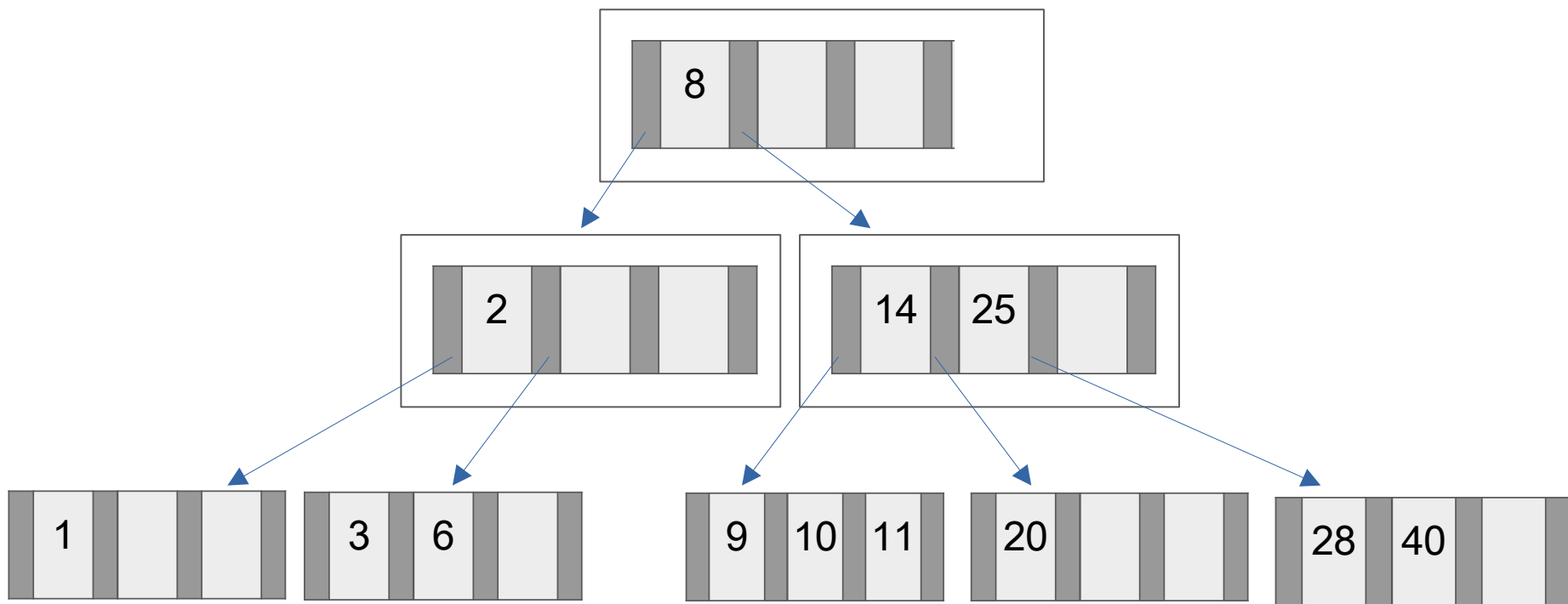
Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$



Operações em árvores B

$M = 3 \rightarrow 25, 10, 3, 8, 14, 40, 20, 9, 2, 6, 28, 11, 1$



Divididos anteriormente

Estrutura da árvores B

- Estrutura da árvore

```
typedef struct _no {  
    int total;  
    int* chaves;  
    struct _no** filhos;  
    struct _no* pai;  
} No;
```

```
struct arvoreB {  
    No* raiz;  
    int ordem;  
} arvore;
```

Operações em árvores B

- Criar uma árvore B com uma ordem predefinida (cont.)

```
No* criaNo() {  
    No* no = malloc(sizeof(No));  
    int max = arvore->ordem * 2;  
  
    no->pai = NULL;  
    no->chaves = malloc(sizeof(int) * max);  
    no->filhos = malloc(sizeof(No) * max);  
    no->total = 0;  
    for (int i = 0; i < max; i++)  
        no->filhos[i] = NULL;  
    return no;  
}
```

Operações em árvores B

- Percorrer uma árvore a partir de um nó

```
void percorreArvore(No* no, void(visita)(int chave)) {  
    if (no != NULL) {  
        for (int i=0; i<no->total; i++) {  
            percorreArvore(no->filhos[i], visita);  
            visita(no->chaves[i]);  
        }  
        percorreArvore(no->filhos[no->total], visita);  
    }  
}
```

Operações em árvores B

- Localizar uma chave em uma árvore

```
int localizaChave(int chave) {  
    No *no = arvore->raiz;  
    while (no != NULL) {  
        int i = pesquisaBinaria(no, chave);  
        if (i < no->total && no->chaves[i] == chave) {  
            return 1; // encontrou  
        } else {  
            No = no->filhos[i];  
        }  
    }  
    return 0; // não encontrou  
}
```

Operações em árvores B

- Localizar um nó a partir de uma chave

```
No* localizaNo(int chave)  {
    No *no = arvore->raiz;
    while (no != NULL)  {
        int i = pesquisaBinaria(no, chave);
        if (no->filhos[i] == NULL)
            return no; //encontrou nó
        else
            no = no->filhos[i];
    }
    return NULL; //não encontrou nenhum nó
}
```

Operações em árvores B

- Localizar uma chave em uma árvore (cont.)

```
int pesquisaBinaria(No* no, int chave) {
    int inicio = 0, fim = no->total - 1, meio;
    while (inicio <= fim) {
        meio = (inicio + fim) / 2;
        if (no->chaves[meio] == chave) {
            return meio; //encontrou
        } else if (no->chaves[meio] > chave) {
            fim = meio - 1;
        } else {
            inicio = meio + 1;
        }
    }
    return inicio; //não encontrou
}
```


Operações em árvores B

- Adicionar chave em um nó

```
void adicionaChaveNo(No* no, No* direita, int chave) {  
    int i = pesquisaBinaria(no, chave);  
    for (int j = no->total - 1; j >= i; j--) {  
        no->chaves[j + 1] = no->chaves[j];  
        no->filhos[j + 2] = no->filhos[j + 1];  
    }  
    no->chaves[i] = chave;  
    no->filhos[i + 1] = direita;  
    no->total++;  
}
```

Operações em árvores B

- Verificar se houve transbordo (*overflow*) em um nó

```
int transbordo(No *no) {  
    return no->total >= 2 * arvore->ordem;  
}
```

Operações em árvores B

- Dividir chaves (*split*) de um nó em um novo nó

```
No* divideNo (No* no) {  
    int meio = no->total / 2;  
    No* novo = criaNo();  
    novo->pai = no->pai;  
    for (int i = meio + 1; i < no->total; i++) {  
        novo->filhos[novo->total] = no->filhos[i];  
        novo->chaves[novo->total] = no->chaves[i];  
        if (novo->filhos[novo->total] != NULL)  
            novo->filhos[novo->total]->pai = novo;  
        novo->total++;  
    }  
    novo->filhos[novo->total] = no->filhos[no->total];  
    if (novo->filhos[novo->total] != NULL)  
        novo->filhos[novo->total]->pai = novo;  
    no->total = meio;  
    return novo;  
}
```

Operações em árvores B

- Adicionar nova chave na árvore

```
void adicionaChave(int chave) {  
    No* no = localizaNo(chave);  
    adicionaChaveRecursivo(no, NULL, chave);  
}
```

Operações em árvores B

- Adicionar nova chave na árvore (cont.)

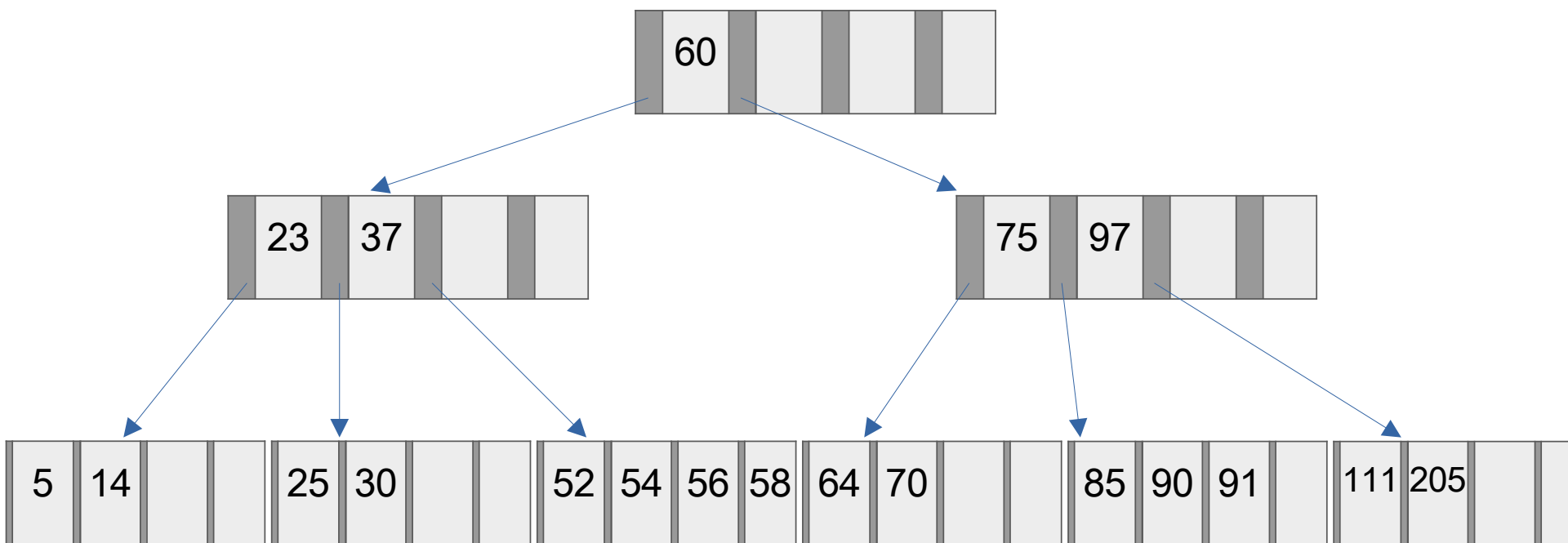
```
void adicionaChaveRecursivo(No* no, No* novo, int chave) {
    adicionaChaveNo(no, novo, chave);
    if (transbordo(no)) {
        int promovido = no->chaves[arvore->ordem];
        No* novo = divideNo(no);
        if (no->pai == NULL) {
            No* raiz = criaNo(); raiz->filhos[0] = no;
            adicionaChaveNo(pai, novo, promovido);
            no->pai = raiz; novo->pai = raiz;
            arvore->raiz = raiz;
        } else
            adicionaChaveRecursivo(no->pai, novo, promovido);
    }
}
```

Operações em árvores B

- Remoção
 - Caso 1: se elemento estiver na folha e a folha mantém 50% de ocupação, basta remover a chave;
 - Caso 2: se o elemento NÃO estiver em folha, tem que ser trocado pelo seu sucessor
 - Não se faz remoção em nós superiores, senão teria que subir algum valor para substituir e a árvore pode ficar desbalanceada
 - Nesse caso, substitui-se pelo valor máximo à esquerda (ou valor mínimo à direita)
 - Caso 3: se a folha ficar com menos de 50% de ocupação e a página irmã puder ceder uma chave
 - Normalmente necessário uma rotação para manter os índices válidos
 - Transbordo ou *overflow*: número de chaves excede a capacidade
 - Caso 4: se folha ficar com menos de 50% de ocupação e as páginas irmãs não puderem ceder uma chave
 - Faz-se a fusão com o irmão à esquerda ou o irmão à direita

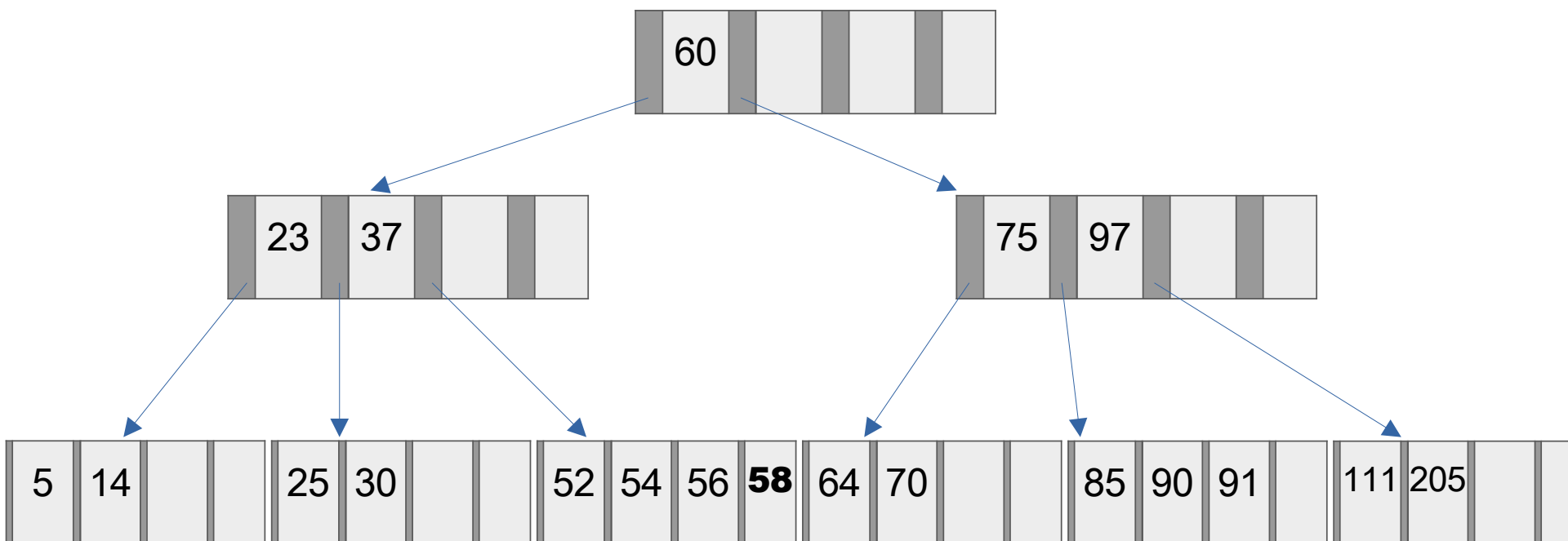
Operações em árvores B

M = 4 – ordem 2



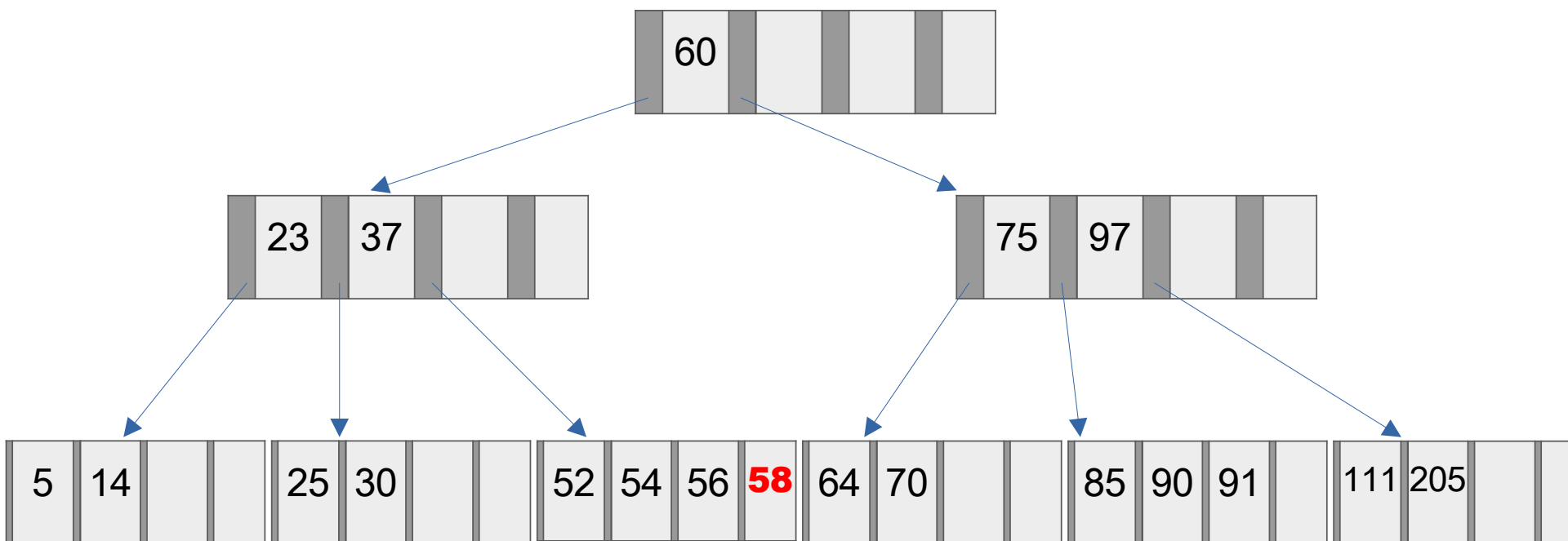
Operações em árvores B

$M = 4 \rightarrow$ **Remove 58**



Operações em árvores B

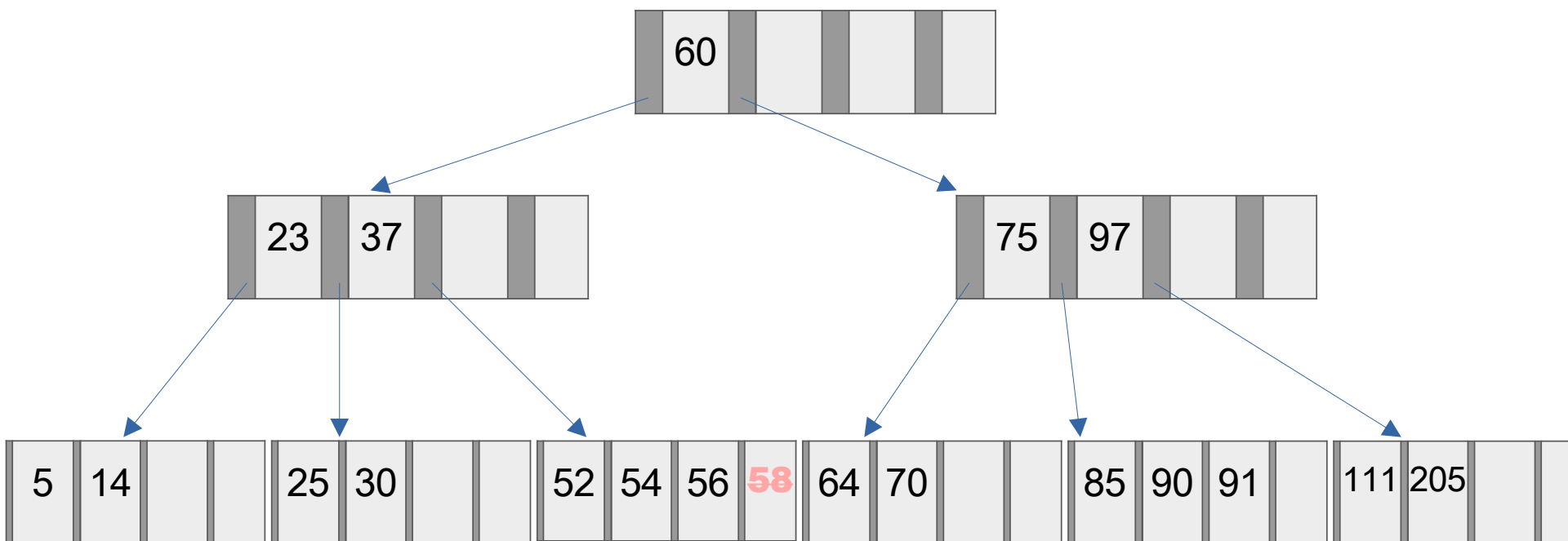
$M = 4 \rightarrow$ Remover 58



Caso 1: se elemento estiver na folha e a folha mantém 50% de ocupação, basta remover a chave;

Operações em árvores B

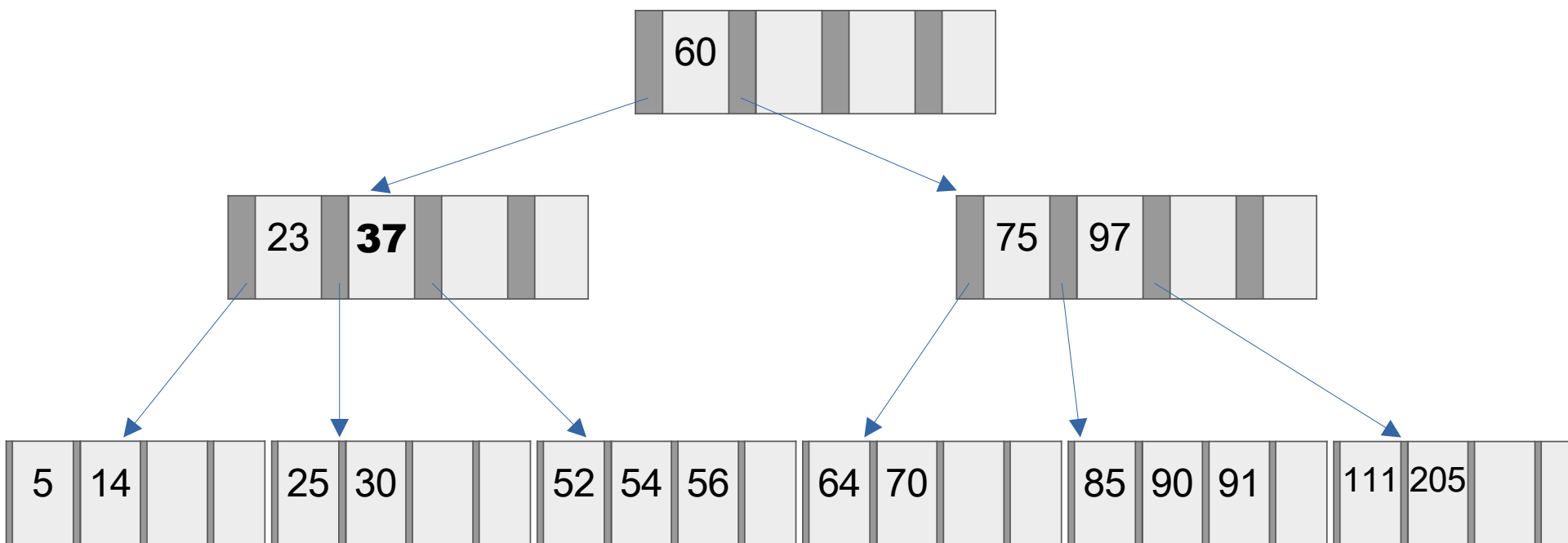
$M = 4 \rightarrow$ Remove 58



Caso 1: se elemento estiver na folha e a folha mantém 50% de ocupação, basta remover a chave;

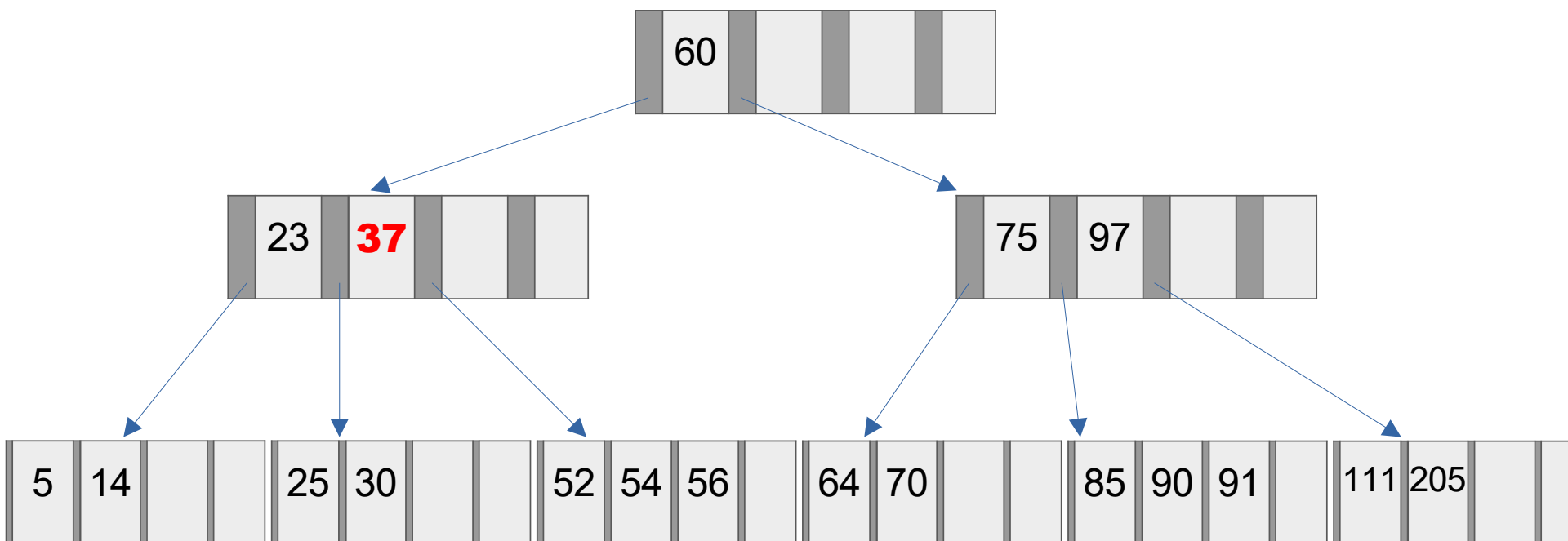
Operações em árvores B

$M = 4 \rightarrow$ **Remover 37**



Operações em árvores B

$M = 4 \rightarrow$ Remove 37



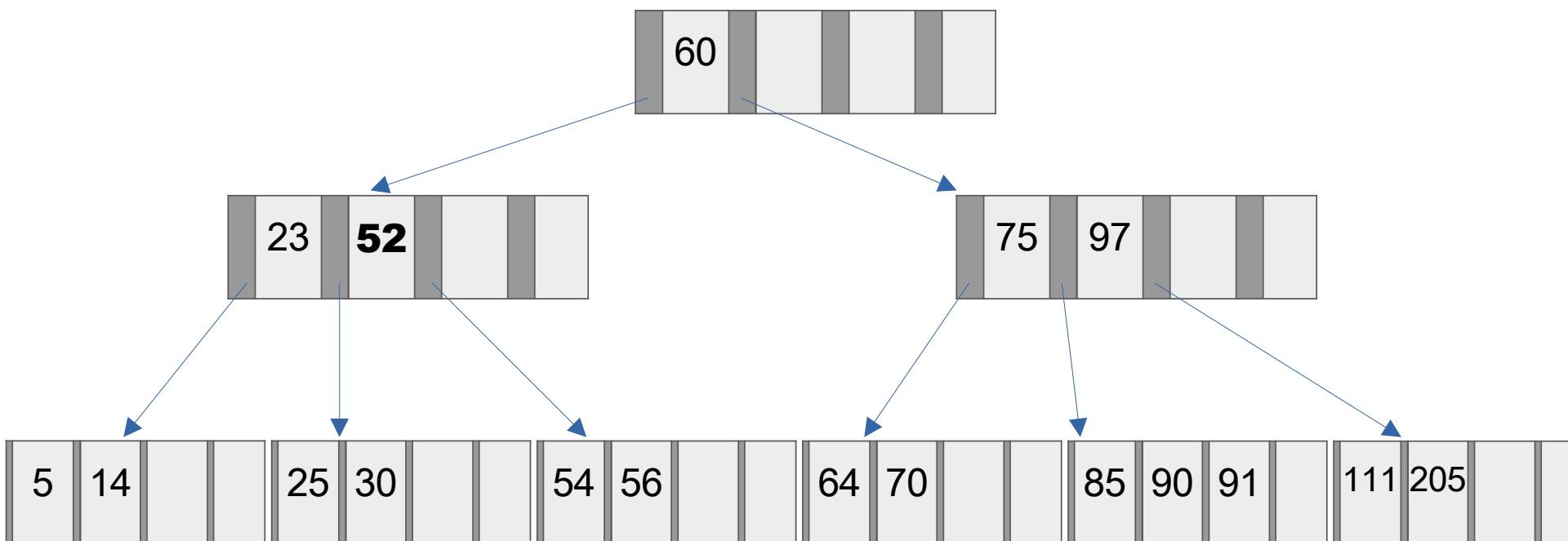
Caso 2: se o elemento NÃO estiver em folha, tem que ser trocado pelo seu sucessor

- Não se faz remoção em nós superiores, senão teria que subir algum valor para substituir e a árvore pode ficar desbalanceada

- Nesse caso, substitui-se pelo valor mínimo à direita

Operações em árvores B

$M = 4 \rightarrow$ Remove 37

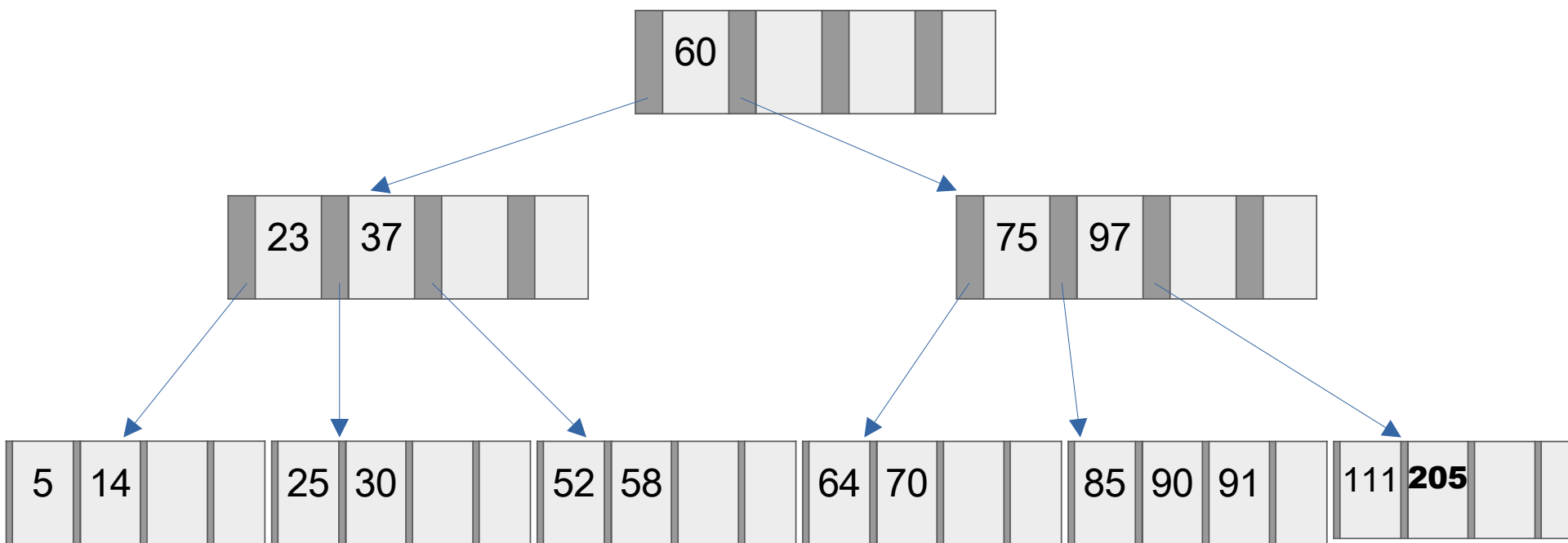


Caso 2: se o elemento NÃO estiver em folha, tem que ser trocado pelo seu sucessor

- Não se faz remoção em nós superiores, senão teria que subir algum valor para substituir e a árvore pode ficar desbalanceada
- Nesse caso, substitui-se pelo valor mínimo à direita (pela esquerda ficaria abaixo de 50%)

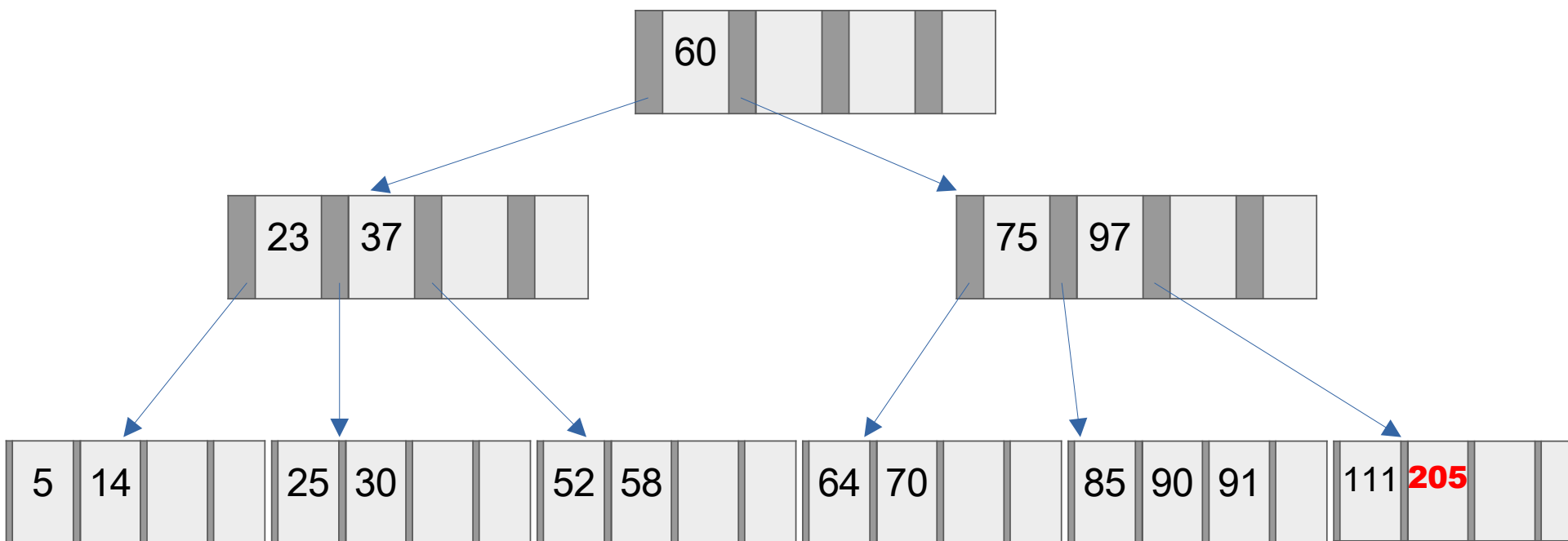
Operações em árvores B

$M = 4 \rightarrow$ **Remover 205**



Operações em árvores B

$M = 4 \rightarrow$ Remove 205

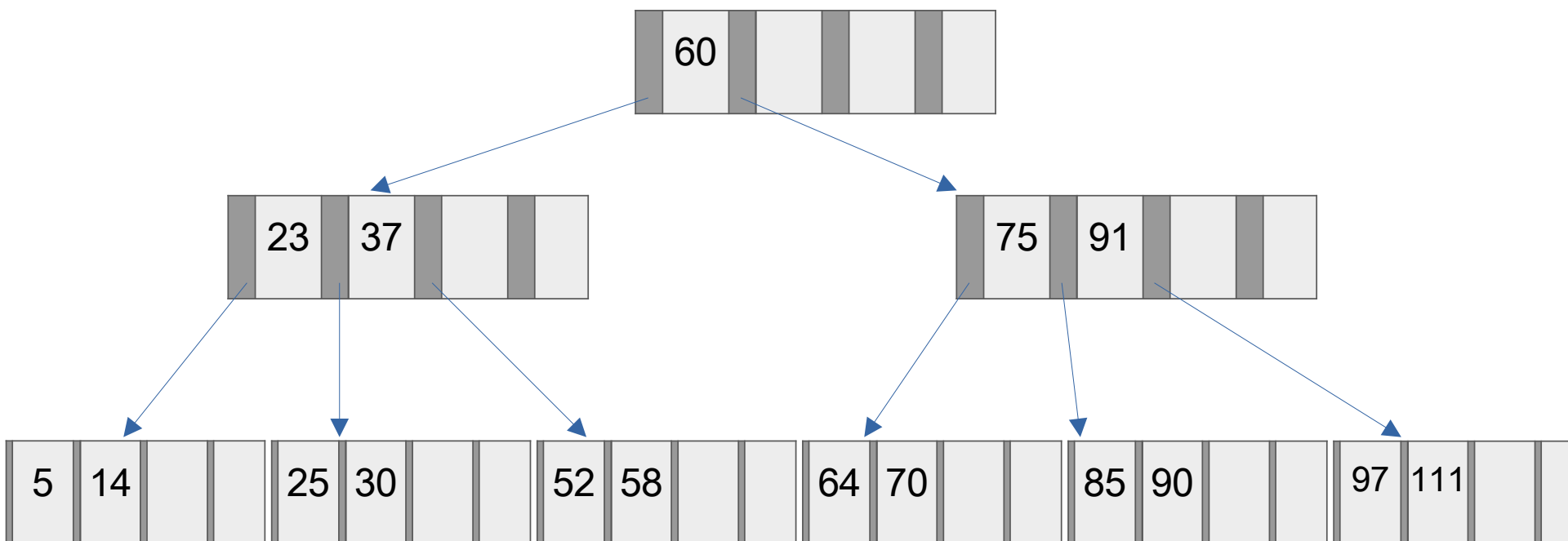


Caso 3: se a folha ficar com menos de 50% de ocupação e a página irmã puder ceder uma chave

- Normalmente necessário uma rotação para manter os índices válidos

Operações em árvores B

$M = 4 \rightarrow$ Remove 205

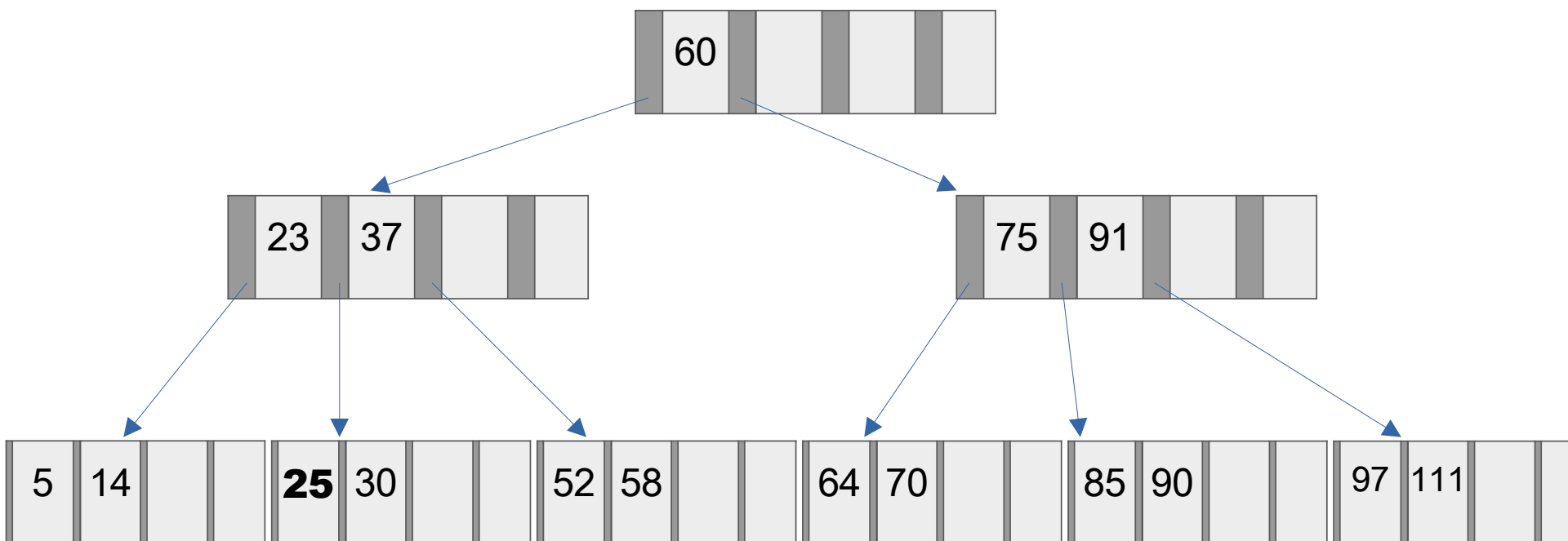


Caso 3: se a folha ficar com menos de 50% de ocupação e a página irmã puder ceder uma chave

- Normalmente necessário uma rotação para manter os índices válidos

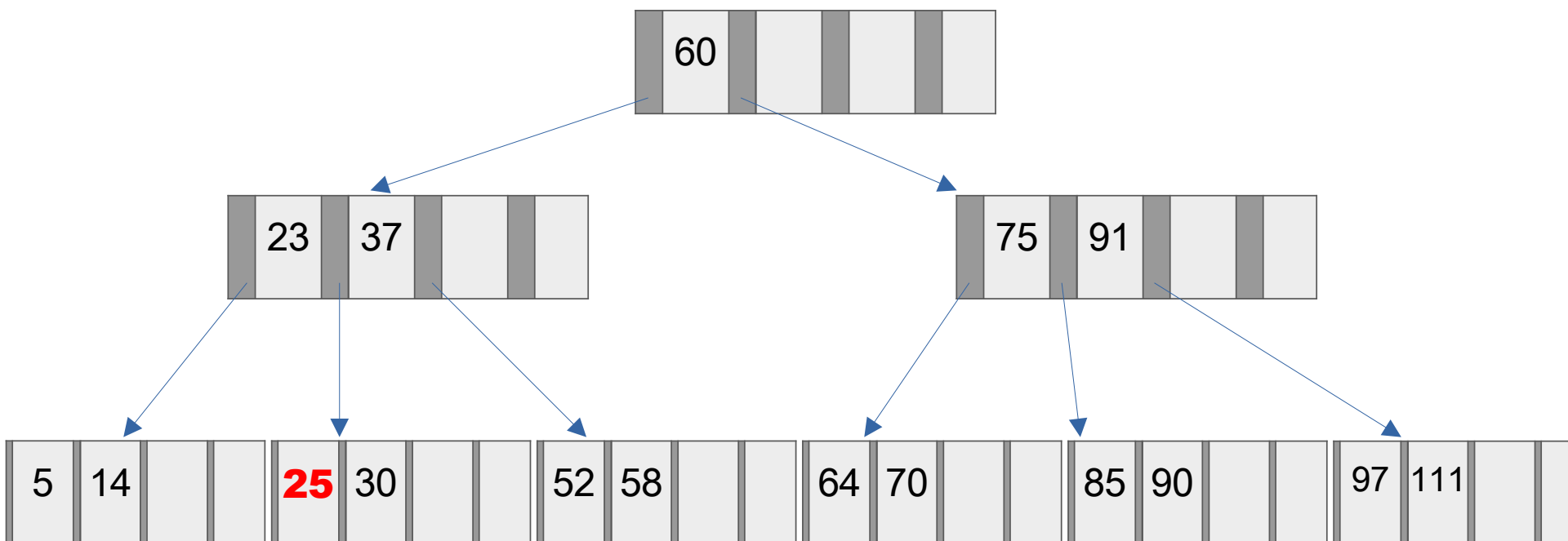
Operações em árvores B

$M = 4 \rightarrow$ **Remover 25**



Operações em árvores B

$M = 4 \rightarrow$ Remove 25

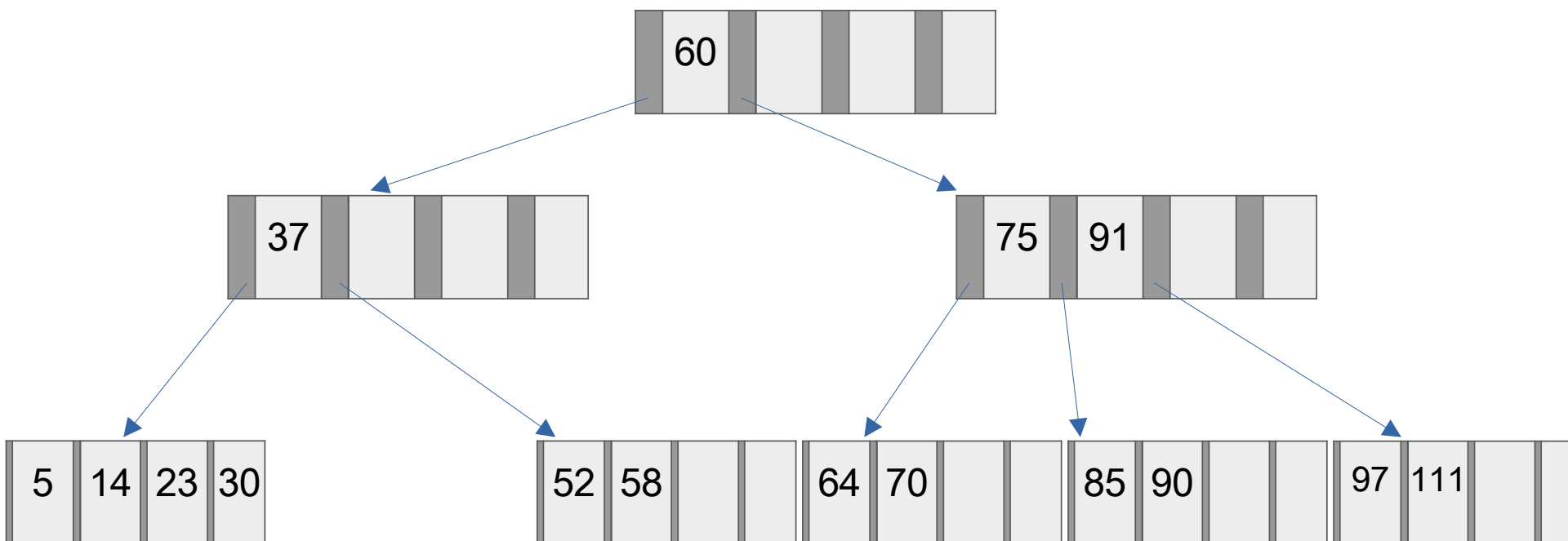


Caso 4: se folha ficar com menos de 50% de ocupação e as páginas irmãs não puderem ceder uma chave

- Faz-se a fusão com o irmão à esquerda ou o irmão à direita

Operações em árvores B

$M = 4 \rightarrow$ Remove 25

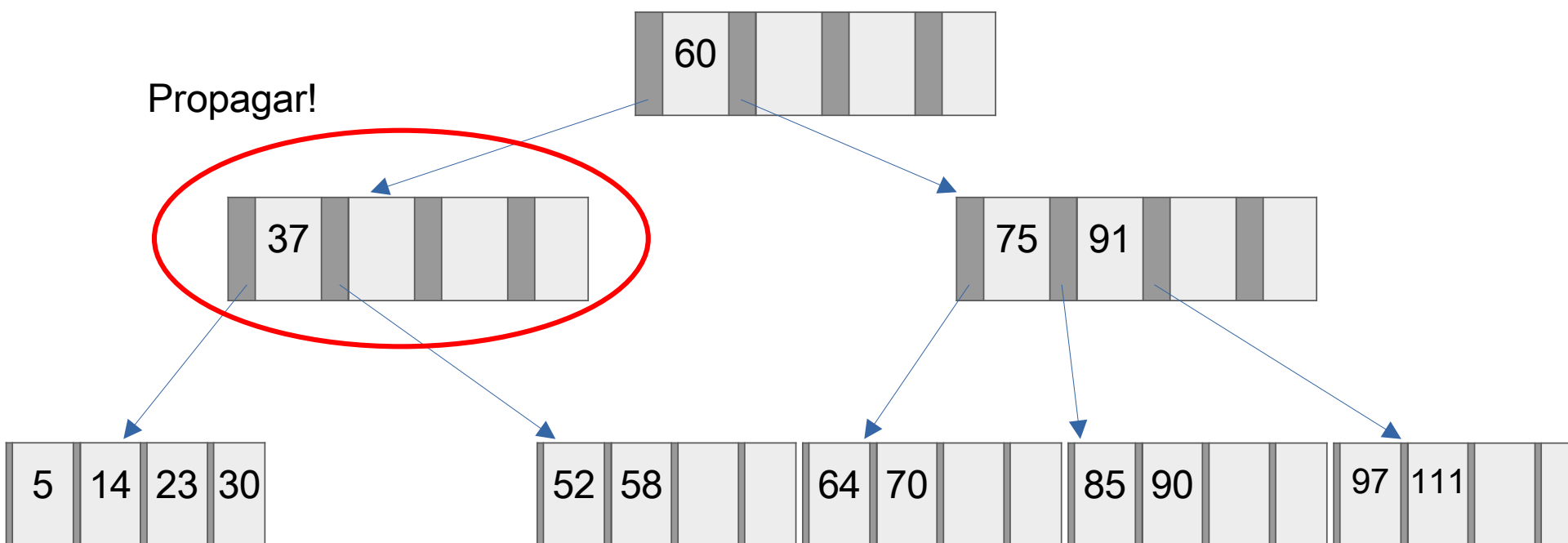


Caso 4: se folha ficar com menos de 50% de ocupação e as páginas irmãs não puderem ceder uma chave

- Faz-se a fusão com o irmão à esquerda ou o irmão à direita

Operações em árvores B

$M = 4 \rightarrow$ Remove 25

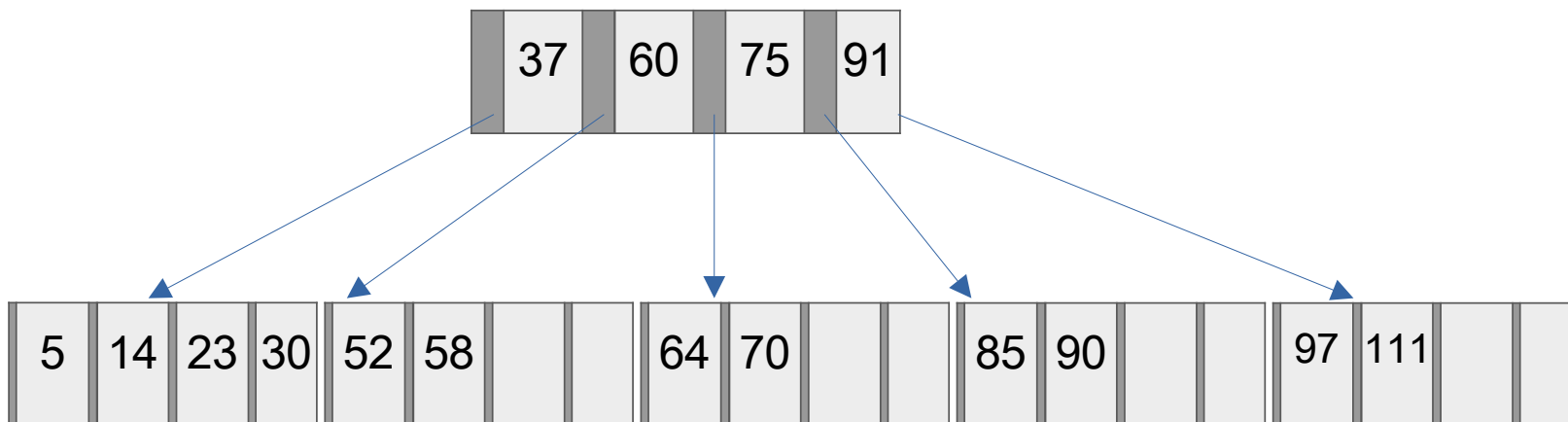


Caso 4: se folha ficar com menos de 50% de ocupação e as páginas irmãs não puderem ceder uma chave

- Faz-se a fusão com o irmão à esquerda ou o irmão à direita

Operações em árvores B

$M = 4 \rightarrow$ Remove 25



Caso 4 (novamente): se folha ficar com menos de 50% de ocupação e as páginas irmãs não puderem ceder uma chave

- *Faz-se a fusão com o irmão à esquerda ou o irmão à direita*

Exercícios

1. Mostre todas as árvores B válidas com grau mínimo 2 ($M=4$) que incluem as chaves 1, 2, 3, 4 e 5.
2. Mostre o resultado da inserção (nesta ordem) das chaves 16, 29, 27, 21, 13, 22, 18, 30, 32, 33, 23, 28, 24, 26, 11, 12, 34, 35, 14, 36, 15 em uma árvore B de grau 3 ($M=6$). Mostre a configuração da árvore após cada inserção e imediatamente antes de cada *split*.
3. Quantas leituras são feitas no pior caso para buscar um elemento em uma árvore B. Quantas leituras e escritas são necessárias para inserir um elemento, no pior caso?

pior caso de busca: quando o elemento não existe

pior caso de lei/escr: quando a árvore tem os nós “cheios” e precisa inserir, então vai ter uma recursividade para todos os nós país

Árvores B+

Árvore B+

- É uma variação da estrutura da árvore B
- Características e principais diferenças para as árvores B:
 - Todas as chaves são mantidas em folhas;
 - As chaves são repetidas em nós não folha que formam índices;
 - As folhas são ligadas oferecendo um caminho sequencial para percorrer as chaves;
 - Os nós folhas apontam para os registros em arquivo. Isso permite que os dados e os índices sejam armazenados em arquivos separados.
- Ordem da árvore
 - O número máximo de chaves representa ordem da árvore
 - Todo nó tem no máximo $M-1$ chaves e no mínimo $M/2$ (50%) de ocupação
 - Essa característica é válida para $M > 3$, já que se com $M=3$, quando dividir sempre ficará uma chave única em algum nó (diferente da árvore B que sempre perde um nó ao dividir).

Árvore B+

- Possui complexidade de tempo logarítmo
 - Acesso aleatório: $O(\log n)$
 - Inserção: $O(\log n)$
 - Remoção: $O(\log n)$
- Aumenta a eficiência da localização do próximo registro
 - Acesso sequencial é de $O(1)$ em vez de $O(\log n)$
- Não é necessário manter nenhum ponteiro de registro em nós não folha
 - Somente os nós folha apontam para os registros
 - Nós não folha só guardam o valor da chave de busca

Utilização da Árvore B+

- Banco de Dados
 - Muitos Bancos de Dados são construídos usando o mecanismo de Árvores B+: SQLServer, Oracle, PostgreSQL, MariaDB, SQLite,...
- Sistemas de Arquivos
 - Sistemas de arquivos como ReiserFS, XFS e JFS2 usam Árvores B+ para operações de leitura e escrita em blocos de disco.
- Sistemas de gerência de dados
 - Sistemas como CouchDB, Tokyo Cabinet, Tokyo Tyrant,....

Árvore B+

- A inserção de uma nova chave em uma árvore B+ é semelhante a inserção em uma árvore B:
 - Inicia localizando o nó folha que deverá ser adicionada a chave
 - Passos
 - Localizar a folha dentro da qual a chave deve ser inserida;
 - Localizar a posição de inserção dentro da folha;
 - Inserir a chave;
 - Se após a inserção a folha estiver completa, realizar a partição (divisão) da página
 - Para a divisão, as $M-1$ chaves são divididas em dois grupos:
 - As $M/2$ chaves menores ficam na folha esquerda;
 - As demais chaves ficam na folha da direita;
 - A menor chave da direita é **copiada** para o nó pai.

Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, 2, 6, 28, 10, 1$

Índices

Dados

Exemplo

$M = 4 \rightarrow$ **25**, 11, 3, 8, 14, 40, 20, 9, 2, 6, 28, 10, 1

25			

Exemplo

$M = 4 \rightarrow 25, \mathbf{11}, 3, 8, 14, 40, 20, 9, 2, 6, 28, 10, 1$

11	25		

Exemplo

$M = 4 \rightarrow 25, 11, \mathbf{3}, 8, 14, 40, 20, 9, 2, 6, 28, 10, 1$

3	11	25	

Exemplo

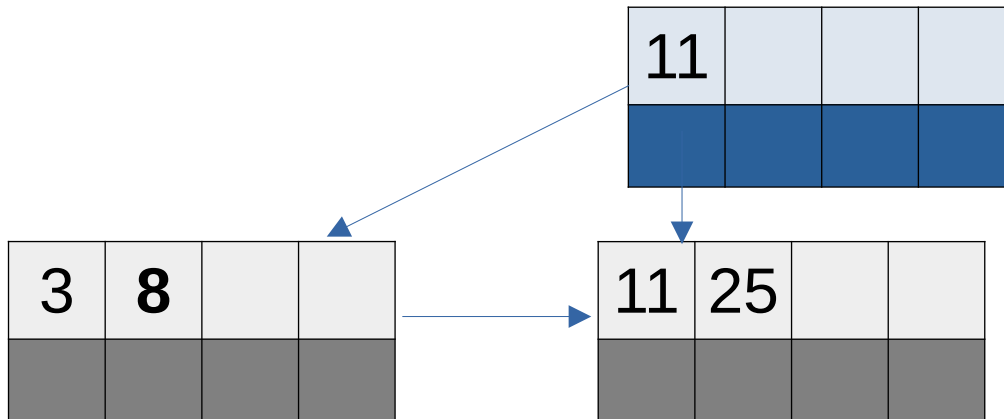
$M = 4 \rightarrow 25, 11, 3, \mathbf{8}, 14, 40, 20, 9, 2, 6, 28, 10, 1$

Split!

3	8	11	25

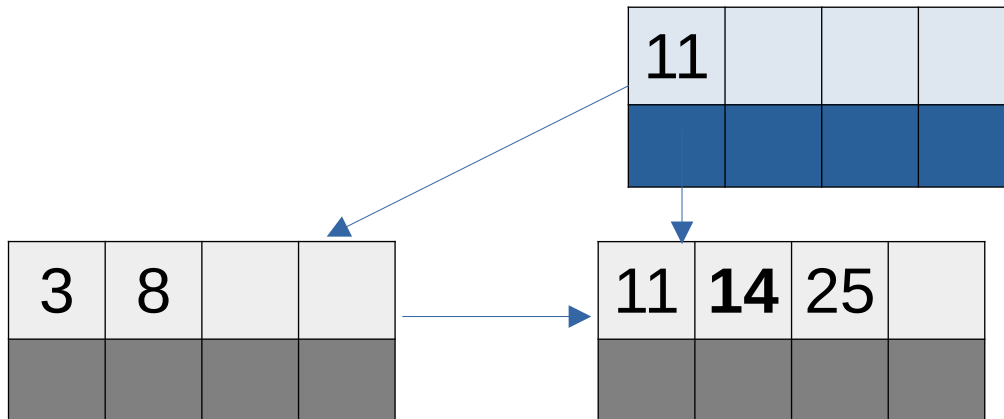
Exemplo

$M = 4 \rightarrow 25, 11, 3, \mathbf{8}, 14, 40, 20, 9, 2, 6, 28, 10, 1$



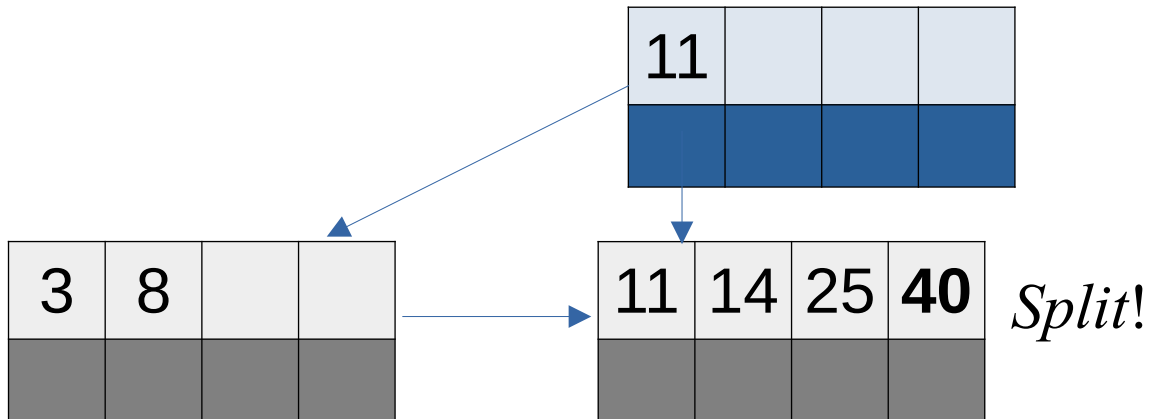
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, \mathbf{14}, 40, 20, 9, 2, 6, 28, 10, 1$



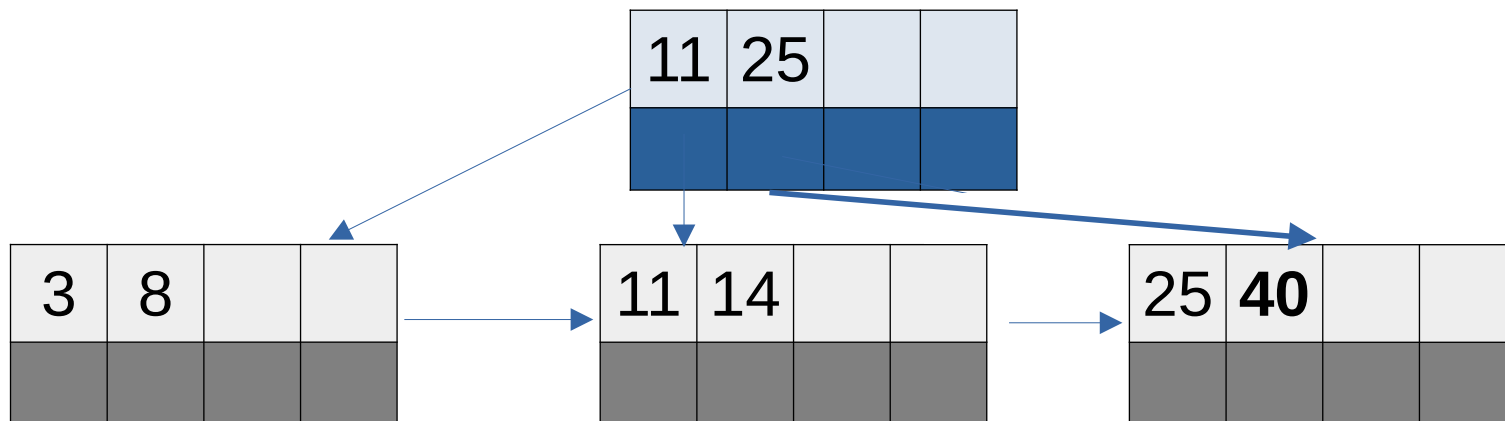
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, \mathbf{40}, 20, 9, 2, 6, 28, 10, 1$



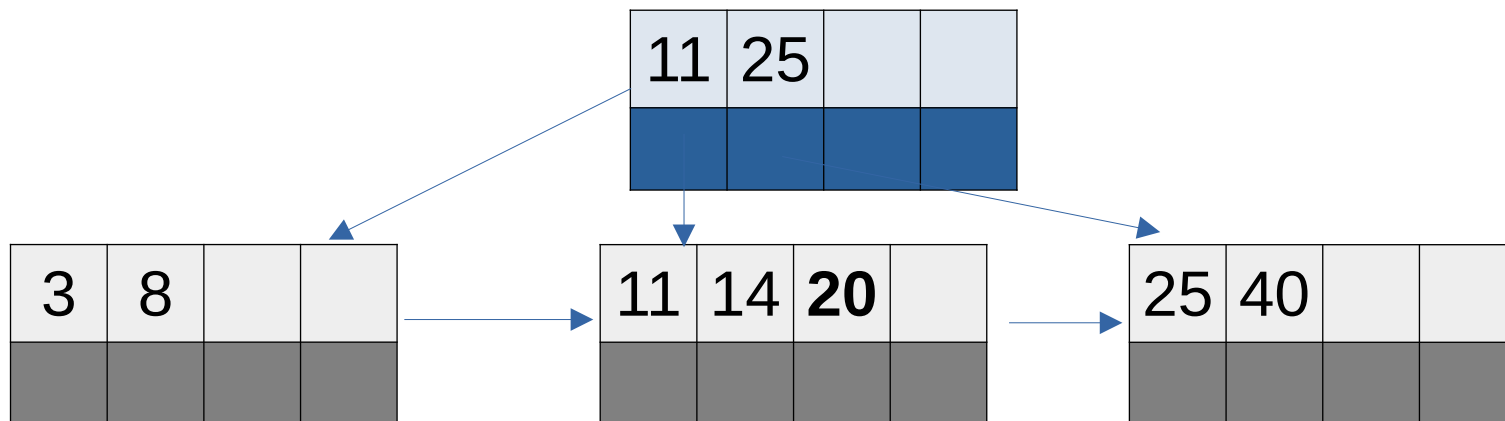
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, \mathbf{40}, 20, 9, 2, 6, 28, 10, 1$



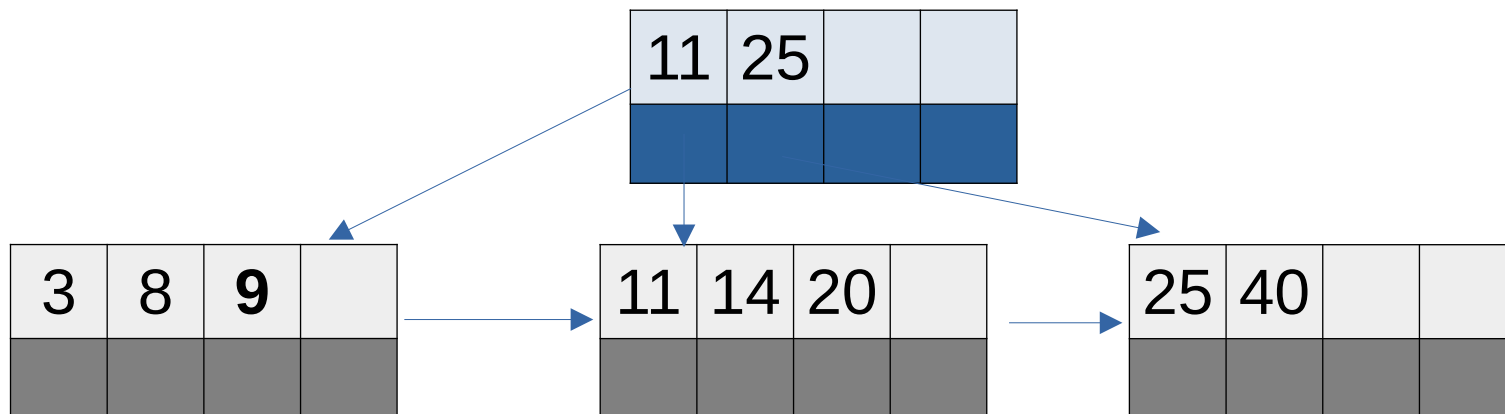
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, \mathbf{20}, 9, 2, 6, 28, 10, 1$



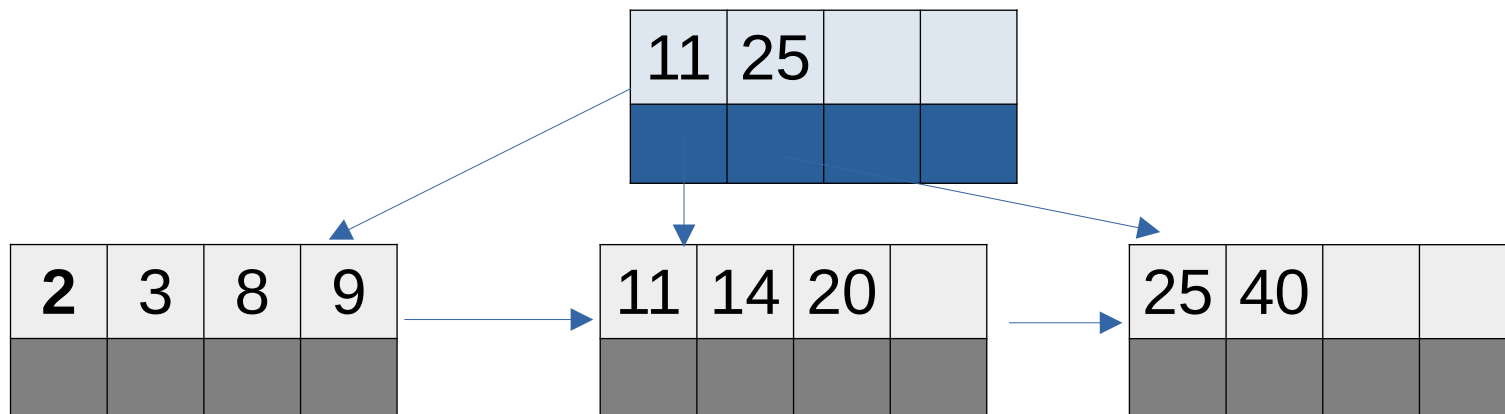
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, \mathbf{9}, 2, 6, 28, 10, 1$



Exemplo

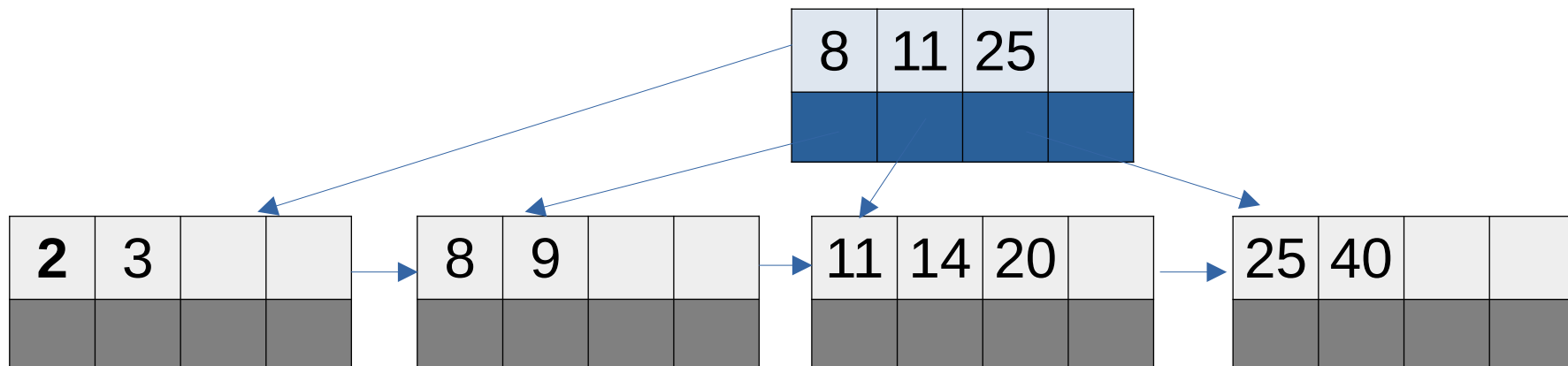
$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, \mathbf{2}, 6, 28, 10, 1$



Split!

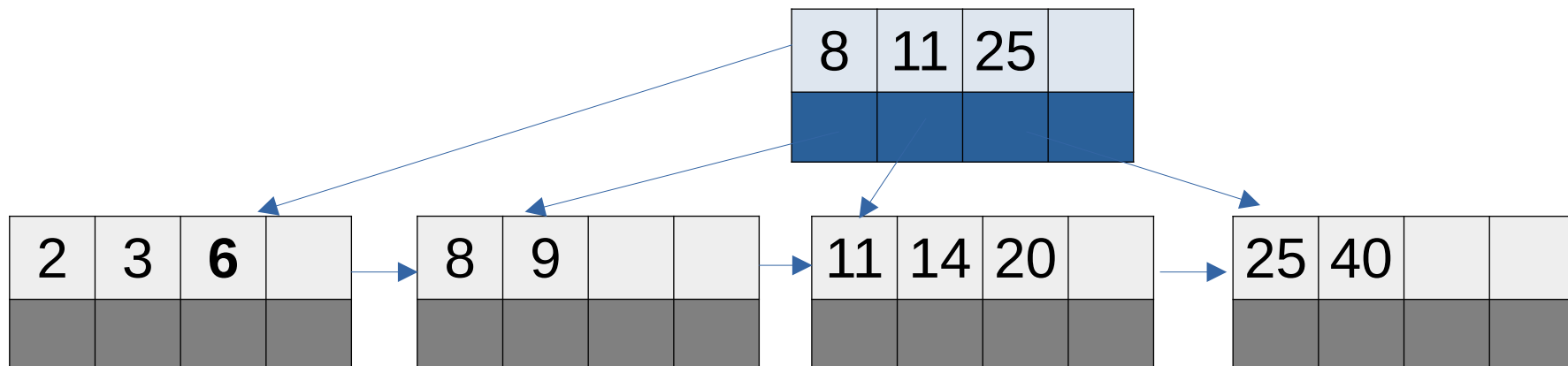
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, \mathbf{2}, 6, 28, 10, 1$



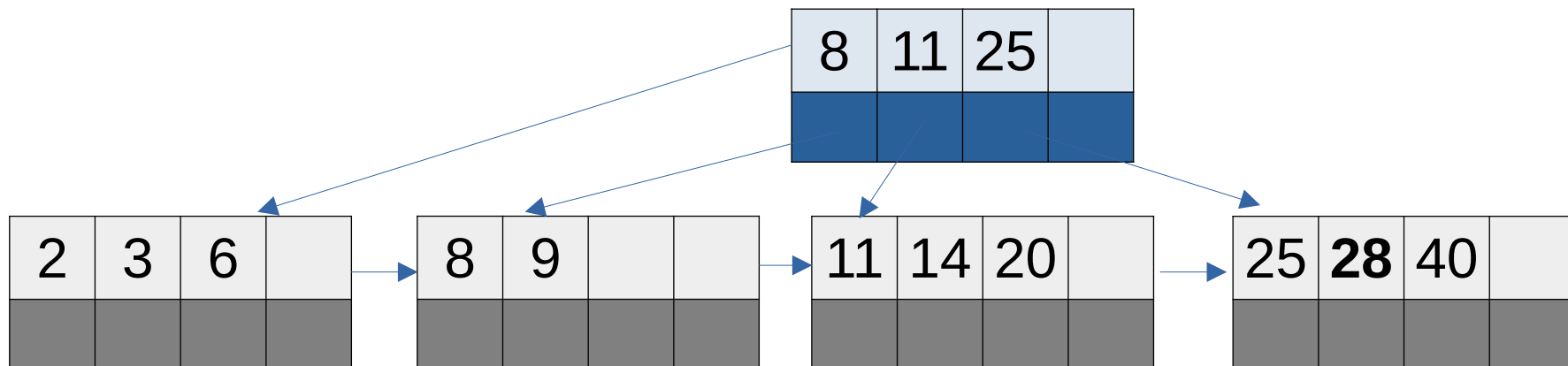
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, 2, \mathbf{6}, 28, 10, 1$



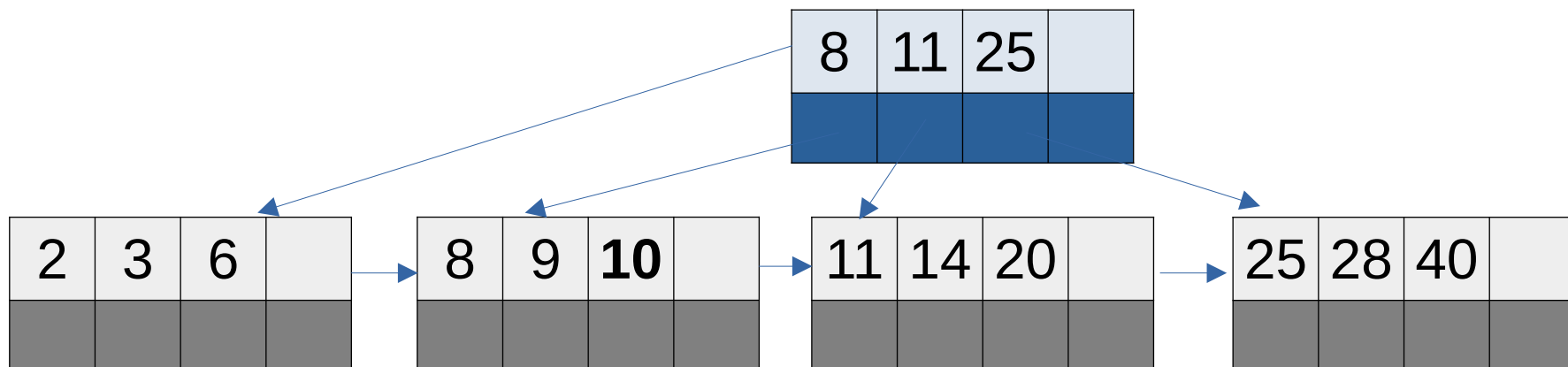
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, 2, 6, \mathbf{28}, 10, 1$



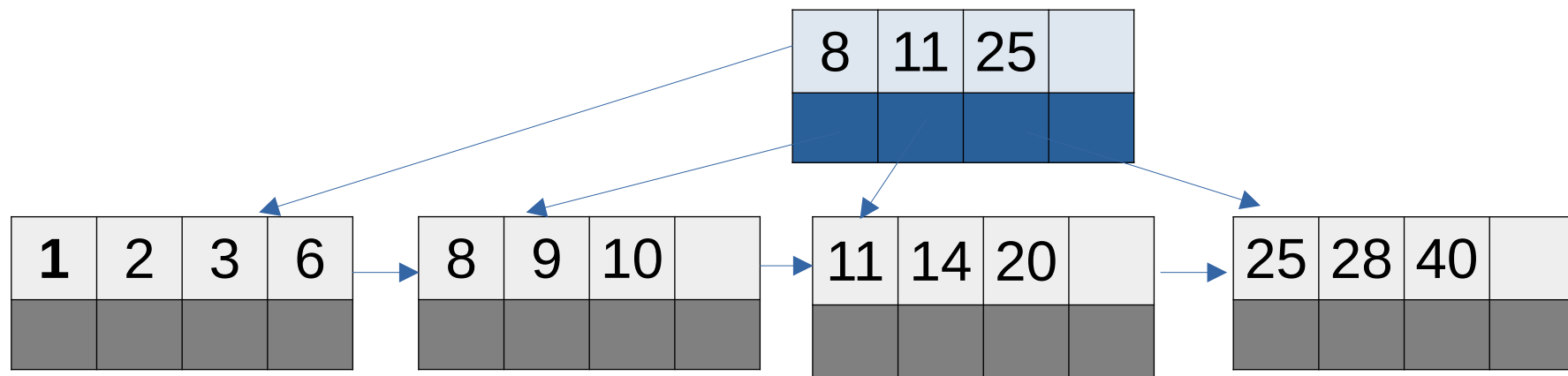
Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, 2, 6, 28, \mathbf{10}, 1$



Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, 2, 6, 28, 10, 1$

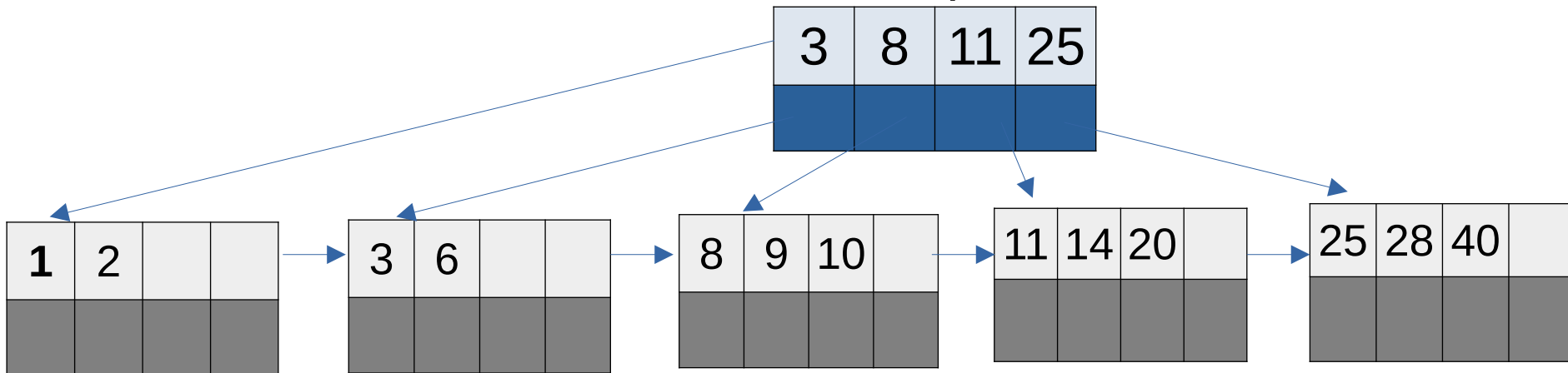


Split!

Exemplo

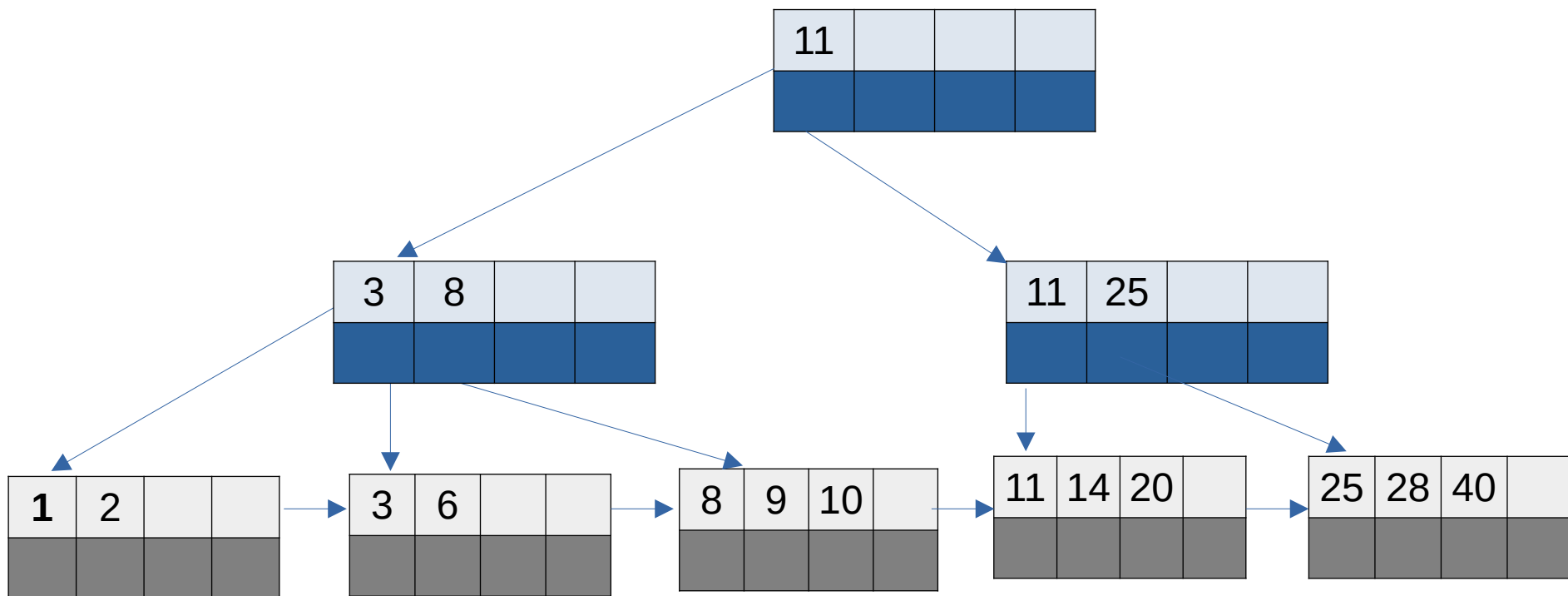
$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, 2, 6, 28, 10, 1$

Split!



Exemplo

$M = 4 \rightarrow 25, 11, 3, 8, 14, 40, 20, 9, 2, 6, 28, 10, 1$



Considerações sobre implementação de árvores B+

- Pode-se usar três arquivos:
 - Um para armazenar os metadados
 - Ponteiro para raiz da árvore
 - *Flag* indicando se a raiz é folha
 - Um arquivo para armazenar os índices (nós internos)
 - Um arquivo para armazenar os dados (folhas)

Considerações sobre implementação de árvores B+

- O **arquivo de índice** é estruturado em nós (blocos/páginas);
- Contém os nós internos da árvore;
- Cada nó possui:
 - Um inteiro m representando o número de chaves no nó;
 - Flag indicando se aponta para nó folha (dados) ou não (índices);
 - Ponteiro para nó pai (para facilitar implementação);
 - $P_0, (s_1, p_1), (s_2, p_2), \dots, (s_m, p_m)$, onde
 - s_x é uma chave
 - p_x é um ponteiro para uma página dentro do arquivo

Considerações sobre implementação de árvores B+

- O **arquivo de dados** é estruturado em nós (blocos/páginas);
- Contém os nós folhas da árvore;
- Cada nó possui:
 - Um inteiro m representando o número de registros no nó;
 - Ponteiro para nó pai (para facilitar implementação);
 - Ponteiro para a próxima página;
 - m registros.

Considerações sobre implementação de árvores B+

- Se um sistema de arquivo tem tamanho de bloco de B bytes e as chaves a serem armazenadas têm tamanho de k bytes, a árvore B+ mais eficiente é com ordem mínima (50% de M) de $d = (B / k) - 1$
- Exemplo prático:
 - Tamanho do bloco B em disco de 4096 bytes;
 - Tamanho da chave k de 4 bytes (inteiro, por exemplo);
 - $d = (4096 / 4) - 1 = 2023$
- Qual o tamanho da ordem máxima nessa situação?
 - $M = 2 * d = 2046$

Considerações sobre implementação de árvores B+

- As funções de inclusão e remoção devem garantir que as ordem máximas ($M-1$) e mínimas ($M/2$) da árvore sejam respeitadas (para $M > 3$);
- Em algumas implementações os nós folhas podem possuir M elementos (*overflow* só ocorre com valor maior que M). Alguns autores consideram que os nós folha (dados) possam ter tamanho distinto dos nós de índice;
- Nós de índice também podem ser representados como nos slides sobre árvores B (diferente dos nós folha):



Considerações sobre implementação de árvores B+

- Podem existir variações de árvores B+, como no caso do sistema de arquivos BTRFS (*B-Trees File System*):
 - **Sem Encadeamento de Folhas (*No Leaf Chaining*)**: As árvores B+ padrão conectam os nós folha entre si em uma lista encadeada para viabilizar varreduras sequenciais rápidas. O Btrfs remove esses vínculos por serem incompatíveis com o CoW (Copy-on-Write). Se as folhas fossem vinculadas, a atualização de uma única folha exigiria a atualização de seus vizinhos, resultando em uma enorme cascata de escritas.
 - **Itens de Comprimento Variável (*Variable-Length Items*)**: Os nós folha no Btrfs utilizam um sistema de "slots". Eles armazenam cabeçalhos de itens de tamanho fixo no início do nó e dados de comprimento variável (como pequenos arquivos inline ou nomes de arquivos) no final, fazendo com que as duas regiões cresçam uma em direção à outra dentro do mesmo bloco.

Exemplo de árvores B+: BTRFS

- Vantagens do BTRFS (em relação à ext4):
 - **Copy-on-Write (CoW):** Modificações são gravadas em novos blocos, garantindo a integridade dos dados e facilitando snapshots.
 - **Snapshots:** Cria "fotos" instantâneas do estado do sistema. Como utiliza CoW, esses snapshots ocupam quase nenhum espaço inicial.
 - **Subvolumes:** funcionam como partições virtuais que compartilham o mesmo espaço livre do disco.
 - **Autocura (Scrub):** Detecta e repara corrupção de dados automaticamente usando *checksums*.
 - **Compressão:** Suporta compressão transparente (zlib, lzo, zstd), economizando espaço em disco e aumentando a vida útil de SSDs.
 - **Desduplicação:** economiza espaço em disco ao identificar dados idênticos e armazenar apenas uma cópia física deles.
 - **RAID Integrado:** Gerencia múltiplos discos em RAID 0, 1 e 10 sem precisar de controladores de hardware ou softwares externos.